



kubernetes



목차

쿠버네티스 101



목차

- 쿠버네티스 소개
- 설치준비
- 쿠버네티스 도구
- 쿠버네티스 아키텍처
- 시작하기 전에
 - 프로젝트 애플리케이션
 - 쿠버네티스 인프라 소프트웨어
 - 쿠버네티스 스토리지



목차

- 기본 명령어
- 고급 명령어
- 쿠버네티스 기능 확장
- 테크톤



과정개요

쿠버네티스



소개

이 교육은 다음과 같은 목표를 가지고 있음.

1. kubeadm기반으로 controller, compute에 대한 bootstrap 명령어 및 **ClusterConfiguration**, **InitConfiguration**에 대한 학습
2. 기본적인 쿠버네티스 명령어 활용
3. 쿠버네티스 기반으로 서비스 구성 및 배포를 위한 방법
4. 실제 프로덕트 환경에서 구축 시 고려할 부분
5. 커널 5/6버전에 따른 쿠버네티스 설치

위와 같은 내용들은 교육에서 다룬다.



소개

01

다만 교육 진행 시, 다음과 같은 조건이 필요하다.

- Docker 혹은 Podman과 같은 고수준 컨테이너 경험
- OCI 표준도구 Buildah, Podman, Skopeo와 같은 도구 사용 경험이 있는 사용자
- 리눅스 커널 / 네트워크 / 스토리지 및 관리도구 사용 경험이 있는 사용자

02

오픈소스 버전으로 쿠버네티스 설치 및 운영 정보 및 기술이 필요한 사용자

03

쿠버네티스와 인공지능 활용 방법에 대한 기술이 필요한 엔지니어



과정개요

현재 CNCF 및 컨테이너 기반 플랫폼의 표준으로 자리 잡은 쿠버네티스(Kubernetes)에 대해 학습하는 과정. 본 교육에서는 쿠버네티스 설치, 구성, 운영을 다루며, OCI 도구와 쿠버네티스의 관계도 함께 살펴본다.

과정은 3일/5일에 따라서 구성이 달라진다. 내용은 다음과 같이 차이가 있다.

길이	설명
3일	설치 및 기본적인 쿠버네티스 명령어 학습
4일	쿠버네티스 확장 기능(보안/네트워크/개발)
5일	3/4일 과정에+DevOPS



강사

강사



강사

이름: 최국현

메일: tang@dustbox.kr, 회신은 다른 메일 주소로 드리고 있습니다. :)

사이트: tang.dustbox.kr

언제든지 질문 및 요청 환영 입니다.



소개

쿠버네티스 역사 및 개념



쿠버네티스

쿠버네티스 본래 구글에서 내부 서비스 관리 용도로 사용함. 구글에서 **Borg**라는 이름으로 2014년도에 릴리즈 하였음. 구글에서는 대략 2006년도에서 내부적으로 서비스 및 컨테이너 관리 방법에 대해서 논의가 됨.

구글은 2014년도에 깃-허브에 처음 코드를 릴리즈 하였으며, 현재 기준 11년 동안 꾸준히 릴리즈가 됨.

현재 CNCF에서는 **Serverless** 및 **Knative**를 구현하기 위해서 쿠버네티스를 사용하고 있으며, 현재는 가상머신도 같이 컨테이너와 동일하게 POD기반으로 관리한다.

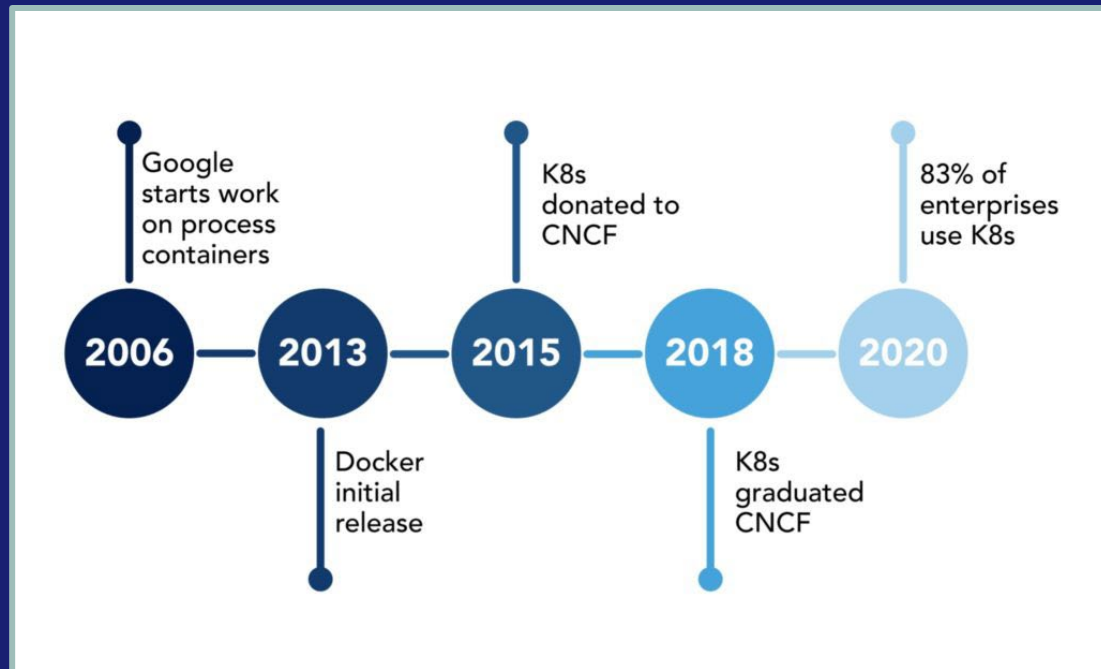
오픈스택은 쿠버네티스를 API기반으로 지원 및 자원 연동을 할 수 있도록 지원하고 있으며, 현재는 STARINGX를 OpenInfra를 통해서 지원하고 있다.



쿠버네티스

쿠버네티스는 구글에서 **Borg**라는 이름으로, 내부적으로 클러스터 관리 및 배포 용도로 사용하였다.

초기 **Borg**는 C++로 작성이 되었으나, 쿠버네티스로 넘어 오면서 Go언어 기반으로 작성이 되었다. 초기 릴리즈는 2015년도, 발표는 2014년도에 이루어 졌다.



가상화/컨테이너

쿠버네티스는 자체적으로 런타임을 지원하지 않는다. 여기서 말하는 런타임(runtime) 컨테이너 이미지에 있는 내용을 메모리에 불러와서 프로세서를 격리 및 추적하여, 호스트와 분리 운영이 가능하도록 환경을 구성을 한다.

이러한 이유 때문에 가상머신과 많은 혼동이 있는데, 둘은 엄연히 큰 차이가 있다. 가상머신은 링 구조(ring structure)를 생성 및 재구성하기 때문에, 더 복잡하고 독립적인 자원을 가지고 동작한다.

가상머신은 런타임에서 동작하는 구조가 아닌, 일반적으로 하이퍼바이저(hypervisor)나 혹은 가속기(kernel module level)에서 동작한다.

컨테이너는 이와 반대로 가상화 같은 하드웨어 수준의 기술이 필요하지 않으며, 커널 수준에서 격리 및 추적 기술 기반으로 프로세스를 관리한다.

이러한 이유로, 가상머신에서 사용하던 기능을 전부 컨테이너로 마이그레이션이 가능하지 않기 때문에, 서비스 아키텍트 및 구성에 따라서 플랫폼을 선택 및 구성하면 된다.



자원 구성

쿠버네티스가 사용하는 물리적 자원은 다음과 같다.

1. Controller(AKA Master)
2. Compute(AKA minion/worker)
3. Infra Node(Optional)

위의 두 개 용어, master, minion, worker는 오랫동안 IT계열에서 많이 사용하였지만, 차별적인 단어로 현재는 취급이 되고 있어서 통상적으로 controller, compute와 같은 단어로 대체가 되어가고 있다.



자원 구성

쿠버네티스는 오래전 버전은 모든 서비스가 hosted형태로 구성이 되어 있었지만, 현재 쿠버네티스는 kubelet 서비스 제외한 나머지 서비스는 containerized가 되었다. 이 말은 kubelet은 Proxy/Bootstrap/Monitor를 hosted상태에서 담당하고, 나머지는 서비스는 Static-POD가 되었다.

Static-POD는 우리가 알고 있는 POD와 동일하지만, 쿠버네티스에서 구성 시 기본적으로 생성하는 POD서비스 이다. Static-POD는 다음과 같다.

- kube-scheduler
- kube-apiserver
- kube-ControllerManager
- CoreDNS
- ETCD



커널 버전에 따른 제한

현재 쿠버네티스는 커널 버전에 따라서 설치 및 기능 제한이 발생하고 있다. 다음과 같이 버전별로 설치가 되는 커널이 다르다.

쿠버네티스 1.32 이하:

- 커널 5버전까지 설치 및 사용 가능

쿠버네티스 1.33 이상

- 커널 6버전부터 사용이 가능
- 1.33은 커널 5에서 설치가 가능하나, 다음과 같은 기능제한이 발생.
- NOSWAP관련 제한
 - kernel 6.3 미만 환경에서는 kubelet이 swap 상태를 엄격히 enforce하지 않고 보조적으로 검사하며 경고 로그를 출력
- User Namespace 관련 제한
 - namespace 관련 보안 기능이 제한되며 관련 경고 로그가 발생



쿠버네티스 릴리즈 형태

Kubernetes 프로젝트는 가장 최신의 세 가지 마이너 릴리스(1.32, 1.31, 1.30)에 대해 릴리스 브랜치를 유지 관리한다. Kubernetes 1.19 이상 버전은 약 1년간 패치 지원을 받으며, Kubernetes 1.18 및 그 이전 버전은 약 9개월간 패치 지원을 받았다.

Kubernetes 버전은 x.y.z 형식으로 표기되며, 여기서 x는 메이저 버전, y는 마이너 버전, z는 패치 버전을 의미한다. 이 표기 방식은 시맨틱 버저닝(Semantic Versioning) 규칙을 따른다.

자세한 내용은 <https://kubernetes.io/releases/>에서 확인이 가능하다.



기업용 쿠버네티스

쿠버네티스



쿠버네티스(OCP/SUSE Rancher)

엔터프라이즈 쿠버네티스 플랫폼에서 많이 사용하는 제품은 다음과 같다.

1. OpenShift Platform
2. SUSE Rancher

둘 다 기반은 쿠버네티스 바닐라 코드 기반으로 하고 있으며, 사용자가 좀 더 사용하기 쉽도록 관리 명령어 및 대시보드와 같은 편의 도구가 강화가 되어 있다. 또한, 커뮤니티에서 많이 사용하는 기능을 플랫폼과 통합하여, 클러스터 업그레이드 및 유지보수 시, 더 편하게 운영 및 운용이 가능하도록 패키지 구성이 되어 있다.



기업용 쿠버네티스 비교

이름	설명	프로덕트
Vanila Kubernetes	순수 쿠버네티스 버전. 기본적인 기능만 있으며, Federation API를 제공하기 때문에, 클러스터 연합 도구를 통해서 다중 클러스터 운영 기능	가능하나 어려움
Redhat OpenShift(Okd)	큰 차이점은 오퍼레이터를 통해서 모든 자원이 구성이 된다. 기존에 사용하던 쿠버네티스 표준 자원도 그대로 활용 및 적용이 가능하다. 다만, 다시 쿠버네티스로 윈백(WINBACK)은 거의 불가능하다.	가능, 종속적
SuSE Rancher	수세 랜처는 바닐라 쿠버네티스 및 통합된 쿠버네티스 클러스터도 같이 제공한다. 기반은 K3S으로 구성이 되어있다. 최대 장점은 여러 클러스터를 다중 구성 및 운영이 손쉽고, 기존에 사용하던 쿠버네티스 클러스터를 그대로 Rancher에 통합하여 운영 및 사용이 가능하다.	가능, 확장적

계속



기업용 제품 제품비교

기능	RKE2	OpenShift(okd)	Kubernetes
설치 편의성	쉬움(스크립트 기반)	어려움 (앤서블/테라폼 혼용)	중간 (kubeadm/YAML)
보안	기본적으로 CIS 하드닝 적용	SELinux, SCC(보안 컨텍스트) 적용	운영체제에 따라서 다름
네트워크	CNI 플러그인 선택 가능(Flannel, Calico 등)	OpenShift SDN (OVN, Multus)	표준 CNI플러그인 전부 지원
서비스 메쉬	Istio, Linkerd 사용 가능	기본 Istio 내장 (Service Mesh Operator)	표준 CNCF사용 가능
스토리지	CSI 지원		



기업용 제품 제품비교

기능	RKE2	OpenShift(okd)	Kubernetes
운영 관리	Rancher UI와 통합	OpenShift 콘솔 제공	kubectl 및 대시보드
업데이트 관리	Rancher UI를 통한 자동화	레드햇 가이드 기반으로 업데이트	수동 업데이트 필요
멀티클러스터	Rancher 기반 멀티클러스터 지원	OpenShift Cluster Manager 제공	Kubernetes Federation V1/V2지원



성능 및 관리 기능 비교

항목	RKE2	OpenShift(Okd)	Kubernetes
오버헤드	낮음 (경량화)	높음	보통
컨테이너 실행 속도	빠름 (Containerd 사용)	빠름 (CRI-O 사용)	빠름 (Containerd, CRI)
네트워크 성능	CNI에 따라 다름	OpenShift SDN, 느림	CNI에 따라 다름
노드 확장성	고성능	엔터프라이즈 환경에서 강력	확장 가능하지만 구성 필요
업데이트 성능	빠른 롤링 업데이트	안정적이지만 복잡함	노드 단위 업데이트



쿠버네티스 설치

간단한 설명

kubeadm

HELM



설치 방법 설명

쿠버네티스 설치하는 기본적으로 kubeadm명령어를 통해서 bootstrap를 수행한다. 이 방법에는 두 가지 방식을 지원한다.

1. With CLI Options

2. With Configuration File for Cluster/Kubelet/NodeJoin(Controller/Compute)

1번 방식은 설치 과정은 단순하지만, 클러스터에 자세한 옵션 설정 혹은 자동화 기반으로 배포는 어렵기 때문에 권장하지 않는다. 이 방식은 이미 널리 공유가 된 방법이기 때문에, 이 교육에서는 사용하지 않는다.

2번 방식, 현재 바닐라 버전으로 프로젝트 모델로 설치 및 유지보수를 위해서 선언기반으로 설치를 권장한다. 이 교육에서는 2번 방식으로 진행한다.



설치 전 네트워크 준비

구성을 하기 위해서 두 가지 조건이 필요하다.

1. kubelet이 API를 받기 위한 네트워크
2. POD가 사용 및 네트워크 구성을 위한 VXLAN/Geneve 용도 네트워크 인터페이스

실제로는 한 개의 네트워크만 있어도 상관 없지만, 최소 조건으로 높은 학습을 하기 위해서 분리 구성을 권장한다. 실무적으로 구성하는 경우, 용도에 따라서 네트워크는 좀 더 많은 개수로 분리가 되기도 한다.

가상머신으로 설치하는 경우, vxlan 기반으로 사용하는 걸 권장한다.



설치 전 런타임

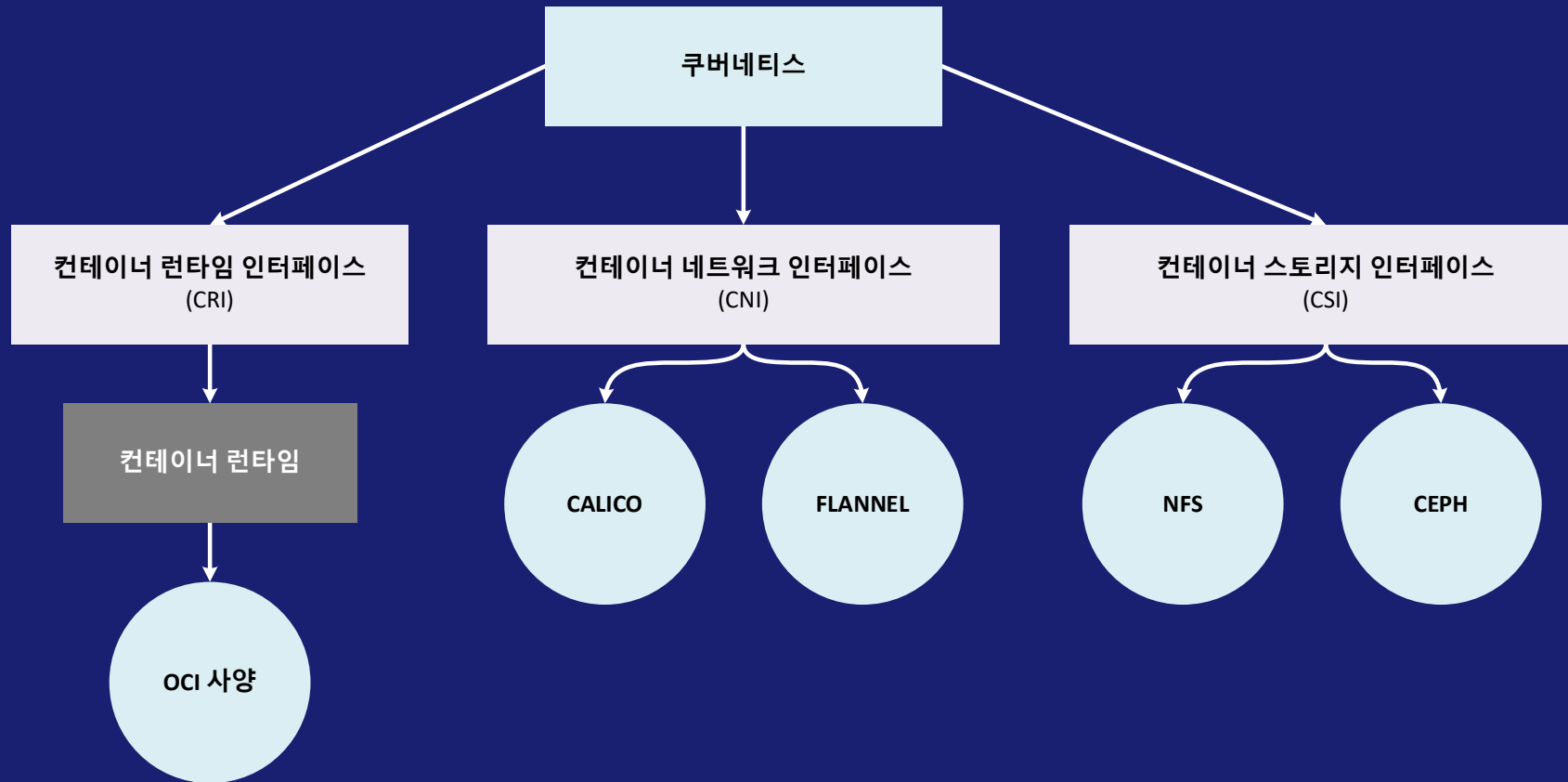
Kubernetes에서 containerd와 CRI-O는 기본 컨테이너 런타임으로 사용되며, gVisor와 Kata Containers는 RuntimeClass를 통해 워크로드 단위로 선택적으로 적용 가능한 보안 런타임이다.

쿠버네티스에서 제공하는 런타임은 다음과 같다.

1. cri-o(랩에서 사용)
2. containerd(기본값)
3. gVisor(runtime class)
4. Kata Containers(runtime class)
5. cri-docker(Mirantis Container)



쿠버네티스 런타임 및 네트워크 구성도



AIO

통합모드



통합모드(control+compute)

상황에 따라서 AIO(All In One) 구성으로 강의 진행이 되는 경우가 있다. 강사의 가이드에 따라서 아래와 같이 실행한다.

```
curl -fsSL http://10.0.0.250/playbooks/k8s_aio/aio_install.sh | sudo bash -  
s -- \  
  --cni flannel \  
  --untaint-control \  
  --enable-local-nfs \  
  --enable-nfs-sc \  
  --storage-test \  
  --local-registry 10.0.0.250:5000
```



일반설치

프로덕트 모드



프로덕트 모드(control/compute)

프로덕트 모드는 일반적으로 현업에서 많이 설치하는 구조. 다만, 수동으로 설치하는데 시간이 오래 걸리기 때문에 다음처럼 스크립트+플레이북 기반으로 설치를 진행한다. 아래처럼 쉘 스크립트로 설치가 가능하다.

```
curl -fsSL http://10.0.0.250/playbooks/k8s_prod/run.sh |\  
sudo bash -s -- \  
  --nfs-server 10.0.2.164 \  
  --cni flannel \  
  --haproxy 10.0.2.164 \  
  --control 10.0.4.171,10.0.4.53,10.0.4.218 \  
  --compute 10.0.2.63,10.0.3.9 \  
  --nm-iface ens3 --nm-conn "System ens3" \  
  --user rocky \  
  --password rocky
```



프로덕트 초기화

프로덕트 모드는 일반적으로 현업에서 많이 설치하는 구조. 다만, 수동으로 설치하는데 시간이 오래 걸리기 때문에 다음처럼 스크립트+플레이북 기반으로 설치를 진행한다. 아래처럼 쉘 스크립트로 설치가 가능하다.

```
curl -fsSL http://10.0.0.250/playbooks/k8s_prod/run.sh |\  
sudo bash -s -- \  
  --nfs-server 10.0.2.164 \  
  --cni flannel \  
  --haproxy 10.0.2.164 \  
  --control 10.0.4.171,10.0.4.53,10.0.4.218 \  
  --compute 10.0.2.63,10.0.3.9 \  
  --nm-iface ens3 --nm-conn "System ens3" \  
  --user rocky \  
  --password rocky --reset --yes
```



kubectl 사용 시 주의 사항

kubectl 명령어는 모든 노드에 설치가 되어있음. 랩 진행 시, HAProxy 에서 kubectl 명령어를 주로 사용하기 때문에 설치 후 HAProxy 노드에서 꼭 kubectl 명령어 실행이 올바르게 되는지 확인이 필요하다.

→ HAProxy Node with rocky user account

```
kubectl get nodes
```



패키지 관리

HELM

Chart Museum



Helm

바닐라 쿠버네티스는 기본적으로 패키징 및 패키징 배포에 대한 기능은 별도로 지원하지 않는다. 이와 비슷한 **kustomize**라는 기능이 있지만, 디렉터리 기반으로 자원 배포를 지원한다.

이러한 이유로, 버전 기반으로 관리가 어렵기 때문에, Helm 기반으로 패키징 및 배포버전 관리한다. 쿠버네티스에서 클러스터에 기능을 추가나 혹은 서비스를 배포하는 경우, 최소 HelmChart 기반으로 구성 및 배포를 권장하고 있다.

여기서 **HelmChart**를 생성 및 배포하는 방법에 대해서는 학습하지 않는다. 또한, 클러스터 소프트웨어는 Chart 기반으로 배포한다.



HelmChart MUSEUM

[차트 뮤지엄 사이트](#)



Helm

Helm에서 사용하는 용어는 다음과 같다.

이름	설명
helm	패키지 관리자 이름 및 명령어
HelmChart	Helm에서 제공하는 패키지
values.yaml	오버라이드(override)가 필요한 경우, 이 파일을 통해서 설정 파일을 오버라이드 한다.



Helm

Helm설치는 다음과 같다. 자동화 스크립트를 사용하는 경우 컨트롤 노드 1번에 설치가 되어있다.

```
dnf install git tar -y
curl -fsSL -o get_helm.sh
https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
mkdir ~/bin
sh get-helm-3
mv /usr/local/bin/helm ~/bin
helm list
```



네트워크

CALICO

FLANNEL



시작 전...

오픈스택 랩에서는 네트워크는 Flannel으로 설치가 되어 있습니다. 아래 내용은 강사와 함께 검토하면 진행 합니다.



네트워크 기능 설명

쿠버네티스에서 대표적으로 지원하는 네트워크 기능은 다음과 같다.

솔루션	기본 개념 및 특징	네트워크 방식	네트워크 정책 지원	사용 사례 및 장점
Calico	컨테이너 네트워킹과 네트워크 정책을 동시에 제공하는 솔루션	기본적으로 BGP 기반 라우팅, 필요 시 VXLAN, IP-in-IP 사용	강력한 정책 엔진 제공(오픈소스 버전은 제한적)	대규모 클러스터 및 고급 보안 요구 환경에 적합
Flannel	간단한 오버레이 네트워크 솔루션로, 클러스터 내부 Pod 간 통신을 지원	주로 VXLAN 또는 Host-GW 모드 사용	기본 기능만 제공	설정과 운영이 간단하여 작은 클러스터나 테스트 환경에 적합

계속



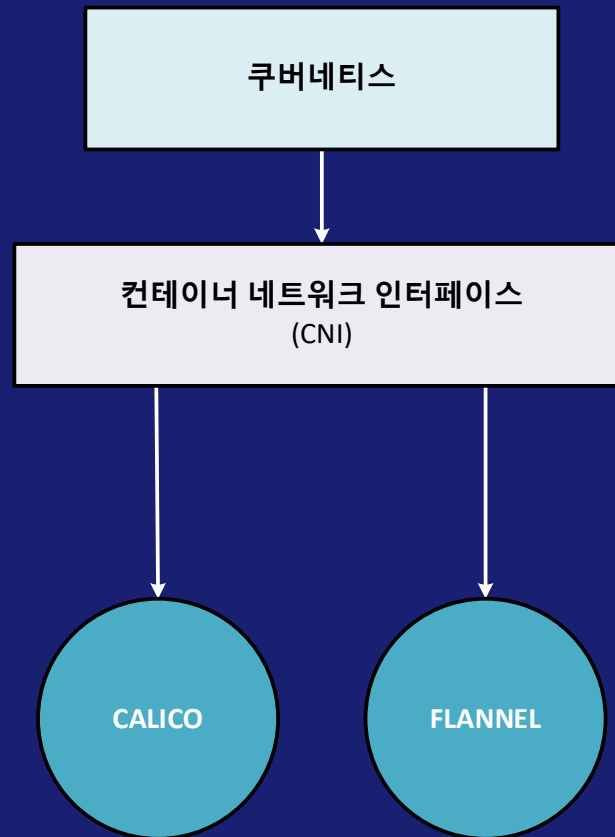
네트워크 기능 설명

위의 내용 계속 이어서...

솔루션	기본 개념 및 특징	네트워크 방식	네트워크 정책 지원	사용 사례 및 장점
Cilium	eBPF 기반으로 고성능 패킷 처리와 네트워크 정책, 보안을 제공하는 솔루션	eBPF를 사용하여 직접 패킷 처리, 오버레이 지원	세밀한 네트워크 정책 및 가시성 제공	클라우드 네이티브, 마이크로서비스 환경에서 고성능, 고가용성 제공
Canal	Flannel의 단순 오버레이와 Calico의 정책 기능을 결합한 통합 솔루션	Flannel 방식의 오버레이와 Calico 정책 적용	Calico 기반의 정책 엔진 활용	간단한 네트워크 구축과 정책 적용을 동시에 원하는 환경에 적합



CNI 동작 구조



/etc/kubernetes/manifests

/etc/cni/net.d/

container runtime
daemon



CNI 지원 기능

네트워크 CNI는 환경 및 구성에 따라서 적절하게 선택해서 사용한다. 위의 내용은 CNI에서 대표적인 내용이며, 추가적으로 더 확인이 필요한 경우 아래 사이트를 참고한다.

<https://landscape.cncf.io/>



CALICO 네트워크 시스템 설명

Calico는 기본적으로 L3 라우팅(BGP)을 중심으로 설계된 CNI이며, 필요에 따라 VXLAN 또는 IP-in-IP 기반 Overlay 모드를 선택적으로 사용할 수 있다. Calico는 Operator 기반으로 설치 및 관리되며, 구성 요소가 삭제되거나 장애가 발생한 경우 자동으로 재 배포 되는 Self-healing 구조를 제공한다.

Calico 사용 시 다음과 같은 고려 사항이 있다.

1. OpenStack와 같은 SDN(Neutron/OVS) 기반 환경에서도 사용은 가능하나, Overlay 네트워크, 보안 그룹, MTU, 라우팅 정책 간 충돌로 인해 네트워크 설계 난이도가 매우 높다.
2. Provider(Network) 기반 또는 L2/L3가 명확한 네트워크 환경에서는 상대적으로 제약이 적고 안정적인 구성이 가능하다.
3. BGP 사용이 제한되거나 IP 프로토콜 제약이 있는 환경에서는 Calico의 L3 라우팅 기반 성능 이점을 충분히 활용하기 어렵다.



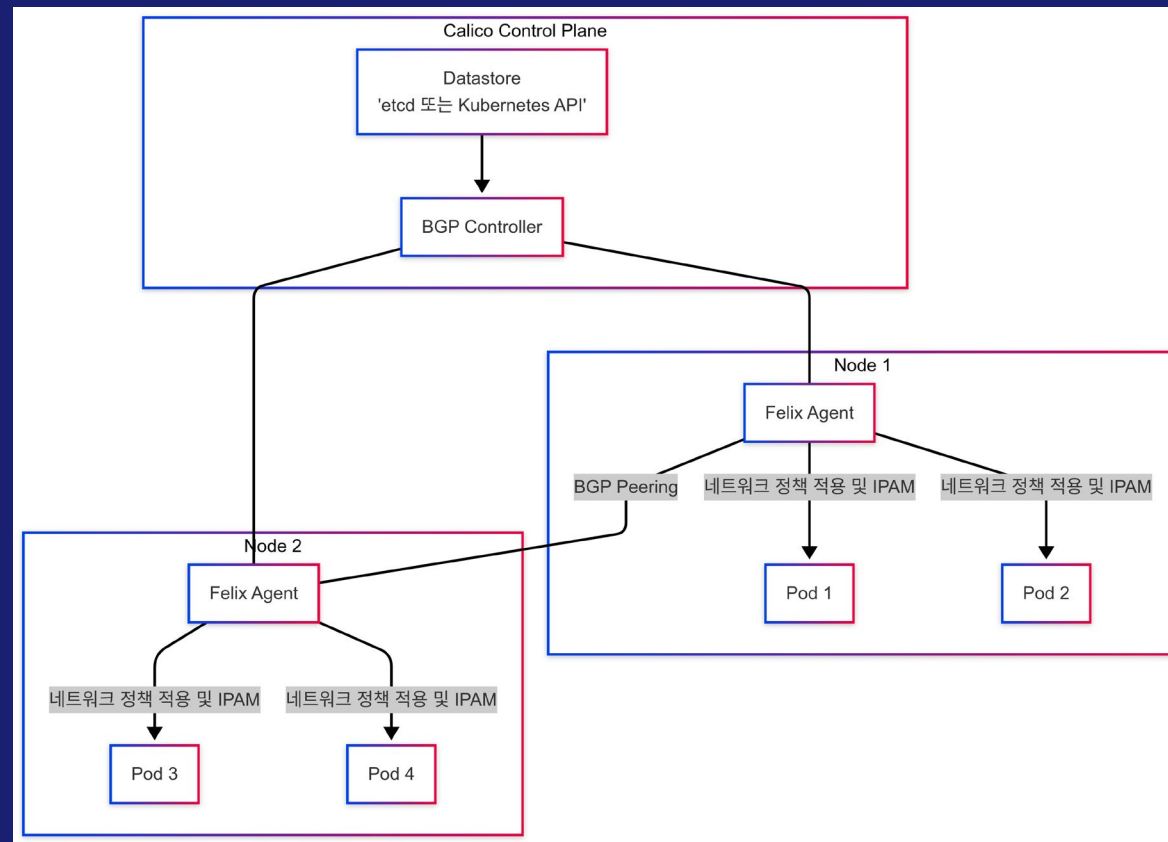
CALICO 네트워크 시스템 설명

특히 OpenStack 환경에서는 Calico의 라우팅 중심 설계와 Neutron의 Overlay 구조가 충돌하는 경우가 많아, Flannel, Cilium(VXLAN/eBPF), OVN-Kubernetes 등 다른 CNI를 고려하는 것이 일반적으로 더 안정적이다.

Calico는 Host 기반 L3 라우팅(ToR-like 구조)을 사용하므로, Overlay 중심 VXLAN 방식 대비 홑 수가 줄어들고 트래픽 처리 성능과 라우팅 효율이 우수한 편이다.



CALICO 구조



CALICO 설치

```
helm repo add projectcalico https://docs.tigera.io/calico/charts  
helm install calico projectcalico/tigera-operator --version v3.27.5 --  
namespace tigera-operator --create-namespace  
helm list --namespace tigera-operator  
kubectl taint node node1.example.com node-role.kubernetes.io/control-  
plane:NoSchedule-  
kubectl -n calico-system get pod -w
```



CALICO 설정

```
cat <<'EOF' | kubectl apply -f -  
---  
apiVersion: operator.tigera.io/v1  
kind: Installation  
aladata:  
  name: default  
spec:  
  calicoNetwork:  
    bgp: Enabled OpenStack에서 사용시 bgp: Disabled  
    hostPorts: Enabled 확장 기능을 사용하지 않으면 Disabled
```



CALICO 설정

```
ipPools:
```

```
- blockSize: 26
```

```
  cidr: 10.0.0.0/16
```

```
  encapsulation: IPIPCrossSubnet
```

```
  natOutgoing: Enabled
```

```
  nodeSelector: all()
```

```
registry: quay.io
```

```
EOF
```



FLANNEL 설명

쿠버네티스에서 가장 기본적으로 사용되는 VXLAN 기반 오버레이 네트워크 CNI 중 하나가 Flannel이다.

다수의 쿠버네티스 플랫폼은 높은 호환성과 단순성을 이유로 기본 네트워크 구성으로 Flannel을 채택한다.

Flannel은 추가적인 정책 기능이나 고급 네트워크 제어 기능을 제공하지 않기 때문에, 플랫폼이나 인프라 환경에 대한 제약 없이 바로 사용이 가능하다. 기본적으로 VXLAN 기반의 UDP 터널링 방식으로 동작하며, 설치와 운영이 매우 단순한 것이 가장 큰 장점이다.



FLANNEL 설명

반면, 네트워크 정책(NetworkPolicy)이나 고급 라우팅 제어 기능이 제공되지 않아 확장성 및 기능적 유연성에는 한계가 있다.

Calico와 달리 BGP 기반 라우팅이나 Host-GW 방식과 같은 고성능·고급 네트워크 모델을 제공하지 않기 때문에, 보안 정책이나 트래픽 제어가 중요한 대규모 쿠버네티스 클러스터에는 적합하지 않다.

따라서 Flannel은 소규모 또는 중간 규모의 쿠버네티스 클러스터에서 빠른 구축과 안정적인 운영을 목적으로 사용할 경우 적합한 선택지이다.



FLANNEL 설치

```
helm repo add flannel https://flannel-io.github.io/flannel/  
helm install flannel --set podCidr="192.168.0.0/16" --namespace kube-  
flannel --create-namespace flannel/flannel --create-namespace  
kubectl label --overwrite ns kube-flannel \  
pod-security.kubernetes.io/enforce=privileged  
kubectl -n kube-flannel get pod
```



정리

위에서 명시한 기능을 정리하면 다음과 같다.

항목	VXLAN	Calico	Flannel
기본 개념	레이어2 오버레이 프로토콜로, L2 네트워크를 L3 환경에서 확장하기 위한 캡슐화 기술입니다.	컨테이너 네트워킹 및 네트워크 정책 솔루션으로, 기본적으로 BGP를 이용한 라우팅 방식이며 필요 시 VXLAN과 같은 오버레이 모드를 지원합니다.	Kubernetes 클러스터에서 간단하게 오버레이 네트워크를 구축하기 위한 솔루션으로, 주로 VXLAN 또는 Host-GW 모드를 사용합니다.
네트워크 방식	UDP를 기반으로 패킷을 캡슐화하여 오버레이 네트워크를 구성합니다.	기본적으로 라우팅(BGP)을 이용해 직접 연결을 구성하며, 오버레이(VXLAN) 방식도 선택적으로 사용합니다.	오버레이 네트워크 방식(VXLAN) 또는 단순 라우팅(Host-GW) 방식을 지원합니다.

계속



정리

위에서 명시한 기능을 정리하면 다음과 같다.

항목	VXLAN	Calico	Flannel
네트워크 정책 지원	단순 캡슐화 기술로 자체 네트워크 정책 기능은 제공하지 않습니다.	강력한 네트워크 정책 및 보안 기능을 내장하여 세밀한 트래픽 제어가 가능합니다.	기본적인 네트워크 구성을 제공하지만, 고급 네트워크 정책 지원은 제한적입니다.
구성 및 사용 사례	주로 오버레이 네트워크 구현의 기술적 기반으로 다른 솔루션에서 채택됩니다.	복잡한 환경이나 대규모 클러스터에서 정책 기반의 네트워킹을 구현할 때 많이 사용됩니다.	설정이 간단해 작은 규모의 클러스터나 기본 네트워크 구축 시 유용합니다.



쿠버네티스 대시보드

Kubernetes Dashboard

Headlamp



Kubernetes Dashboard vs Headlamp 개요

쿠버네티스 클러스터를 시각적으로 관리하기 위한 대표적인 웹 기반 UI 도구로는 Kubernetes Dashboard와 Headlamp가 있다.

Kubernetes Dashboard는 공식 프로젝트에서 제공하는 기본 웹 UI로, 쿠버네티스 리소스를 조회하고 간단한 작업을 수행할 수 있도록 설계된 관리 도구이다. 클러스터 상태 확인, Pod 로그 확인, Deployment 스케일링 등 기본적인 관리 기능을 제공하며 주로 **교육 환경**이나 **간단한 운영 환경**에서 활용된다.

반면 Headlamp는 현대적인 UI와 확장 기능을 제공하는 쿠버네티스 관리 도구로, 복잡한 클러스터 환경에서도 효율적인 운영을 지원하도록 설계되었다. kubeconfig 기반 인증, 멀티 클러스터 지원, CRD 자동 인식 등 다양한 기능을 제공하여 **운영 환경** 및 **DevOps 환경**에서 활용도가 높다.

최근에는 쿠버네티스 생태계에서 다양한 확장 리소스(CRD)가 증가하면서, 이러한 리소스를 자동으로 탐색하고 관리할 수 있는 Headlamp와 같은 차세대 UI 도구의 활용도가 점점 높아지고 있다.



운영 환경에서의 활용 비교

쿠버네티스 Dashboard는 기본적인 클러스터 관리 도구로 제공되지만, 운영 환경에서는 몇 가지 제한 사항이 존재한다.

첫 번째로 인증 방식이 토큰 기반이기 때문에 관리자가 매번 토큰을 생성해야 하는 불편함이 있다. 두 번째로 확장 리소스(CRD)에 대한 지원이 제한적이기 때문에 다양한 플랫폼 도구를 사용하는 환경에서는 관리 효율이 떨어질 수 있다.

반면 Headlamp는 kubeconfig 기반 인증을 사용하여 기존 kubectl 환경과 자연스럽게 연동되며, 여러 개의 클러스터를 동시에 관리할 수 있는 멀티 클러스터 기능을 제공한다. 또한 CRD 리소스를 자동으로 인식하여 UI에 표시하기 때문에 DevOps 및 플랫폼 환경에서 매우 유용하다.



운영 환경에서의 활용 비교

이러한 이유로 최근 쿠버네티스 운영 환경에서는 다음과 같은 형태의 관리 도구 구성이 많이 사용된다.

1. 기본 클러스터 관리: Headlamp
2. GitOps 관리: ArgoCD UI
3. 모니터링: Grafana / Prometheus
4. 플랫폼 관리: Rancher 또는 자체 관리 도구

따라서 교육 환경이나 간단한 실습 환경에서는 Kubernetes Dashboard가 충분하지만, 실제 운영 환경에서는 Headlamp와 같은 확장형 UI 도구를 사용하는 것이 더 효율적이다.



대시보드 기능비교

항목	Kubernetes Dashboard	Headlamp
개발 단체 혹은 회사	Kubernetes SIG UI	Kinvolk (현재 Microsoft)
UI 스타일	전통적 UI	현대적 UI
접근 방식	토큰 로그인 필요	kubeconfig 기반
설치 난이도	중간	매우 쉬움
멀티 클러스터	제한적	기본 지원
RBAC 표시	제한적	상세 표시
리소스 탐색	기본	훨씬 직관적
플러그인	없음	플러그인 지원
로컬 실행	불가	가능 (Desktop app)
운영 편의성	낮음	매우 좋음



대시보드 기능 지원 및 유지보수

Kubernetes Dashboard는 쿠버네티스 공식 프로젝트에 포함되어 있지만, 최근 몇 년 동안 기능 개발은 거의 이루어지지 않고 있으며 주로 버그 수정이나 보안 패치 중심의 유지보수 모드(Maintenance Mode)로 운영되고 있다.

이 때문에 실제 운영 환경에서는 Dashboard 대신 다른 UI 도구를 사용하는 경우가 증가하고 있다.

반면 Headlamp는 Microsoft에서 적극적으로 개발하고 있는 프로젝트로, 멀티 클러스터 관리와 확장 가능한 UI 구조를 목표로 지속적인 기능 업데이트가 이루어지고 있다.

항목	Kubernetes Dashboard	Headlamp
프로젝트 시작	2017	2021
유지보수 조직	Kubernetes SIG UI	Microsoft
현재 상태	유지보수 모드	적극 개발
기능 업데이트	매우 제한적	지속적 업데이트
Kubernetes 지원 버전	최신 버전 제한적 지원	최신 Kubernetes 적극 지원
장기 전망	점진적 축소 가능성	플랫폼 UI로 확장 중



대시보드 설치

쿠버네티스 대시보드는 다음과 같이 설치한다. 설치 시, 아래와 같이 Kong에 노드 포트 명시가 가능하다.

```
helm repo add kubernetes-dashboard https://kubernetes.github.io/dashboard/  
helm repo update  
helm search repo kubernetes-dashboard/kubernetes-dashboard
```

```
helm -n kubernetes-dashboard upgrade --install kubernetes-dashboard  
kubernetes-dashboard/kubernetes-dashboard \  
--set kong.proxy.service.type=NodePort \  
--set kong.proxy.service.nodePort=32443
```

**DEPRECATED DISABLED
CHART REPO!**



대시보드 설정

설정은 values.yaml 파일을 통해서 간단하게 구성이 가능하다.

```
cat <<EOF> values-dashboard.yaml
namespaceOverride: kubernetes-dashboard
ingress:
  enabled: false
kong:
  proxy:
    service:
      type: NodePort
      nodePort: 32443
resources: {}
EOF
```

**DEPRECATED DISABLED
CHART REPO!**



대시보드 설정

values.yaml 아래 명령어로 리비전 한다.

```
helm -n kubernetes-dashboard upgrade --install kubernetes-dashboard  
kubernetes-dashboard/kubernetes-dashboard \  
  -f values-dashboard.yaml  
helm -n kubernetes-dashboard get values kubernetes-dashboard
```

```
kubectl -n kubernetes-dashboard get svc kubernetes-dashboard-kong-proxy
```

**DEPRECATED DISABLED
CHART REPO!**



설치를 원하는 경우...

아래 명령어로 설치가 가능하다. 하지만, 구 버전이니 가급적이면 사용하지 않는 걸 권장한다.

```
kubectl apply -f https://raw.githubusercontent.com/cncf/devstats-  
kubernetes-dashboard/refs/heads/master/manifests/kubernetes-dashboard.yaml
```



로그인용 ServiceAccount 1

```
cat <<'EOF' | kubectl apply -f -
apiVersion: v1
kind: ServiceAccount
metadata:
  name: dashboard-admin
  namespace: kube-system
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: dashboard-admin
```



로그인용 ServiceAccount 2

```
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: cluster-admin
subjects:
- kind: ServiceAccount
  name: dashboard-admin
  namespace: kube-system
EOF
```



올바르게 동작하지 않는 경우

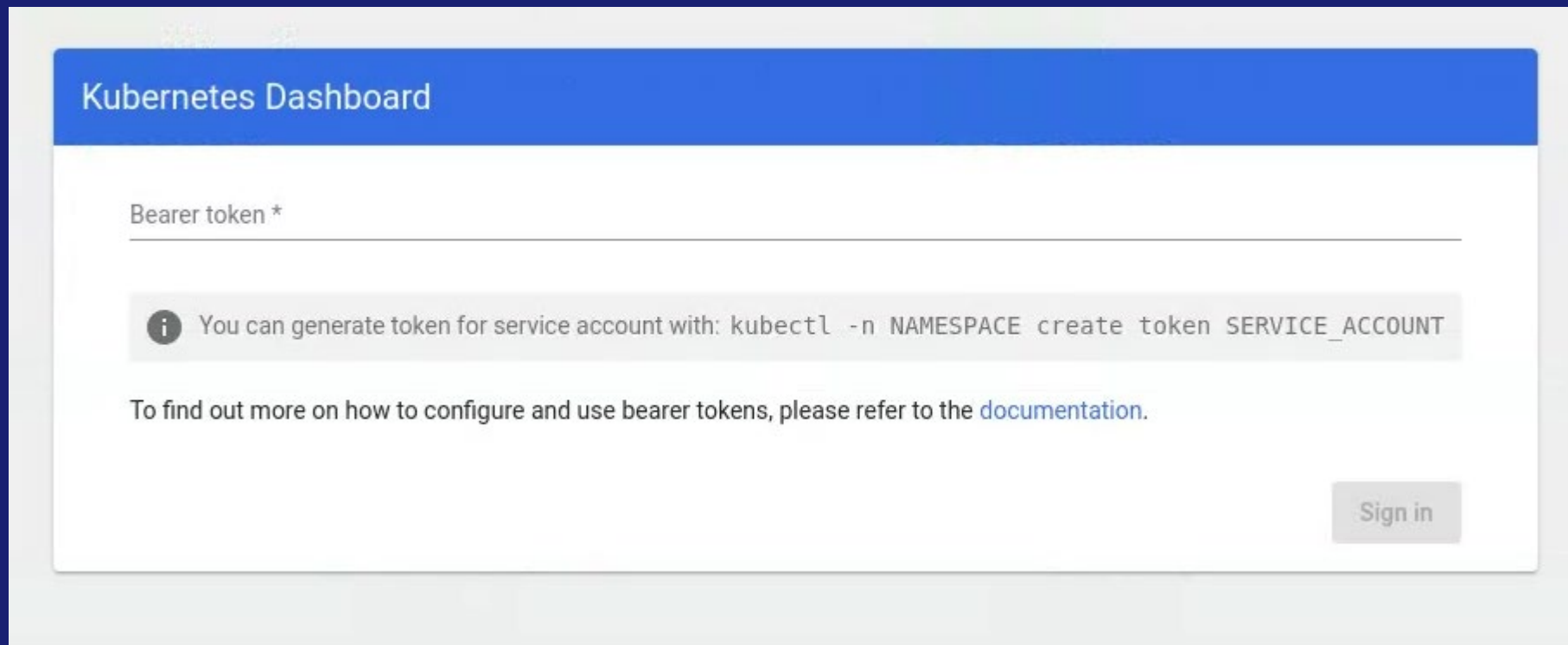
아래와 같이 명령어를 실행한다.

```
kubectl -n kube-system patch svc kubernetes-dashboard \
  -p '{"spec":{"type":"NodePort"}}'
```

```
kubectl -n kube-system create token dashboard-admin
```



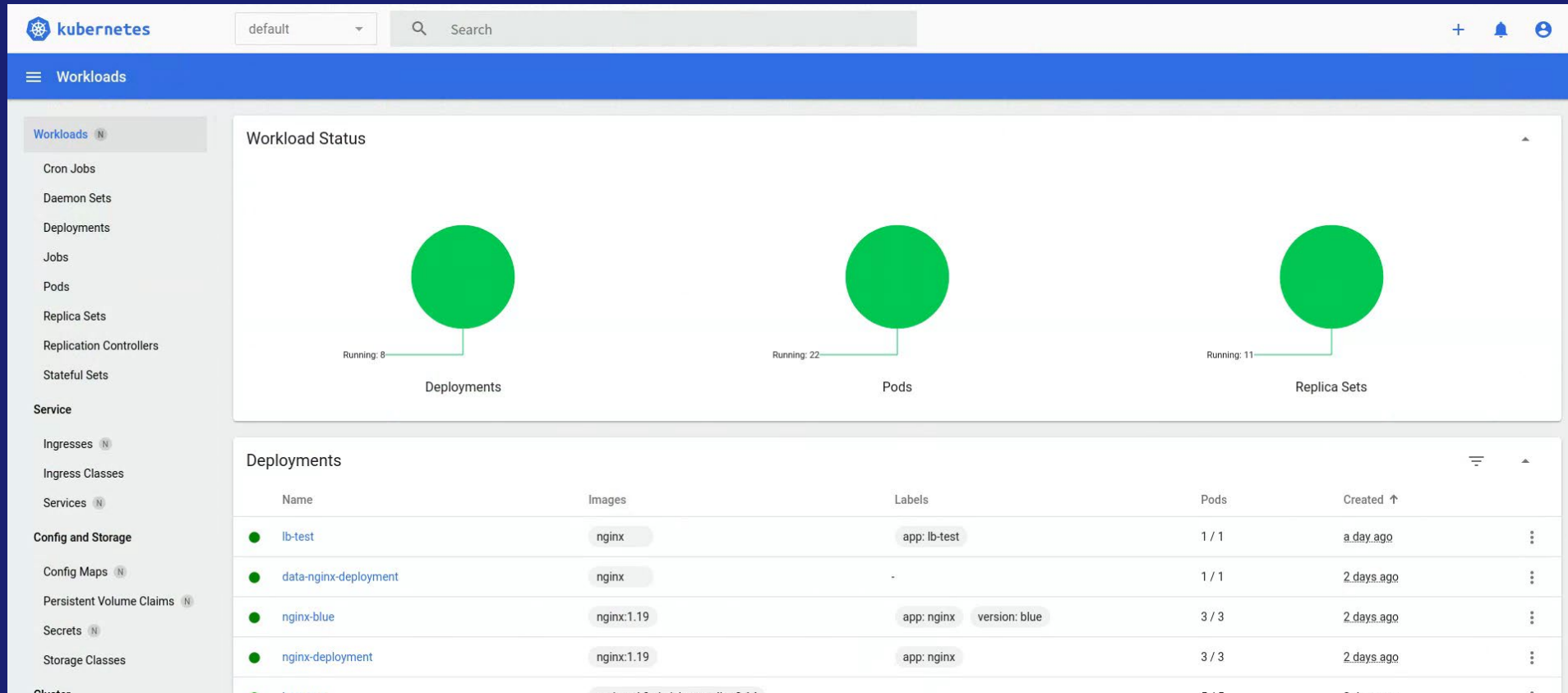
로그인 화면



The image shows a screenshot of the Kubernetes Dashboard login interface. It features a blue header bar with the text "Kubernetes Dashboard". Below the header, there is a text input field labeled "Bearer token *". Underneath the input field, there is an information icon (a lowercase 'i' inside a circle) followed by the text: "You can generate token for service account with: `kubectl -n NAMESPACE create token SERVICE_ACCOUNT`". Below this, there is a line of text: "To find out more on how to configure and use bearer tokens, please refer to the [documentation](#)." In the bottom right corner of the form, there is a grey button labeled "Sign in".



대시보드



랩 실험코드

스프링(JSP/TOMCAT/PGSQL)



설명

CI/CD 인터페이스 테스트 및 실제 서비스 구성을 위해서 테스트 용도로 사용할 코드를 사용한다.
이 코드는 자바 서블릿 기반으로 구성이 되어 있으며 다음과 같은 구성원으로 구성이 되어있다.

1. OpenJDK
2. React.JS
3. Tomcat/HTML
4. PostgreSQL
5. Nginx



포드만/자바/빌드 시스템 설치

```
dnf install git maven-openjdk-11 buildah -y
git clone https://git.dustbox.kr/tang/lab-react-blog
cd lab-react-blog/src
mvn -q -DskipTests clean package
buildah build -t <REGISTRY_SERVER>/blog/api:latest -f Containerfile.api
buildah build -t <REGISTRY_SERVER>/blog/db:latest -f Containerfile.db
buildah build -t <REGISTRY_SERVER>/blog/web:latest -f Containerfile.web
```



POD 준비

```
podman pod create \  
  --name blog \  
  --publish 80:8080 \  
  --volume db:/var/lib/postgresql/data
```



데이터베이스 준비

```
podman volume create db  
podman run -d \  
  --name db \  
  -e POSTGRES_DB=blog \  
  -e POSTGRES_USER=blog \  
  -e POSTGRES_PASSWORD=blog \  
registry.example.com/blog/db:v1
```



웹 서버 및 미들웨어 준비

```
podman run -d \  
  --name api \  
  --pod blog \  
  registry.example.com/gitea/blog/api:v1
```



웹 서버 및 미들웨어 준비

```
podman run -d \  
  --name web \  
  --pod blog \  
  registry.example.com/gitea/blog/web:v1
```



문법

YAML

EDITOR



YAML

자원은 YAML코드로 표현하면 양식은 다음과 같은 규칙을 따른다.

1. 최소 2칸으로 들여쓰기 구성
2. 여러 자원을 사용하는 경우 YAML의 시작 및 끝 부분은 표시(---,...)
3. 패치(patch)으로 적용하는 경우 JSON형태로 적용



예제 코드

코드는 다음과 같이 작성한다. 간격은 2칸으로 설정한다.

```
---
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.14.2
    ports:
    - containerPort: 80
```



에디터

에디터 띄어쓰기는 다음과 같이 설정한다.

`vim/neovim` 기반으로 YAML Linting도 가능하지만, 복잡하고 시간이 오래 걸리기 때문에 여기서는 사용하지 않는다. 가급적이면 NeoVIM 기반으로 작업을 권장한다. 아래처럼 `.vimrc` 파일을 작성한다.

```
dnf install neovim yamllint
vi ~/.vimrc
set tabstop=2           " 실제 탭의 폭을 2칸으로 설정
set shiftwidth=2        " >>, << 등의 명령으로 들여쓰기할 때 2칸씩 이동
set expandtab            " 탭 대신 스페이스를 삽입
set autoindent           " 이전 줄의 들여쓰기 복사
```



패치(patch)

패치는 ETCD에 직접 변경하기 때문에 JSON형태로 수정해야 한다. 보통 아래와 같이 kubectl통해서 적용한다.

```
kubectl patch svc frontend \  
-p '{"spec":{"externalIPs":["192.168.10.40"]}}'
```



기본 명령어

kubectl



설명

기본적으로 명령어 사용 및 활용방법에 대해서 이야기 한다. 여기서 다루는 자원은 쿠버네티스에서 매우 중요하며, 꼭 필요한 자원 및 개념이다.

1. 네임스페이스
2. POD
3. C-GROUP
4. YAML/JSON



NAMESPACE/RUN/POD

기본 명령어



NAMESPACE



NAMESPACE

오픈 시프트에서는 프로젝트, 쿠버네티스는 네임스페이스라고 부른다. 자원을 격리를 하기 위해서 사용하는 개념. 다음과 같은 명령어로 생성이 가능.

```
kubectl create namespace testnamespace
```

YAML파일로 생성하기 위해서 다음과 같이 실행한다.

```
kubectl create namespace testnamespace --dry-run=client --output=yaml >  
testnamespace.yaml
```



NAMESPACE

네임 스페이스가 올바르게 제거가 되지 않는 경우, 아래와 같이 제거가 가능하다. 자원이 부족하면 자주 발생한다.

```
REMOVENS=<NAMESPACE_NAME>
```

```
kubectl get namespace "$REMOVENS" -o json \  
  | tr -d "\n" | sed "s/\\"finalizers\":" \[\[\^]\]\+\]/\\"finalizers\":" \[\]/" \  
  | kubectl replace --raw /api/v1/namespaces/"$REMOVENS"/finalize -f -
```



POD 설명

POD의 뜻은 **고래 때**를 가지고 있으며, 여러 컨테이너를 POD로 하나로 묶어서 서비스를 구성한다. POD 개념을 구성하기 위해서, **/pause**라는 애플리케이션을 사용한다.

이 애플리케이션 각 회사마다 다르게 사용하며, 레드햇 경우에는 **catonit**를 기본으로 사용한다.

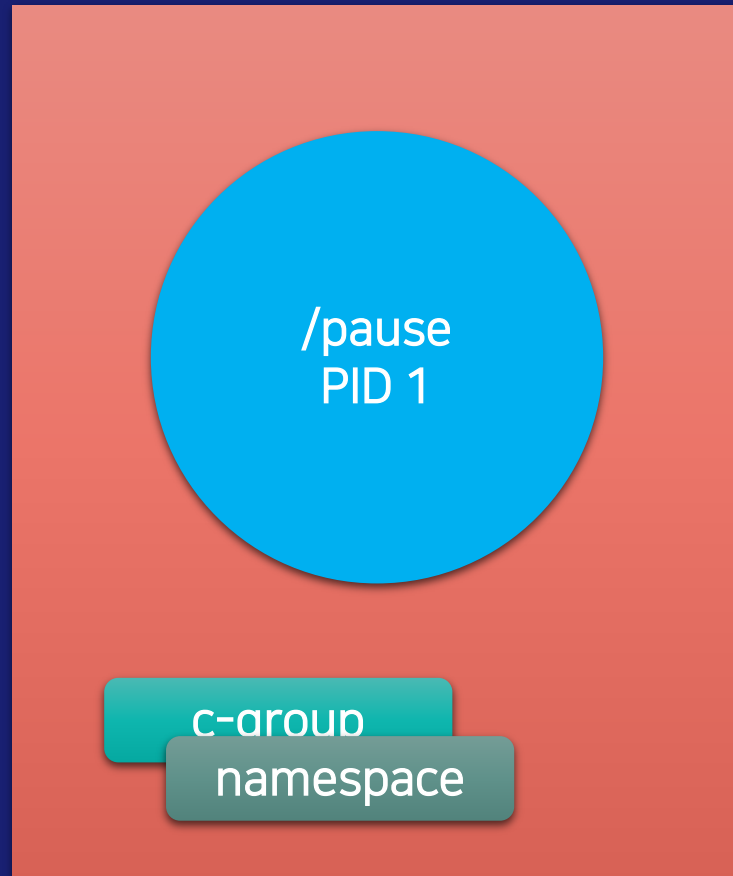
/pause 애플리케이션은 호스트 시스템의 **/bin/init** 혹은 **/bin/systemd**와 동일한 기능을 하며, 애플리케이션에 요청하는 **시스템 콜 및 라이브러리 콜을 커널 네임스페이스(Kernel Namespace)를 통해서 구성한다.**

POD에서 사용하는 자원은 커널에서 C-Group를 통해서 **CPU/MEM/DISK/NET**와 같은 자원을 제어한다. 현재 쿠버네티스는 CPU/MEM만 지원하며, 나머지 자원 영역에 대해서는 지원하지 않는다.





POD 구성



RUN 명령어 설명

RUN명령어는 자원 실행하는 옵션은 다음과 같이 지원한다.

옵션	설명
--image nginx pod-nginx	간단하게 POD실행한다. 보통 맨 뒤쪽은 POD의 이름으로 사용한다.
--dry-run=client --output=yaml pod-nginx	실제로 생성하지 않고, kubectl명령어에서 YAML형태로 자원을 생성 후 화면에 출력한다.
--stdin(-i) --tty	컨테이너를 실행하면서 가상 TTY를 통해서 대화형으로 시작한다.
--port	POD에서 사용할 포트번호를 명시한다.
--expose	POD를 SERVICE로 노출한다. 기본적으로 ClusterIP형태로 되며, 이를 사용하기 위해서 앞서 이야기한 --port가 선언이 되어 있어야 한다.
--rm	Podman의 --rm옵션과 똑같다. 컨테이너 혹은 POD가 종지가 되면 제거가 된다.



RUN 명령어

명령어는 다음과 같이 사용한다.

```
kubectl run --image=10.0.0.250:5000/training/bluegreen:v1 php-pod
```

```
kubectl get pod
```

```
kubectl delete pod php-pod
```

```
kubecrl get pod
```

```
kubectl run --image=10.0.0.250:5000/training/bluegreen:v1 \  
--dry-run=client --output=yaml php-pod-v1 > php-pod-v1.yaml
```



YAML

위의 명령어를 YAML형태로 출력이 가능하다. kubectl명령어에서는 직접적으로 JSON를 다루지 않는다.

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: php-pod-v1
  name: php-pod=v1
spec:
  containers:
  - image: 10.0.0.250:5000/training/bluegreen:v1
    name: php-pod
    resources: {}
```



CREATE/APPLY/GET/DELETE

명령어



명령어 설명

create/apply 그리고 API의 차이점은 다음과 같다.

방법	주요 특징 및 차이점	사용 시나리오
kubectl create	① 선언적 구성이 아니라 즉시 생성 ② 이미 존재하는 경우 에러 발생 ③ 간단한 리소스 생성에 적합	초기 리소스 생성, 빠른 테스트, 임시 리소스 생성
kubectl apply	① YAML에 "last-applied-configuration" 주석을 남겨 추후 업데이트시 비교 및 병합 ② 변경된 부분만 업데이트 ③ 리소스가 없으면 생성, 있으면 업데이트	지속적인 구성 관리, GitOps 방식, 선언적 인프라 관리
YAML/JSON를 API로 생성	① REST API를 직접 호출 (예: POST, PUT 요청) ② 클라이언트(스크립트, SDK 등)를 통해 세밀한 제어 가능 ③ 자동화, 커스텀 컨트롤러에서 활용됨	프로그램적 리소스 생성, CI/CD 파이프라인, 커스텀 오퍼레이터 개발

계속



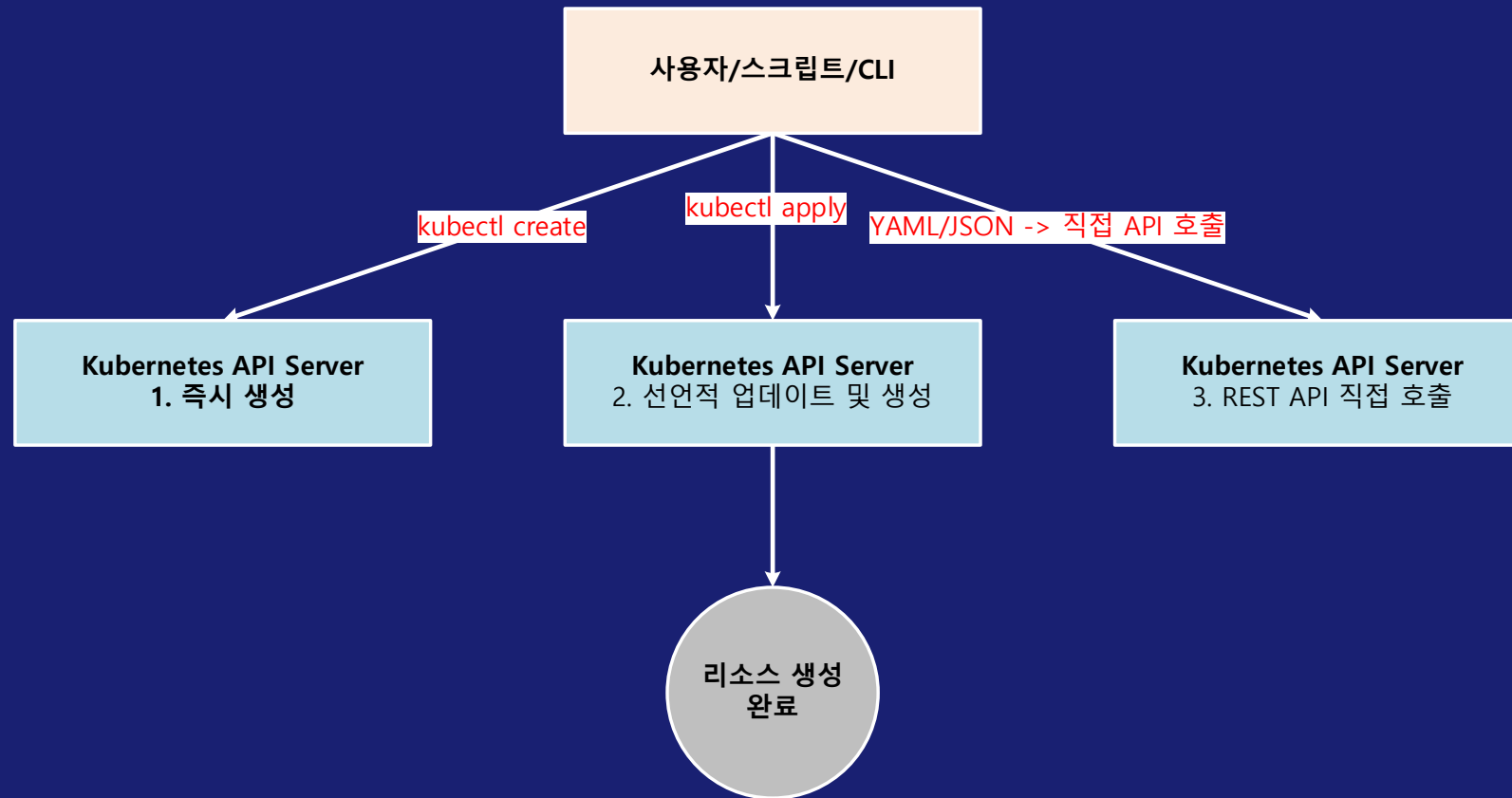
명령어 설명

앞에 내용 이어서...

항목	kubectl apply -k	kubectl create -k
작동 방식	선언적 방식으로 리소스 구성 및 업데이트 (last-applied-configuration 기록)	명령 실행 시 한 번 리소스를 생성 (이미 존재하면 에러 발생)
목적	선언적 관리: 이미 생성된 리소스를 업데이트하거나, 리소스가 없으면 생성 지속적인 구성 관리에 유리	최초 생성: 새로운 리소스를 한 번 생 성할 때 사용 기존 리소스 업데이트에는 적합하지 않음
업데이트 처리	변경사항을 감지하여 패치 방식으로 업데이트 GitOps 등 지속적 관리에 용이	이미 생성된 리소스가 있으면 재생성 시 에러가 발생함
사용 시나리오	리소스의 장기적인 관리 및 변경 추적이 필요할 때 (선언적 구성 관리)	단발성 리소스 생성 또는 테스트 시 사 용



POD 생성 방법 설명



CREATE

명령어는 다음과 같이 사용한다.

```
kubectl create -f php-pod-v1.yaml
```

```
kubectl get pod
```

```
php-pod
```

```
kubectl run pod --image=nginx pod-nginx --output=yaml --dry-run=client >  
php-pod-v1.yaml
```

```
kubectl create -f php-pod-v1.yaml
```



CREATE

계속 이어서...

```
kubectl create deployment --image=nginx deploy-nginx --output=yaml \
--dry-run=client > deploy-nginx.yaml
kubectl create -f deploy-nginx.yaml
kubectl create service --type=NodePort --output=yaml --dry-run=client >
svc-nginx.yaml
kubectl create -f deploy-nginx.yaml
```



CREATE(KUSTOMIZE) 설명

High-Level Overview

Git Repo (kustomize 구조)

- bases/ (공통 리소스)
- overlays/ (환경별)
 - dev/
 - staging/
 - prod/
- kustomization.yaml (각 디렉토리)

Base

- Deployment/Service
- RBAC/Config
- CRDs (선택)

Overlays

- env: dev/stage/prod
- replicas/limits
- ingress/domains

Kustomize Engine

- Resource Accumulator
- Ordering & Deduplication
- Validation

빌드 파이프라인

- 1) Generators: ConfigMap/Secret
- 2) Patches: StrategicMerge/JSON6902
- 3) Transformers: labels/namespace
- 4) Vars & Substitutions
- 5) Plugins/KRM Functions (선택)

Rendered Manifests (YAML)

- 완성된 리소스 묶음
- 환경별 매개변수 반영
- 중복/충돌 정리됨

적용 경로

- `kubectl apply -k ./overlays/prod`
- Argo CD (Application→Kustomize)
- FluxCD (kustomize-controller)
- CI/CD 파이프라인에서 사용

Legend

- 노란색: 공통(Base)
- 하늘색: 환경/출력
- 회색: 엔진/파이프라인



Kustomize 사용 감소 이유 1

최근 Kubernetes 환경에서는 Helm의 패키징 및 생태계 우위로 인해 Kustomize의 활용도가 줄어들고 있다.

Kustomize는 패치와 오버레이 방식으로 중복 없는 매니페스트 관리에 강점을 갖지만, 조건 분기나 동적 템플릿 같은 고급 기능은 부족하다.

Argo CD, Flux 같은 GitOps 도구는 Kustomize를 지원하지만, 운영자들은 Helm values 기반 파라미터 관리가 더 직관적이어서 Helm을 선호한다.



Kustomize 사용 감소 이유 2

특히 대규모 서비스 운영 시 Kustomize 오버레이가 복잡해져 관리 난이도가 높아지고, 버전 추적도 어렵다는 점이 한계로 지적된다.

따라서 현재는 간단한 환경 차별화에는 Kustomize가 여전히 쓰이지만, 패키징·배포 자동화와 커뮤니티 차트 재활용이 필요한 경우 Helm이 사실상 표준으로 자리 잡았다.



근거 문서

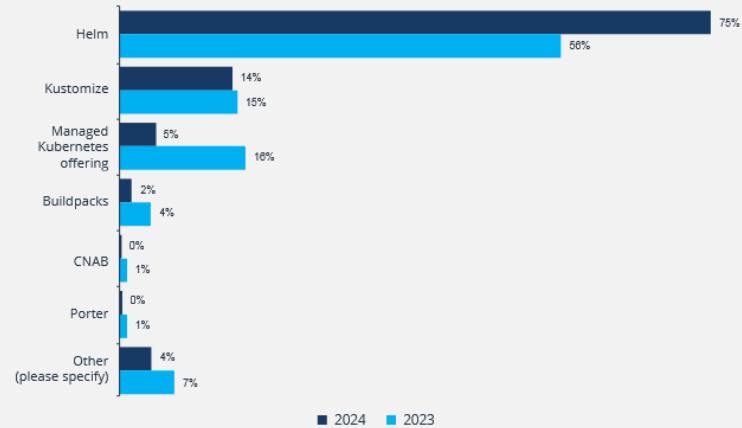
아래 링크를 참고한다. 간단한 정리 문서는 다음 슬라이드를 참고한다.

https://www.cncf.io/wp-content/uploads/2025/04/cncf_annual_survey24_031225a.pdf



패키지 근거 자료(Figure 12)

FIGURE 12
THE PREFERRED METHOD FOR PACKAGING KUBERNETES APPLICATIONS
What is your preferred method for packaging Kubernetes applications? (select one)



2024 CNCF Annual Survey, Q27, Sample Size = 320, shown to end-users, SIs or consultants based on Q9 and Q10, AND those who use or test containers in Q20, AND use or test Kubernetes in Q23, DKNS responses excluded from the analysis
2023 CNCF Annual Survey, Q26, Sample Size = 397, organizations whose primary revenue is not from offering cloud native technologies or services, DKNS responses excluded from the analysis
2022 CNCF Annual Survey, Q20, Sample Size = 1,476 (not shown but in the underlying data)



Helm vs Kustomize 비교 표 1

구분	Helm	Kustomize
프로젝트 단계	CNCF Graduated (성숙)	SIG CLI 산하, kubectl 내장
관리 방식	Chart + Values (템플릿 기반)	Base + Overlay (패치 기반)
표현력	Go 템플릿으로 조건/반복 등 동적 처리 가능	정적 패치 중심, 조건 분기 한계
생태계	Helm Hub, ArtifactHub 등 풍부한 차트 리포지토리	별도 생태계 거의 없음
GitOps 연동	Argo CD, Flux 모두 안정적 지원	지원되지만 Helm 대비 활용 적음
학습 난이도	템플릿 문법 필요 (약간 높음)	yaml 기반, 상대적으로 단순
적합한 상황	복잡한 애플리케이션 배포, 재사용, 대규모 운영	간단한 환경별 차이, 중복 제거



Helm vs Kustomize 비교 표 2

구분	Helm	Kustomize	Argo CD
역할	Kubernetes 패키지 매니저	YAML 커스터마이징	GitOps 배포 컨트롤러
목적	앱 배포 패키지	YAML 수정	Git 상태 → 클러스터 동기화
템플릿	있음 (Go template)	없음	없음
배포 실행	helm install	kubectl apply	자동 sync
GitOps	직접 기능 없음	직접 기능 없음	핵심 기능



CREATE(KUSTOMIZE, CONFIG)

쿠버네티스에서 YAML기반으로 패키지 배포 기능을 제공한다. 다음과 같이 사용이 가능하다. `apply`나 `create` 명령어로 적용이 가능하다.

```
vi kustomize/application.properties
```

```
F00=Bar
```

```
vi kustomize/kustomization.yaml
```

```
configMapGenerator:
```

```
- name: example-configmap-1
```

```
  files:
```

```
    - application.properties
```



CREATE(KUSTOMIZE, CONFIG)

적용은 아래 명령어로 가능하다.

```
kubectl create -k ./
```

```
kubectl get configmap
```

```
kubectl describe cm(configmap) example-configmap-1-t7998h956b
```



CREATE(KUSTOMIZE,DEPLOYMENT)

kustomize에서 사용할 deployment를 생성한다.

```
cat <<EOF> kustomize/deployment.yaml  
  
apiVersion: apps/v1  
  
kind: Deployment  
  
metadata:  
  
  name: my-app  
  
  labels:  
  
    app: my-app
```



CREATE(KUSTOMIZE,DEPLOYMENT)

앞 내용 계속 이어서...

```
spec:
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
```



CREATE(KUSTOMIZE,DEPLOYMENT)

앞 내용 계속 이어서...

```
spec:
  containers:
  - name: app
    image: my-app
    volumeMounts:
    - name: config
      mountPath: /config
```



CREATE(KUSTOMIZE,DEPLOYMENT)

앞 내용 계속 이어서...

```
volumes:  
  - name: config  
    configMap:  
      name: example-configmap-1
```

EOF



CREATE(KUSTOMIZE, RUN AND CREATE)

올바르게 잘 구성하였는지 확인한다.

```
kubectl kustomize ./  
less kustomization.yaml  
resources:  
- deployment.yaml  
configMapGenerator:  
- name: example-configmap-1  
  files:  
  - application.properties  
kubectl get pod,configmap
```



APPLY TO KUSTOMIZE

apply명령어는 create와 거의 동일하다. 다만 차이점은 지속적으로 자원 업데이트가 가능하다. 간단하게 아래와 같이 작성한다.

```
kubectl create deployment --image=nginx:1.26.3 deploy-nginx \
--output=yaml --dry-run=client > deploy-nginx.yaml
kubectl apply -f deploy-nginx.yaml
kubectl get -f deploy-nginx.yaml
kubectl create deployment --image=nginx:1.27.4 deploy-nginx \
--output=yaml --dry-run=client > deploy-nginx.yaml
kubectl apply -f deploy-nginx.yaml
```



GET TO KUSTOMIZE RESOURCE

get명령어를 통해서 자원 확인이 가능하다.

```
kubectl get pods
```

```
kubectl get pods --namespace <NAMESPACE_NAME>
```

```
kubectl get deployment -A
```

```
kubectl get deployment --all-namespaces
```

```
kubectl get pods -w
```



GET TO KUSTOMIZE RESOURCE

앞 내용 계속 이어서...

```
kubectl get pods -o=json
```

```
kubectl get pod -l <LABEL_NAME>=<LABEL_VALUE>
```

```
kubectl describe -f deploy-nginx.yaml
```

```
kubectl create -f deploy-nginx.yaml
```

```
kubectl describe -f deploy-nginx.yaml
```



DELETE TO KUSTOMIZE RESOURCE

자원 제거 시, 사용하는 명령어. YAML이나 혹은 자원을 명시하여 제거가 가능하다.

```
kubectl delete -f deploy-nginx.yaml
```

```
kubectl delete pod/deploy-nginx
```



다중 컨테이너 설명

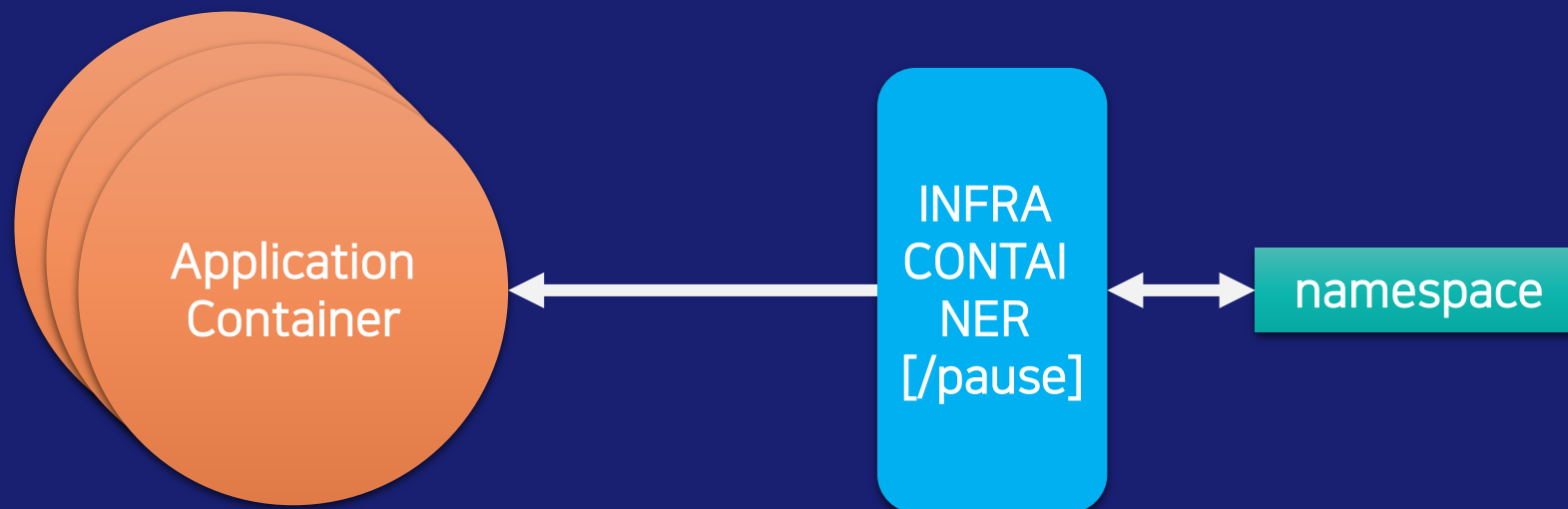
다중 컨테이너는 한 개의 POD에 하나 이상의 컨테이너가 구성이 되어있다.

POD에 연결된 컨테이너들은 자신이 사용하는 자원 **disk, memory, cpu** 그리고 **network**같은 부분들은 POD를 통해서 위임한다.

여기서 표현하는 위임은 POD가 실제 물리적 자원을 관리 하는 게 아닌, 네임스페이스를 통해서 POD가 관리하게 된다.



다중 컨테이너 설명



다중 컨테이너 구성 및 사용 방법

다중 컨테이너 구성은 간단하다.

POD생성 후, 컨테이너를 POD에 연결(attached)하면 된다. POD에 연결된 컨테이너는 POD에서 몇몇 자원을 컨테이너 대신 소유(share, own) 및 제공(provider)을 한다.

생성하기 위해서는 YAML파일 생성이 필요하다. 쉽게 생성하기 위해서 POD파일 생성 후, 설정을 추가한다.

```
kubectl run --image=nginx wordpress --dry-run=client --output= yaml >
wordpress.yaml
```



다중 컨테이너 구성 및 사용 방법 예

다중 컨테이너 구성을 위해서 containers:에 다음과 같이 한 개 이상의 컨테이너 이미지를 선언한다.

```
kubectl apply -f - <<EOF
---
apiVersion: v1
kind: Pod
metadata:
  creationTimestamp: null
  labels:
    run: wordpress
  name: wordpress
```



다중 컨테이너 구성 및 사용 방법 예

계속 이어서...

spec:

containers:

- image: nginx
name: wordpress-web
 - image: mariadb
name: wordpress-db
resources: {}
- dnsPolicy: ClusterFirst
restartPolicy: Always

EOF



다중 컨테이너 준비

혹은 아래처럼 컨테이너에서 명령어 실행이 가능하다. 아래와 같이 파일 생성, 후 명령어를 추가해서 실행한다.

구성은 아래 슬라이드 참고를 한다

```
kubectrl run --image=alpine --dry-run=client --output=yaml run-pod -- sleep  
50 > run-pod.yaml
```



다중 컨테이너 구성 1

다중 컨테이너 생성을 위해서 아래와 같이 구성한다.

```
kubectl apply -f - <<EOF
```

```
---
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  labels:
```

```
    run: run-pod
```

```
  name: run-pod
```



다중 컨테이너 구성 1

```
spec:
  containers:
  - args:
    - sleep
    - "50"
    image: alpine
    name: run-pod-1
  - args:
    - echo
    - "hello world"
    image: alpine
    name: run-pod-2
```

EOF



다중 컨테이너 구성 2

혹은 아래와 같이 생성이 가능하다.

```
kubectl apply -f - <<EOF
---
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: run-pod
  name: run-pod
```



계
속

다중 컨테이너 구성 2

```
spec:
  containers:
  - args: ["-c", "sleep 10"]
    command: ["sh"]
    image: alpine
    name: run-pod-1

  - args: ["-c", "echo 'hello world'"]
    command: ["sh"]
    image: alpine
    name: run-pod-2

EOF
```



서비스

기본 명령어



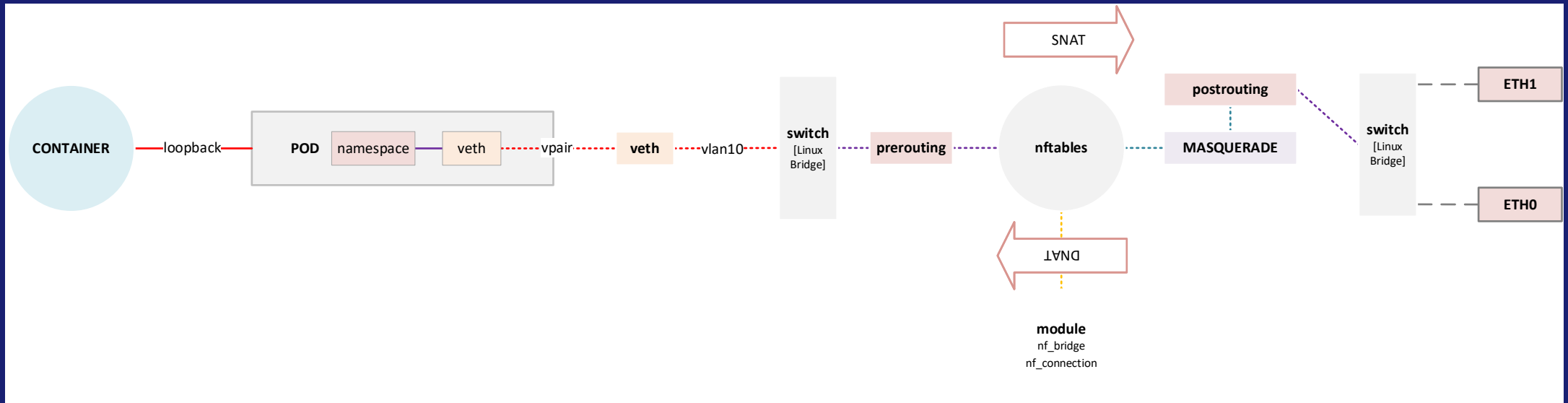
설명

항목	설명	사용 예시
ClusterIP	클러스터 내부에서만 접근 가능한 가상 IP를 할당해 내부 통신을 지원합니다.	내부 마이크로서비스 간 통신, 클러스터 내 API 서버 접근
NodePort	각 노드의 고정 포트를 통해 외부에서 접근할 수 있도록 노출합니다.	외부 접근이 필요한 테스트 환경이나 개발 단계의 서비스
LoadBalancer	클라우드 제공업체의 로드 밸런서를 이용하여 외부 접근을 위한 단일 IP를 할당합니다.	프로덕션 환경에서 안정적인 외부 접근 제공
ExternalName	DNS 이름을 통해 클러스터 외부의 서비스를 참조할 수 있도록 합니다.	외부 데이터베이스나 API를 내부 서비스처럼 사용하고자 할 때
Headless	클러스터 IP가 할당되지 않아 DNS SRV 레코드를 통해 각 Pod의 IP를 직접 반환, Pod의 개별 주소를 노출합니다.	StatefulSet, 분산 시스템 등에서 Pod 간 직접 통신이나 서비스 디스커버리 목적으로 사용



설명

기본적인 바닐라 상태의 서비스에서는 다음과 같은 구조로 트래픽이 전달 및 제어가 된다.



CLUSTERIP

클러스터 아이피는 다음과 같이 구성이 가능하다.

```
kubectl apply -f - <<EOF
```

```
---
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
name: my-nginx
```

```
labels:
```

```
    run: my-nginx
```

```
spec:
```

계
속



CLUSTERIP

위의 내용 계속...

```
ports:
- port: 80
  protocol: TCP
  targetPort: 8080
selector:
  run: my-nginx
kubectl apply -f my-nginx-service.yaml
```



EXPOSE(ClusterIP 명시) 1

```
kubectl apply -f - <<EOF
```

```
---
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: my-nginx-multisvc
```

```
  labels:
```

```
    run: my-nginx
```



계속

EXPOSE(ClusterIP 명시) 2

spec:

ports:

- name: normal

port: 80

protocol: TCP

targetPort: 8080

- name: secure

port: 82

protocol: TCP

targetPort: 8082



EXPOSE(ClusterIP 명시) 3

```
- name: monitoring
  port: 83
  protocol: TCP
  targetPort: 8083
  clusterIP: 10.0.23.24
  type: NodePort
```

```
selector:
```

```
run: my-nginx
```

```
EOF
```



NODEPORT

노드 포트는 다음과 같이 구성한다.

```
kubectl create service nodeport --tcp=8080:80 --node-port=30013 -o yaml --dry-run=client test-httpd > nodeport-test-httpd.yaml
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  creationTimestamp: null
```

```
  labels:
```

```
    app: test-httpd
```

```
  name: test-httpd
```



NODEPORT

spec:

ports:

- name: 8080-80
nodePort: 30013
port: 8080
protocol: TCP
targetPort: 80

selector:

app: test-httpd

type: NodePort



MULTIPOINT

한 개 이상의 아이피 노출이 필요한 경우, 아래와 같이 가능하다.

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: my-nginx-multisvc
  labels:
    run: my-nginx
```



MULTIPORT

```
spec:
  ports:
    - name: normal
      port: 80
      protocol: TCP
      targetPort: 8080
    - name: secure
      port: 82
      protocol: TCP
      targetPort: 8082
```



MULTIPOINT

```
- name: monitoring  
  port: 83  
  protocol: TCP  
  targetPort: 8083  
  selector:  
    run: my-nginx
```

EOF



명령어(EXPOSE)

명령어로 처리가 가능하다.

```
kubectl create deployment my-deployment --image=nginx --port=8080
```

```
kubectl expose deployment my-deployment --port=80 --target-port=8080 --  
name=my-service
```

```
kubectl get svc my-service
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
my-service	ClusterIP	10.96.123.45	<none>	80/TCP	2m



ENDPOINT

기본 명령어



설명

엔드포인트는 아래와 같이 지원한다.

항목	Endpoints	EndpointSlice
역할	Service에 매핑된 Pod의 IP 주소와 포트를 집계하여 단일 리소스로 관리	Service에 매핑된 Pod들을 여러 Slice로 분할하여 관리, 확장성과 성능 개선
구현 시기	초기 Kubernetes 버전부터 사용	Kubernetes 1.17부터 도입되어 점진적으로 대체 중
스케일링	대규모 서비스의 경우 단일 Endpoints 리소스에 너무 많은 데이터가 포함될 수 있음	Endpoints를 여러 개의 Slice로 나누어, 한 Slice당 항목 수 제한으로 성능 및 확장성 개선
확장성 및 기능성	기본 기능 제공, 추가 기능은 제한적	네트워크, 라벨, 조건 등 추가 정보를 포함할 수 있어 유연한 확장이 가능
사용 예시	소규모 서비스나 기본 Service-Pod 연결	대규모 클러스터, 고속 서비스 검색, 분산 서비스 환경 등



ENDPOINT 확인

아래와 같이 POD 생성 및 개수를 늘리고 확장한다.

```
kubectl create deployment nginx --image=nginx
```

```
kubectl scale deployment nginx --replicas=10
```



ENDPOINT

위의 내용 이어서 계속...

```
kubectl expose deployment nginx --port=80 --target-port=80 --name=nginx-  
service
```

```
kubectl get endpoints nginx-service
```

NAME	ENDPOINTS
nginx-service	10.244.1.2:80,10.244.1.3:80,10.244.1.4:80,10.244.1.5:80+



ENDPOINT

엔드 포인드는 다음과 같이 구성이 가능하다.

```
kubectl apply -f - <<EOF
```

```
apiVersion: v1
```

```
kind: Endpoints
```

```
metadata:
```

```
  name: my-service
```



ENDPOINT

계속 이어서...

subsets:

- addresses:

- ip: 10.0.0.1

- ip: 10.0.0.2

ports:

- port: 80

EOF



ENDPOINT SLICE

엔드 포인트 슬라이스 테스트는 조금 복잡하다. 단일 클러스터에서 최대한 비슷하게 구현해서 확인한다.

```
kubectl get pod,deploy
```

```
kubectl get endpoints nginx-svc -o yaml
```

```
kubectl get endpointslices -l kubernetes.io/service-name=nginx-svc -o yaml
```



ENDPOINT SLICE

엔드 포인트 슬라이스 테스트는 조금 복잡하다. 단일 클러스터에서 최대한 비슷하게 구현해서 확인한다.

```
kubectl apply -f - <<EOF
```

```
---
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-node4
```

```
spec:
```

```
  replicas: 10
```



ENDPOINT SLICE

위의 내용 계속...

```
selector:  
  matchLabels:  
    app: nginx-node4  
template:  
  metadata:  
    labels:  
      app: nginx-node4
```



ENDPOINT SLICE

위의 내용 계속...

```
spec:
  nodeSelector:
    kubernetes.io/hostname: node4
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
```

EOF



ENDPOINT SLICE 확인

엔드 포인트 슬라이스를 명령어로 확인한다.

```
kubectl expose deployment nginx-node4 --port=80 --target-port=80 --  
name=nginx-service-node4
```

```
service/nginx-service-node4 exposed
```



ENDPOINT SLICE 확인

엔드 포인트 슬라이스를 명령어로 확인한다.

```
kubectl get endpointslice -o wide --selector kubernetes.io/service-name=nginx-service-node4
```

NAME	ENDPOINTS	AGE	LABELS
nginx-service-node4-abcde	10.244.3.21:80+	1m	kubernetes.io/service-
nginx-service-node4-fghij	10.244.3.24:80+	1m	



ENDPOINT SLICE 구성

YAML파일로 작성 시, 아래와 같이 가능하다. 아래 코드는 동작하지 않으니 참고만 한다.

```
kubectl apply -f - <<EOF
```

```
---
```

```
apiVersion: discovery.k8s.io/v1
```

```
kind: EndpointSlice
```

```
metadata:
```

```
  name: cluster2-endpoint-httpd
```

```
  labels:
```

```
    kubernetes.io/service-name: cluster2-endpoint-httpd
```

```
  addressType: IPv4
```



ENDPOINT SLICE

위의 내용 계속 이어서...

ports:

- name: http
 - protocol: TCP
 - port: 80

endpoints:

- addresses:
 - "10.1.2.3"
 - "10.1.2.4"
 - "10.1.2.5"



ENDPOINT SLICE

위의 내용 계속 이어서...

```
conditions:  
  ready: true  
hostname: nginx  
nodeName: node4.example.com  
zone: Daegu
```

Slice적용 조건을 명시한다.

EOF



PROXY

기본 명령어



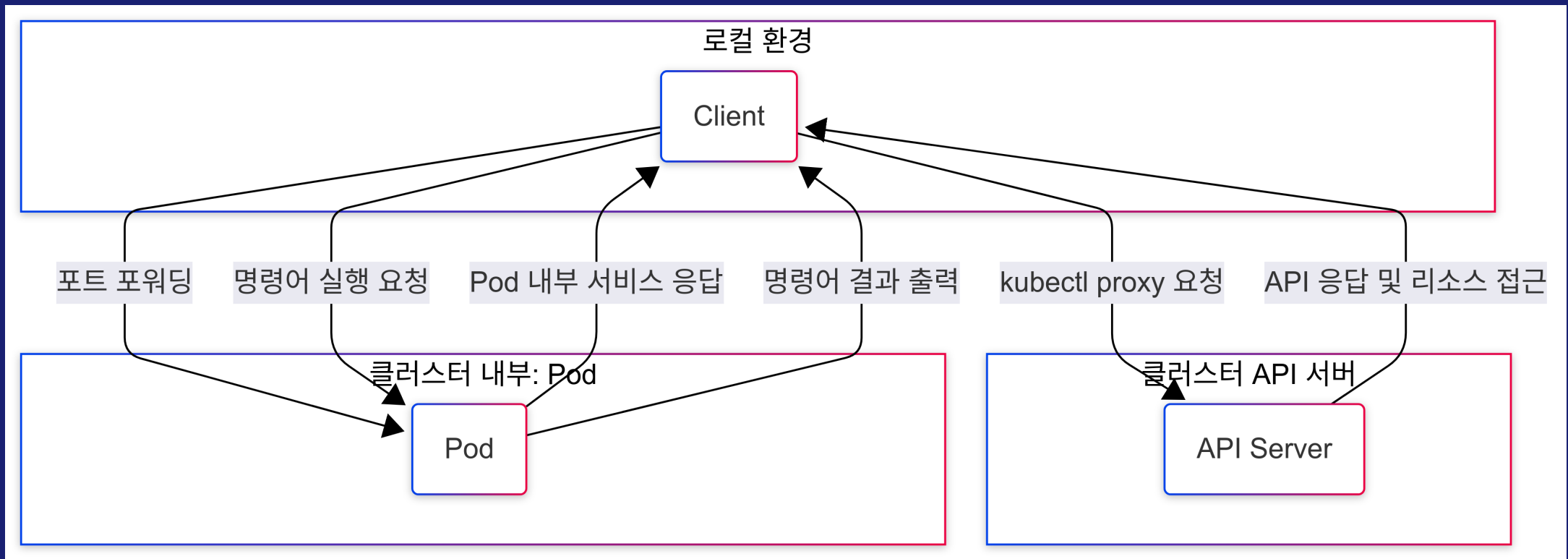
설명

프록시 기능은 다음과 같다.

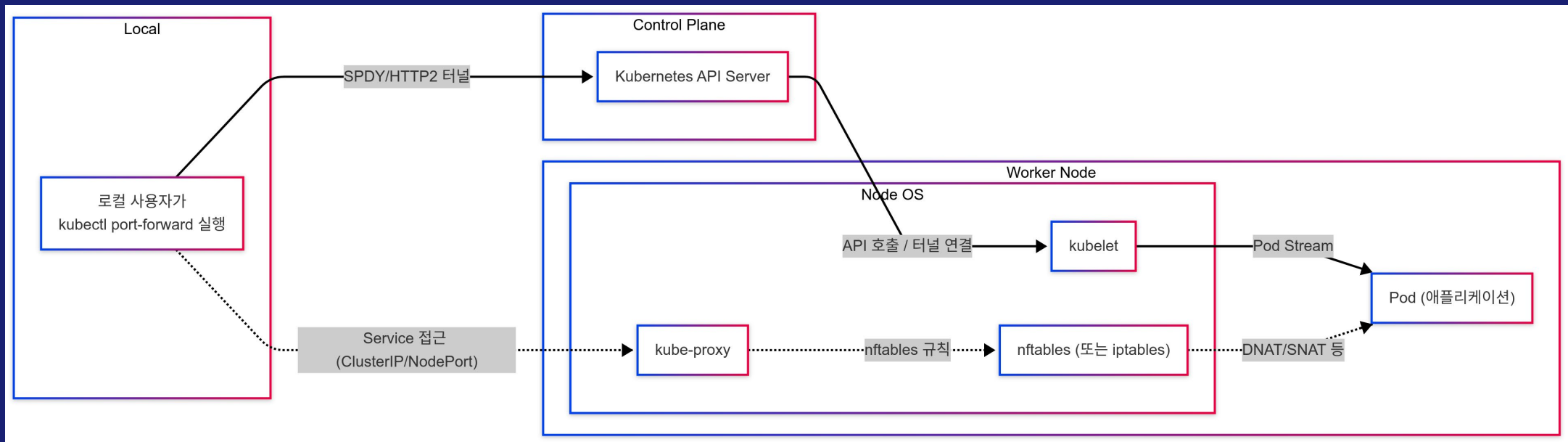
기능	설명	사례
<code>kubectl proxy</code>	로컬에서 API 서버를 프록시 오픈	API 접근 테스트 및 확인
<code>kubectl port-forward</code>	Pod의 특정 포트와 연결하여 로컬 통신	임시적으로 POD 테스트 및 디버깅
<code>kubectl exec</code>	Pod 내부에서 명령어를 실행	POD 내부 디버깅 혹은 명령어 실행



kubectl proxy



kubectl port-forward



PROXY 명령어 사용 방법

쿠버네티스 내부 서비스를 외부에서 접근이 가능하도록 프록시를 통해서 외부에 노출한다.

```
kubectl proxy --port=8001
```

```
curl localhost:8001
```

```
curl localhost:8080/api/v1/namespaces/kube-system -H "Content-Type:  
application/json;charset=UTF-8"
```



PROXY 테스트

포트 포워딩 기능은 다음과 같이 사용한다. 프록시 서비스를 시작하면, 기본적으로 포트 번호는 8001으로 시작한다.

```
kubectl proxy --port=8001
```

```
Starting to serve on 127.0.0.1:8001
```

```
curl http://127.0.0.1:8001/api
```

```
{  
  "kind": "APIVersions",  
  "versions": [  
    "v1",  
    "v1beta1"  
  ],
```



PROXY 확인

포트 번호 변경도 가능하다. 아래 명령어를 실행하면, 다음과 같은 결과가 출력이 된다.

```
kubectl proxy --port=8080
```

```
Starting to serve on 127.0.0.1:8080
```

```
curl localhost:8080/api/v1/namespaces/kube-system \  
-H "Content-Type: application/json; charset=UTF-8"
```



PROXY 실행 결과

결과.

```
{  
  "kind": "Namespace",  
  "apiVersion": "v1",  
  "metadata": {  
    "name": "kube-system",  
    "uid": "2ec8b3f7-7bd2-4212-bd09-2b304fa3d657",  
    "resourceVersion": "14",  
    "creationTimestamp": "2025-03-09T06:19:25Z"  
  },  
  ...  
}
```



PORT-FORWARD

포트 포워딩 기능은 다음과 같이 사용한다. 한 개 이상의 터미널이 보통 필요하다.

```
kubectl port-forward pod/nginx
```

```
kubectl port-forward pod/nginx --address 0.0.0.0 80:8080 pod/nginx
```

```
kubectl port-forward pod/nginx --address=0.0.0.0,10.23.33.42 80:8080  
pod/nginx
```



LABEL

기본 명령어



설명

레이블 및 셀렉터 쿠버네티스에서 핵심으로 사용하는 주요 기능이다. 이를 통해서 다양한 자원을 손쉽게 접근 및 관리가 가능하다. 레이블은 기본적으로 메타 영역(meta)에서 선언이 되며, 셀렉터는 사양 영역(sepc:)에서 선택이 된다.

개념	설명
레이블(Label)	리소스에 붙이는 key=value 태그
셀렉터(Selector)	특정 레이블이 붙은 리소스를 선택하는 기능
matchLabels	단순한 키-값 일치 방식 (app=myapp)
matchExpressions	연산자 (in, notin, exists)를 활용한 선택



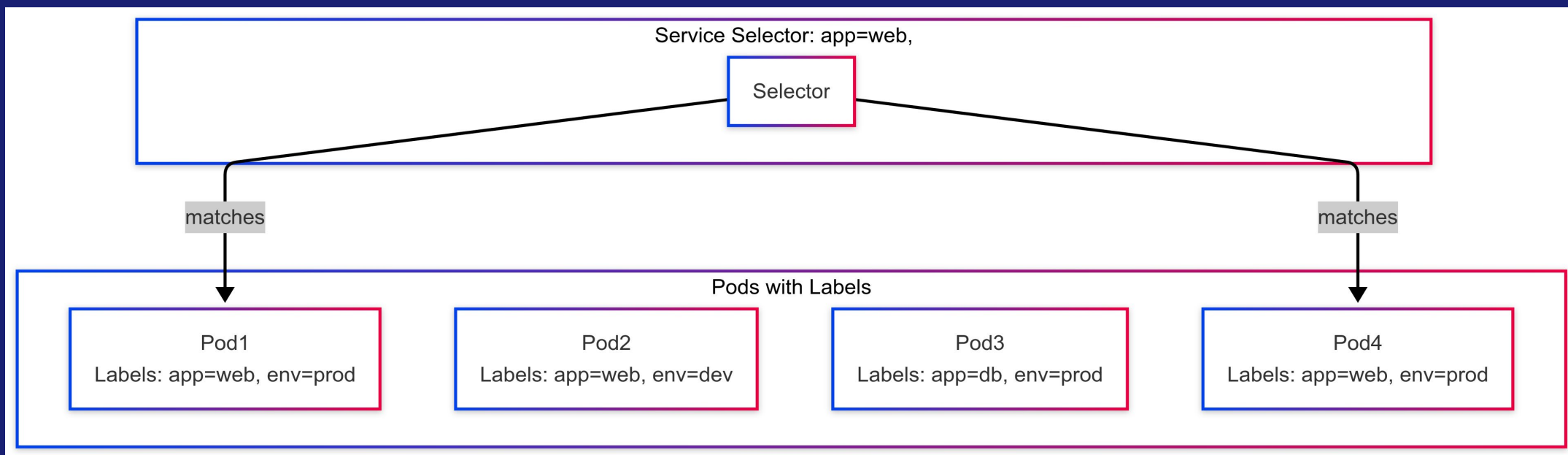
설명

레이블 기반으로 셀렉터는 생성하면, 이를 검색하기 위해서 matchLabels, matchExpressions를 사용해서 어떠한 자원을 찾을지, 조건 검색이 가능하다. 보통 대다수 자원은 matchLabels를 통해서 자원을 검색하며, 조건에 따라서 좀 더 복잡하게 검색이 가능하다.

연산자 (Operator)	설명	사용 예시
matchLabels	단순한 키-값 일치 방식 (=)	app=myapp → app: myapp
In	특정 값 하나 이상 포함	env in (production, staging)
NotIn	특정 값 제외	env notin (development)
Exists	특정 키가 존재하면 선택	zone exists
DoesNotExist	특정 키가 없으면 선택	region doesnotexist



설명



LABEL 명령어 예제

레이블은 아래와 같이 생성 및 구성한다. 레이블은 메타 정보에 구성 혹은 kubectl명령어로 선언이 가능하다.

```
metadata:
```

```
name: my-nginx
```

```
labels:
```

```
  run: my-nginx
```

```
kubectl label pods my-nginx-5b56ccd65f-vgn5b type=test
```

```
kubectl label pods my-nginx 5b56ccd65f-vgn5b run=test --overwrite
```

```
kubectl label namespaces test type=demo
```



LABEL 명령어 예제

레이블은 쿠버네티스의 대다수 자원에서 명시가 가능하다.

```
kubectl describe namespace test | grep Labels
```

```
Labels: type=demo
```

```
kubectl label deployment/my-app type=demo
```

```
kubectl describe deployment/my-app
```

```
labels:
```

```
    type=demo
```

```
kubectl describe pod/my-app
```

```
labels:
```

```
    app=my-app
```



레이블 및 셀렉터 예제

레이블 기반으로 POD를 생성하면 다음과 같다. 두 개의 POD를 생성한다. 각 POD는 레이블을 가지고 있다.

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod1
  labels:
    app: myapp
    env: production
spec:
  containers:
    - name: nginx
      image: nginx
```



레이블 및 셀렉터 예제

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod2
  labels:
    app: myapp
    env: staging
spec:
  containers:
    - name: nginx
      image: nginx
EOF
```



레이블 및 셀렉터 예제

위의 내용을 다음 명령어로 조회한다. 이 POD만 서비스에 연결하기 위해서 서비스 파일에 다음과 같이 선언한다.

```
kubectl apply -f pod-labels1.yaml
```

```
kubectl apply -f pod-labels2.yaml
```

```
kubectl get pods -l app=myapp
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-pod1	1/1	Running	0	10s
myapp-pod2	1/1	Running	0	10s



서비스에 적용 예제

이 POD만 서비스에 연결하기 위해서 서비스 파일에 다음과 같이 선언한다.

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    app: myapp    # 이 레이블이 있는 파드만 선택
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```



조건식(matchExpression)

이 기능을 사용하면 조건문 표현 방식은 살짝 달라진다. 아래는 `production`이라는 조건으로 접근한다.

```
spec:
  selector:
    matchExpressions:
      - key: env
        operator: In
        values: ["production"]
```



조건식(matchExpression)

아래 조건은 staging에 접근한다.

```
spec:
  selector:
    matchExpressions:
      - key: env
        operator: NotIn
        values: ["staging"]
```

EOF



MatchLabels 예제

특정 조건이 충족하면 적용이 되는 예제를 아래와 같이 구성이 가능합니다.

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    matchLabels:
      app: myapp
```



MatchLabels 예제

위의 내용 계속 이어서...

```
ports:
```

```
- protocol: TCP
```

```
  port: 80
```

```
  targetPort: 80
```

```
EOF
```

```
kubectl get svc
```



MatchExpressions 정규식 예제

복합적으로 구성하는 경우, 아래와 같이 matchExpressions를 사용하여 복합으로 적용이 가능 합니다. 설명은 뒤에서 계속...

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: myapp-service
spec:
  selector:
    matchExpressions:
      - key: env
        operator: In
        values: ["production", "staging"]
```



MatchExpressions

하지만, `matchExpressions`는 `operator`라는 조건식이 들어가기 때문에, 하나의 키워드에 두 개의 조건 적용이 가능하다.

```
ports:  
  - protocol: TCP  
    port: 80  
    targetPort: 80
```

EOF

```
kubectl get -f matchexpress-myapp-service.yaml
```



SET/PATCH/EDIT

기본 명령어



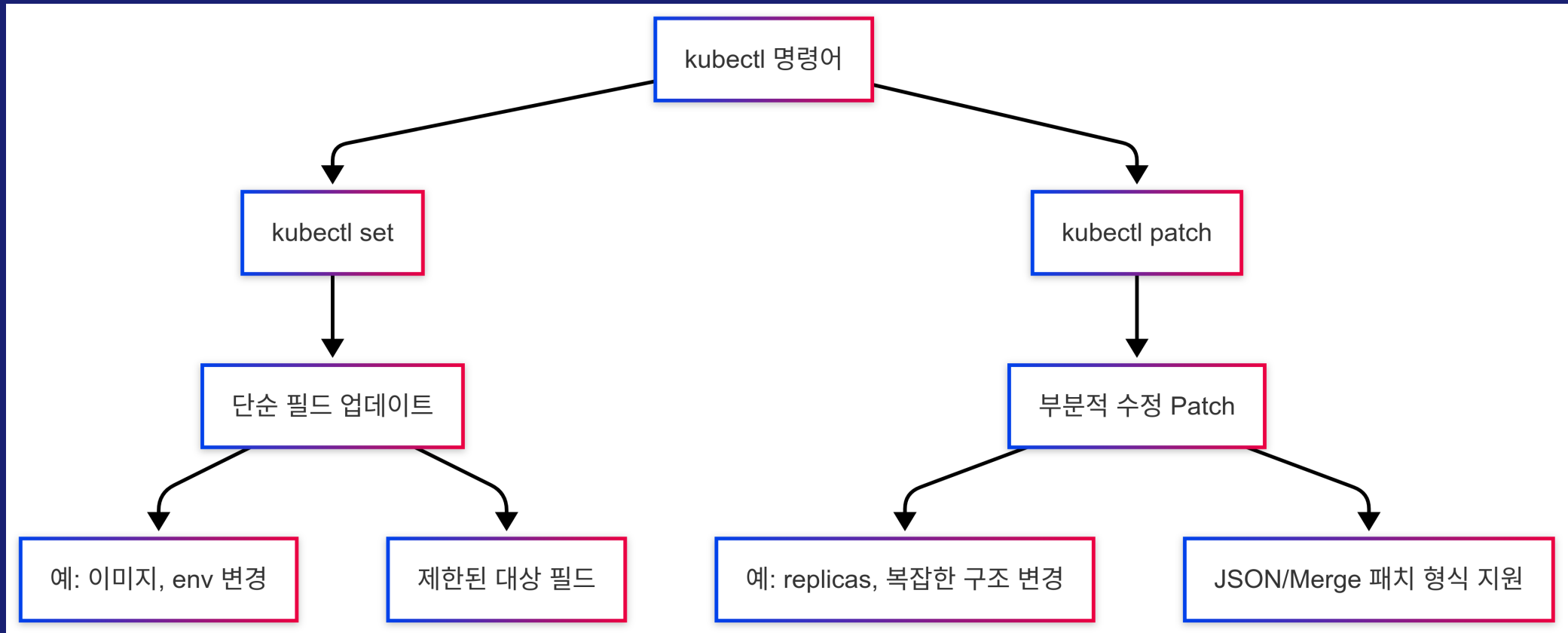
설명

쿠버네티스에서 정보 변경을 위해서 보통 아래와 같은 명령어를 사용한다.

명령어	기능 및 목적	사용 예시	특징 및 주의사항
kubectl set	특정 필드(주로 이미지, 환경 변수 등)를 간단하게 업데이트	<code>kubectl set image deployment/nginx nginx=nginx:1.16</code>	단순 변경에 적합하며, 일부 필드에 국한됨
kubectl patch	리소스의 일부를 JSON 또는 YAML 포맷의 패치로 수정	<code>kubectl patch deployment/nginx -p '{"spec":{"replicas": 3}}'</code>	복잡한 구조의 부분 수정이 가능하며, 다양한 패치 방식 지원



설명



SET 명령어 예제

이미지 정보를 수정 시, set명령어를 통해서 변경이 가능하다.

```
kubectl run --image=nginx my-httpd
```

```
kubectl get pod
```

```
my-httpd
```

```
kubectl set image pod/my-httpd my-httpd=image:alpine
```

```
kubectl set image pod/my-httpd my-httpd=docker.io/library/httpd:latest
```

```
kubectl describe pods/my-httpd
```

```
Image: docker.io/library/httpd:latest
```

```
kubectl label deployment my-deployment env=prod
```

```
deployment.apps/my-deployment labeled
```



PATCH 명령어 예제

특정 자원을 변경하기 위해서 PATCH로 변경이 가능하다. set명령어와 다르게 대다수 영역에 적용이 가능하다.

```
kubectl run --image=nginx --port=80 patch-nginx-pod
```

```
kubectl expose pod/patch-nginx-pod --type=NodePort
```

```
kubectl get svc
```

```
patch-nginx-pod
```

```
NodePort
```

```
kubectl patch svc patch-nginx-pod \
```

```
-p '{"spec":{"externalIPs":["192.168.0.194"]}}'
```



PATCH 명령어 예제

```
kubectl get svc
```

```
patch-nginx-pod, NodePort, 10.90.224.183, 192.168.0.194
```

```
kubectl patch svc patch-nginx-pod -p '{"spec":{"type":"LoadBalancer"}}'
```

```
kubectl get svc
```

```
patch-nginx-pod, LoadBalancer, 10.90.224.183, 192.168.0.194
```



EDIT 명령어 예제

위의 사용 방법이 어려운 경우 에디터 기반으로 수정이 가능하다.

```
kubectl edit deployment my-httpd
```

```
1. EDITOR=nano kubectl edit deployment my-httpd
```

```
2. KUBE_EDITOR=nano kubectl edit deployment my-httpd
```

```
echo "KUBE_EDITOR=nano" >> ~/.bash_profile
```



DEBUG

기본 명령어



DEBUG/LOG 설명

쿠버네티스에서 로그 및 디버깅하기 위해서 아래 하위 명령어 사용이 가능하다.

명령어	설명
debug	디버그 명령어는 노드 및 POD의 컨테이너에 직접적으로 상태 확인을 위해서 사용한다. 현재 사용하는 디버그는 노드의 루트 디스크 접근이 가능하기 때문에, 권한 설정에 따라서 노드의 루트 디스크 접근이 가능하다.
log	로그는 POD에서 발생한 기록 확인이 가능하다. POD기반으로 동작하는 cronjobs, daemonset 과 같은 자원에서 발생 기록 확인도 가능하다.



POD 디버그

```
kubectl debug -it my-httpd-6796cbbc4c-lr9qg --image=busybox \  
--target=my-httpd
```

Targeting container "my-httpd". If you don't see processes from this container it may be because the container runtime doesn't support this feature.

Defaulting debug container name to debugger-dvtvc.

If you don't see a command prompt, try pressing enter.

/ #



노드 디버그

```
kubectl debug node/node1.example.com -it \  
--image=docker.io/library/busybox:latest
```

Warning: metadata.name: this is used in the Pod's hostname, which can result in surprising behavior; a DNS label is recommended: [must not contain dots]

If you don't see a command prompt, try pressing enter

```
> / #
```



LOG 예제

```
kubectl logs -f my-httpd-6796cbbc4c-lr9qg
```

```
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 10.244.221.20. Set the 'ServerName' directive globally to suppress this message
```

```
AH00558: httpd: Could not reliably determine the server's fully qualified domain name, using 10.244.221.20. Set the 'ServerName' directive globally to suppress this message
```

```
[Sun Sep 26 13:54:00.609837 2021] [mpm_event:notice] [pid 1:tid 140142545593472] AH00489: Apache/2.4.49 (Unix) configured -- resuming normal operations
```

```
[Sun Sep 26 13:54:00.609935 2021] [core:notice] [pid 1:tid 140142545593472] AH00094: Command line: 'httpd -D FOREGROUND'
```



DELETE/DIFF/EXPLAN REPLACE/CP/EXEC

기본 명령어



명령어 정리

기본 명령어로 포함이 되지만, 실제 실무에서는 아주 가끔 사용한다.

1. **delete**: 자원을 제거 시 사용하는 명령어. 모든 자원에 대해서 제거가 가능하며 롤백 기능은 지원하지 않는다.
2. **diff**: YAML에 구성된 자원과 시스템 자원의 차이점에 대해서 확인한다.
3. **explain**: 이 명령어는 쿠버네티스에서 제공하는 명령어 사용방법에 대해서 설명한다.
4. **replace**: YAML에 등록된 내용으로 자원을 변경한다.
5. **cp**: 외부 파일을 POD안으로 복사한다.
6. **exec**: 컨테이너 내부의 바이너리를 실행한다. 이를 통해서 셸 구동이 가능하다.



DELETE 명령어 예제

```
kubectl delete pod -l run=my-nginx
```

```
kubectl delete service my-nginx
```

```
kubectl delete pods --all
```

```
kubectl delete all --all
```

```
kubectl get pods
```

```
kubectl delete -f my-httpd.yaml
```



DIFF 명령어 예제

```
kubectl diff -f apache-demo-create.yaml
```

```
- replicas: 2
```

```
+ replicas: 1
```



EXPLAIN 명령어 예제

```
kubectl explain pod
```

```
KIND:      Pod
```

```
VERSION:   v1
```

```
DESCRIPTION:
```

```
    Pod is a collection of containers that can run on a host. This resource is
```

```
    created by clients and scheduled onto hosts.
```

```
FIELDS:
```

```
    apiVersion    <string>
```



REPLACE 명령어 예제

```
kubectl run --image=nginx -l app=test -o=yaml --dry-run=client replace-nginx-pod > replace-nginx-pod.yaml
```

```
kubectl create -f replace-nginx-pod.yaml
```

```
metadata:
```

```
name: my-httpd
```

```
labels:
```

```
  type: test
```

```
  system: linux
```



REPLACE 명령어 예제

```
kubectl replace -f apache-demo-create.yaml
```

```
kubectl replace -f apache-demo-create.yaml --force
```

```
kubectl get pod
```

```
replace-nginx-pod, 16s
```



CP 명령어 예제

```
kubectl run --image=nginx nginx --port=80
```

```
kubectl get pod
```

```
nginx
```

```
kubectl describe pod/nginx
```

```
kubectl exec -it nginx -- bash
```

```
cat <<EOF> index.html
```

```
Hello This is my-nginx
```

```
EOF
```

```
kubectl cp index.html nginx:/usr/share/nginx/html/index.html
```



EXEC 명령어 예제

```
kubectl exec my-nginx -- ls /usr/share/nginx/html
```

```
kubectl exec -it my-nginx bash
```



확장 명령어

kubectl



Helm

확장 명령어



설명

쿠버네티스 패키지 관리자는 보통 두 가지를 많이 활용한다.

1. Kustomize

2. Helm

현재 서비스 패키징은 Kustomize보다는 HelmChart기반으로 더 많이 선호하고 있다. Kustomize는 쿠버네티스 1.14버전부터 도입이 되었다.

Helm은 쿠버네티스 1.6버전부터 사용 되었다. Helm v1/v2에서 2019년도 v3로 전환, 지금까지 사용하고 있다. v2와 v3는 생각보다 큰 차이는 없지만, 보안상 문제로 Tiller와 같은 기능을 제거하였다. Helm은 Kustomize와 제일 큰 차이점은 외부도구 및 명령어를 사용해야 한다.



표준 패키지 관리자 설명

대표적인 패키지 관리자의 차이점은 다음과 같다.

측면	kubectl 기반 설치 (YAML/Kustomize)	Helm 기반 설치
장점	단순하고 선언적인 구성 방식	패키지화된 차트로 재사용성 및 배포 자동화
	별도의 도구 설치 필요 없음	템플릿을 통한 커스터마이징 용이
	기본 기능에 충실함	롤백 및 버전 관리 지원
단점	복잡한 설치 및 배포 구성은 어려움 의존성 및 구성이 번거로울 수 있음 릴리즈 추적이 되지 않음	Helm 차트의 학습 곡선 존재
		차트 유지 관리 및 업데이트가 복잡할 수 있음



확장 패키지 관리자

현재는 대다수 클러스터가 OLM과 Helm기반으로 구성이 된다.

- 클러스터: HelmChart
- 서비스: OLM

도구	설명
Carvel	(이전 k14s) 도구 모음으로, ytt(템플릿 도구), kapp(애플리케이션 배포 및 업데이트 도구), vendir(외부 의존성 관리) 등 다양한 유틸리티를 포함
	복잡한 애플리케이션 구성에 적합
Operator Lifecycle Manager	Operator 패키지 관리 도구로, 쿠버네티스 클러스터에서 Operator의 배포, 업그레이드 및 관리를 자동화
	Kubernetes Operator 기반 애플리케이션에 특화됨



HELM 자원 구성

Helm에서는 다음과 같은 자원을 지원하고 있다. Helm은 Healing 기능이 없기 때문에, 자원이 설치 혹은 동작 중에 문제가 발생하면 복구/복원 기능을 제공하지 않는다.

이름	설명
repo	helm에서 제공하는 애플리케이션 설정파일 가지고 있는 저장소 위치. 최소 한 개의 저장소가 등록이 되어야, 서비스 확장 혹은 패키지 배포가 가능하다.
chart	helm에서 사용하는 패키지 이름. chart를 통해서 서비스나 혹은 프로그램을 패키징하여 클러스터에 배포 및 구성한다.



설치(HELM)

설치는 매우 간단하다. 먼저 Helm 명령어를 설치한다. 몇몇 배포판은 패키지로 지원하기도 한다. 앞에서 사용한 Ingress, LoadBalancer를 Helm기반으로 설치한다.

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |  
bash  
helm version  
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx  
helm repo update  
helm install ingress-nginx ingress-nginx/ingress-nginx --namespace ingress-  
nginx --create-namespace  
kubectl get pods -n ingress-nginx
```

NAME	READY	STATUS	RESTARTS	AGE
ingress-nginx-controller	1/1	Running	0	2m

계속



설치(INGRESS)

앞에서 사용한 **Ingress**를 **Helm**기반으로 설치한다.

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install ingress-nginx ingress-nginx/ingress-nginx --namespace ingress-
nginx --create-namespace
kubectl get pods -n ingress-nginx
```

NAME	READY	STATUS	RESTARTS	
AGEingress-nginx-controller	1/1	Running	0	2m



설치(METALLB)

LoadBalancer, MetalLB를 Helm기반으로 설치한다.

```
helm repo add metallb https://metallb.github.io/metallb
helm repo update
helm install metallb metallb/metallb --namespace metallb-system --create-namespace
```



설치(METALLB)

아래와 같이 구성이 되었는지 확인한다. 자세한 구성 방법은 뒤에서 더 다루도록 한다.

```
kubectl get pods -n metallb-system
```

NAME	READY	STATUS	RESTARTS	AGE
speaker	1/1	Running	0	2m
controller	1/1	Running	0	2m



자원 미터링

확장 명령어



미터링 설명

쿠버네티스에서 기본적으로 제공하는 미터링 서비스는 metrics라는 서비스가 유일하다. 하지만, 이 서비스는 모니터링 용도가 아닌, HPA, VPA와 같은 서비스를 활용하기 위해서 사용한다.

구성 요소	설명	용도 및 특징
Metrics Server	클러스터 내 노드와 Pod의 CPU, 메모리 사용량 등 리소스 메트릭을 집계하는 경량 서비스	Horizontal Pod Autoscaler(HPA)와 Vertical Pod Autoscaler(VPA) 등 자동 스케일링에 사용됨. 클러스터 내부에서 실시간 리소스 사용량 제공
kube-state-metrics	Kubernetes 오브젝트(Deployment, Pod, Node 등)의 상태 정보를 메트릭 형태로 제공	클러스터 상태 모니터링, 경보(Alerting) 및 시각화(Grafana 등)와 연동하여 클러스터 전반의 상태를 모니터링하는 데 활용됨



미터링 설명

위의 내용 계속 이어서...

구성 요소	설명	용도 및 특징
cAdvisor	각 노드에서 컨테이너의 CPU, 메모리, 디스크 I/O 등 세부 리소스 사용량을 수집하는 에이전트	노드 단위의 컨테이너 메트릭을 수집하며, Metrics Server와 함께 사용되어 클러스터 전체의 리소스 사용량 파악
Prometheus	오픈소스 모니터링 시스템으로, 다양한 메트릭(애플리케이션, 인프라, 클러스터 상태 등)을 스크랩하고 저장	Alerting 및 시각화(Grafana 연동) 기능 제공. Metrics Server, kube-state-metrics 등 다양한 데이터 소스를 통합하여 모니터링 환경 구축 가능



설명

metrics-server는 가급적이면 HelmChart기반으로 설치를 권장한다. 그렇지 않는 경우, 버전관리가 어렵다.



메트릭 서비스 설치

표준 설치에 아래와 같이 보통 가이드 한다. 이와 같은 방식으로 설치하면 올바르게 구성이 되지 않는다.

```
helm repo add metrics-server https://kubernetes-sigs.github.io/metrics-server/
```

```
helm install metrics-server metrics-server/metrics-server \
--namespace=kube-system
```

```
helm list -n kube-system
```

```
metrics-server
```



메트릭 서비스 설치

수동으로 TLS설정 부분 변경이 가능하다.

```
kubectl edit -n kube-system deployments.apps metrics-server
```

```
spec:
```

```
  containers:
```

```
  - args:
```

```
    - --cert-dir=/tmp
```

```
    - ---secure-port=4443
```

```
    - --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
```

```
    - --kubelet-use-node-status-port
```

```
    - --kubelet-insecure-tls
```

계
속



미터링

HelmChart Value에 적용하는 경우, 아래와 같이 적용이 가능하다.

```
vi values.yaml
```

```
defaultArgs:
```

- --cert-dir=/tmp
- --kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname
- --kubelet-use-node-status-port
- --metric-resolution=15s
- --kubelet-insecure-tls

```
helm upgrade --install metrics-server metrics-server/metrics-server --  
namespace=kube-system -f values.yaml
```



미터링

설치 혹은 환경에 따라서 설치 방법이 다르다. 미터링 환경이 큰 경우(HPA/VPA), H/A기능 기반으로 Metrics 서비스를 구성해야 한다.

형식	조정
HelmChart	Replicas: 1에서 이 이상 값으로 values.yaml를 수정한다.
kubectl	1.12+경우에는 <code>kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/high-availability-1.21+.yaml</code> 명령어로 설치가 가능하다.



미터링

문제 없이 구성이 되면 보통 다음과 같이 화면에 출력이 된다.

```
kubectl get pods -n kube-system -l app.kubernetes.io/instance=metrics-server
```

```
metrics-server-fb889bc84-22zkc    1/1    Running    0    4m21s
```

```
kubectl top nodes
```

```
node1.example.com    56m    2%    1633Mi    45%
```



사용자 생성 및 구성

확장 명령어



설명

쿠버네티스는 기본적으로 사용자라는 개념은 문자열로 존재하고, 실제 사용자는 인증서 기반으로 구분 및 처리한다. 이러한 이유로 처음 접근하는 쿠버네티스 사용자는 사용자 개념으로 많은 부분에 대해서 혼란이 발생한다.

특히, 퍼블릭 클라우드에서 전환한 쿠버네티스 관리자 경우에는 IAM 때문에 더욱 이러한 문제가 발생한다. 이러한 문제를 해결하기 위해서 **Scratch from CLI**를 통해서 직접 구성한다.

1. CLI 기반으로 생성 및 구성
2. KAM(Kubernetes Account Manager)를 통한 계정 생성 및 관리



KAM(Kubernetes Account Manager)

간단하게 사용자 구성 및 관리를 위해서 별도로 KAM라는 명령어 제공. 이를 통해서 간단하게 사용자 생성 및 구성이 가능하다.

보통 쿠버네티스 클러스터를 운영하면 아래와 같이 보조 명령어를 만들어서 배포 및 운영한다.

```
# 사용자 생성 및 네임스페이스/역할 할당
kam user create gookhyun -n dev --role view
# 사용자 네임스페이스 및 역할 변경
kam user set gookhyun -n dev --role edit
# 사용자 제거
kam user delete gookhyun
# 사용자 목록(기존 쿠버네티스 목록은 미포함)
kam user list
```



일반적인 사용자 생성 및 구성

일반적으로 쿠버네티스에서 사용자는 다음과 같이 생성 및 관리한다.

```
kubectl create namespace user1-namespace
namespace/user1-namespace created
openssl genrsa -out user1.key 2048
openssl req -new -key user1.key -out user1.csr -subj "/CN=user1/O=group1"
openssl x509 -req -in user1.csr -CA /etc/kubernetes/pki/ca.crt \
-CAkey /etc/kubernetes/pki/ca.key \
-CACreateserial -out user1.crt -days 365
```



사용자 인증서 컨텍스트 등록

등록된 사용자를 인증서에 등록한다.

```
kubectl config set-credentials user1 --client-certificate=user1.crt \  
--client-key=user1.key
```

User "user1" set.

```
kubectl config set-context user1-context \  
--cluster=kubernetes --namespace=user1-namespace --user=user1
```

Context "user1-context" created.



추가 인증서를 통해서 구성한 경우

아래와 같이 CSR 인증서를 등록 및 허가해야 한다. CSR 인증서는 다음 슬라이드에서 설명.

```
kubectl get csr
user1, pending
kubectl certificate approve <USERNAME>
kubectl get csr
user1, Approved, Issued
```



CSR 인증서는...

CSR 인증서는 아래와 같이 생성 및 구성이 가능하다.

```
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: user1
spec:
  request: <base64 csr>
  signerName: kubernetes.io/kube-apiserver-client
  usages:
    - client auth
```



RBAC 기반으로 자원 관리 및 접근제한

확장 명령어



ROLE구성

생성된 사용자를 사용하기 위해서 자원에 할당해야 한다. 자원에 할당하기 위해서는 다음과 같이 작업을 수행한다.

```
kubectl apply -f - <<EOF
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: user1-namespace
  name: user1-role
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "create", "delete"]
EOF
kubectl apply -f user1-role.yaml
role.rbac.authorization.k8s.io/user1-role created
```



ROLE구성

사용자와 그리고 생성된 네임스페이스를 연결한다. 이때, roleBinding를 사용해서 구성한다.

```
kubectl apply -f - <<EOF
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: user1-namespace
  name: user1-rolebinding
subjects:
- kind: User
  name: user1
  apiGroup: rbac.authorization.k8s.io
EOF
```



ROLE구성

앞에 내용 계속 이어서...

```
roleRef:
  kind: Role
  name: user1-role
  apiGroup: rbac.authorization.k8s.io
```

```
kubectl apply -f user1-roleBinding.yaml
rolebinding.rbac.authorization.k8s.io/user1-rolebinding created
```



사용자 및 권한 확인하기

생성된 사용자 및 네임스페이스 자원이 올바르게 바인딩이 되었는지 확인한다. 앞에서 만든 RBAC는 POD만 허용하였다. 확인 시, `--context` 라는 옵션을 통해서 임의로 명시가 가능하다.

```
kubectl --context=user1-context get pods
```

```
No resources found in user1-namespace namespace.
```

```
kubectl --context=user1-context get deployments
```

```
Error from server (Forbidden): deployments.apps is forbidden: User "user1" cannot list resource "deployments" in API group "apps" in the namespace "user1-namespace"
```

```
kubectl get deployments
```

```
No resources found in default namespace.
```



자원 접근을 위한 역할 확장(YAML)

기존 내용은 POD만 허용했기 때문에, user1-namespace의 모든 자원에 접근이 가능하도록 수정한다. 아래와 같이 내용을 수정한다.

```
vi user1-role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: user1-namespace
  name: user1-role
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["*"]
```



자원 접근을 위한 역할 확장(적용)

자원 접근이 올바르게 되는지 적용 및 확인한다.

```
kubectl apply -f user1-role.yaml
role.rbac.authorization.k8s.io/user1-role configured
kubectl --context=user1-context get all -n user1-namespace
No resources found in user1-namespace namespace.
kubectl --context=user1-context create deployment nginx --image=nginx -n
user1-namespace
deployment.apps/nginx created
kubectl --context=user1-context expose deployment nginx --port=80 --target-
port=80 --type=ClusterIP -n user1-namespace
service/nginx exposed
```



결과

결과는 다음과 같이 출력이 된다.

```
kubectl get pod,svc,deployment,rs
```

NAME	READY	STATUS	RESTARTS	AGE
pod/nginx-6799fc88d8-xxxxx	1/1	Running	0	10s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/nginx	ClusterIP	10.96.12.34	<none>	80/TCP	5s

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/nginx	1/1	1	1	10s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/nginx-6799fc88d8	1	1	1	10s



KUBERNETES-ADMIN

쿠버네티스에서 기본적으로 제공하는 계정은 다음과 같다. 매뉴얼 기준으로 바닐라 버전은 다음과 같이 구성이 되어 있다.

역할이름	역할연결	설명
cluster-admin	system:masters group	슈퍼유저(super-user) 권한을 부여하여 모든 리소스에 대해 모든 작업 수행. ClusterRoleBinding에 사용될 경우, 클러스터 전체 및 모든 네임스페이스의 모든 리소스에 권한 부여 RoleBinding에 사용될 경우, 바인딩된 네임스페이스 내의 모든 리소스(해당 네임스페이스 자체 포함)에 대한 완전한 제어 권한.



KUBERNETES-ADMIN

쿠버네티스에서 기본적으로 제공하는 계정은 다음과 같다. 매뉴얼 기준으로 바닐라 버전은 다음과 같이 구성이 되어 있다.

역할이름	역할연결	설명
admin	None	관리자(admin) 권한을 부여하며, RoleBinding을 통해 네임스페이스 내에서 사용할 목적으로 설계. RoleBinding에 사용될 경우, 네임스페이스 내 대부분의 리소스에 대해 읽기/쓰기(read/write) 권한을 부여하며, 해당 네임스페이스 내에서 Role 및 RoleBinding을 생성할 수 있는 권한도 포함. 단, 이 Role은 리소스 쿼터(resource quota) 또는 네임스페이스 자체에 대한 쓰기 권한은 부여하지 않음. 또한, Kubernetes v1.22 이상에서 생성된 클러스터에서는 EndpointSlices (또는 Endpoints) 에 대한 쓰기 권한도 부여하지 않음.

계
속



KUBERNETES-ADMIN

앞에 내용을 이어서...

역할이름	역할연결	설명
edit	None	네임스페이스 내 대부분의 오브젝트에 대해 읽기/쓰기(read/write) 권한부여. 이 Role은 Role이나 RoleBinding을 조회하거나 수정하는 권한 없음. Secret에 접근하거나 해당 네임스페이스 내의 어떤 ServiceAccount로도 Pod을 실행할 수 있는 권한 허용 이 Role을 통해 해당 네임스페이스 내의 ServiceAccount가 가진 API 접근 권한 수준을 간접적으로 접근
view	None	네임스페이스 내 대부분의 오브젝트를 볼 수 있는 읽기 전용(read-only) 권한 이 Role은 Role이나 RoleBinding을 조회할 수 있는 권한은 없음 Secret을 조회할 수 있는 권한도 포함되어 있지 않음 접근이 필요한 경우 SA계정을 통해서 접근이 가능

계
속



새로운 클러스터 관리자 생성

기존에 사용한 쿠버네티스 관리자 계정 **kubernetes-admin** 계정을 **k8s-admin**으로 변경한다. 앞에서 사용한 명령어를 활용한다. 기본적으로 부트스트랩 계정은 설치 후 폐쇄한다.

```
openssl genrsa -out k8s-admin.key 2048
openssl req -new -key k8s-admin.key -out k8s-admin.csr -subj "/CN=k8s-admin/O=system:masters"
openssl x509 -req -in k8s-admin.csr -CA /etc/kubernetes/pki/ca.crt -CAkey /etc/kubernetes/pki/ca.key -CAcreateserial -out k8s-admin.crt -days 365
kubectl config set-cluster kubernetes \
  --certificate-authority=/etc/kubernetes/pki/ca.crt \
  --server=https://10.0.2.5:6443 \
  --embed-certs=true \
  --kubeconfig=k8s-admin.kubeconfig
```

계속



새로운 클러스터 관리자 인증서

아래와 같이 CA/PKI키를 설정 및 구성한다.

```
kubectl config set-credentials k8s-admin \  
  --client-certificate=k8s-admin.crt \  
  --client-key=k8s-admin.key \  
  --embed-certs=true \  
  --kubeconfig=k8s-admin.kubeconfig  
kubectl config set-context k8s-admin@kubernetes \  
  --cluster=kubernetes \  
  --user=k8s-admin \  
  --kubeconfig=k8s-admin.kubeconfig
```



새로운 클러스터 관리자 계정 설정

위의 구성이 올바르게 되면, context 변경하여 올바르게 동작이 되는지 clusterrole를 관리자 권한으로 할당한다.

```
# kubectl config use-context k8s-admin@kubernetes \
--kubeconfig=k8s-admin.kubeconfig
# kubectl create clusterrolebinding k8s-admin-binding \
--clusterrole=cluster-admin --user=k8s-admin
clusterrolebinding.rbac.authorization.k8s.io/k8s-admin-binding created
```



새로운 클러스터 관리자 계정 확인 및 제거

올바르게 구성이 되었는지 확인한다.

```
# kubectl delete clusterrolebinding kubernetes-admin
clusterrolebinding.rbac.authorization.k8s.io "kubernetes-admin" deleted
# kubectl create clusterrolebinding k8s-admin-binding --
clusterrole=cluster-admin --user=k8s-admin
clusterrolebinding.rbac.authorization.k8s.io/k8s-admin-binding created
# kubectl get clusterrolebinding | grep admin
```



서비스 계정

확장 명령어



서비스 계정 설명

서비스 계정은 직접적으로 자원에 할당 및 접근하는 게 아니라, 특정 계정을 통해서 작업을 대신 수행한다. 서비스 계정은 다음과 같이 기능을 제공한다.

기능	설명
Pod에 인증 정보 제공	Pod가 쿠버네티스 API 서버와 상호작용할 때 사용하는 계정
RBAC(Role-Based Access Control)과 연동 가능	특정 역할(Role)을 할당하여 접근 권한을 제한 가능
네임스페이스별로 관리 가능	네임스페이스 단위로 서비스 계정을 생성하고 관리 가능
자동 토큰 생성 및 마운트	기본적으로 토큰이 Secret 리소스로 생성되며, Pod가 실행될 때 자동으로 마운트됨
Pod가 API 요청 시 신뢰할 수 있는 ID 제공	Pod가 API 요청을 보낼 때, 해당 서비스 계정을 인증 정보로 사용



계정 종류 비교

서비스 계정/사용자 계정 및 역할을 비교하면 아래와 같이 정리가 된다.

기능	ServiceAccount	User Account	Role & RoleBinding
대상	Pod (애플리케이션)	사람 (관리자, 개발자 등)	사용자 및 서비스 계정
용도	Pod가 쿠버네티스 API에 접근할 때	사용자가 kubectl이나 API를 사용할 때	특정 리소스에 대한 접근 권한을 관리
기본 제공 여부	네임스페이스마다 default 서비스 계정이 있음	기본적으로 존재하지 않음 (외부 인증 필요)	RBAC 설정을 해야 함
권한 설정	Role, ClusterRole과 연계하여 권한 관리 가능	RBAC을 통해 권한 부여 가능	특정 리소스(예: Pod, Secret, Node 등)에 대한 권한 부여

계속



계정 종류 비교

기능	ServiceAccount	User Account	Role & RoleBinding
토큰 사용 여부	Secret을 통해 토큰 제공 (JWT 기반)	일반적으로 패스워드 또는 인증 프로바이더 사용	자체적으로 토큰을 제공하지 않음
네임스페이스 제한 여부	네임스페이스 단위로 존재	클러스터 전역적으로 존재	Role은 네임스페이스 범위, ClusterRole은 클러스터 전체에 적용 가능



서비스 계정 생성

CLI기반으로 생성은 매우 간단하다. 아래와 같이 실행하면 NS/SA/POD가 생성이 된다.

```
kubectl create namespace test-sa
kubectl create serviceaccount test-sa-pod -n test-sa
kubectl create role test-sa-role \
--verb=get,list,watch \
--resource=pods,services,configmaps,secrets \
--namespace=test-sa
kubectl create rolebinding test-sa-role-binding \
--role=test-sa-role \
--serviceaccount=test-sa:test-sa-pod \
--namespace=test-sa
```



서비스 계정 활용(POD)

CLI기반으로 생성은 매우 간단하다. 아래와 같이 실행하면 NS/SA/POD가 생성이 된다.

```
kubectl run nginx-pod --image=nginx --namespace=test-sa --  
serviceaccount=test-sa-pod --port=80  
kubectl get namespace  
kubectl get serviceaccount -n test-sa  
kubectl get pod nginx-pod -n test-sa \  
-o jsonpath='{.spec.serviceAccountName}'  
test-sa-pod
```



YAML 기반으로 SA 구성

YAML 기반으로 SA 계정을 생성 및 구성 시, 아래 단계와 같이 작성 및 생성한다. 위의 명령어 기반이기 YAML으로 작성 및 배포하는 방법에 대해서 알아본다.



1단계 생성(NS)

네임스페이스를 YAML기반으로 생성한다. SA 계정은 최소 한 개의 네임스페이스와 연결이 되어야 한다.

```
vi test-sa.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: test-sa
kubectl apply -f my-service-account.yaml
```



2단계 생성(SA)

서비스 계정을 YAML으로 생성 및 구성한다. 앞서 이야기 하였지만 최소 한 개의 네임스페이스가 필요하다.

```
vi test-sa-pod.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: test-sa-pod
  namespace: test-sa
kubectl apply -f test-sa-pod.yaml
```



3단계 생성(POD)

올바르게 구성이 되었는지 POD 생성하여 확인한다. POD는 nginx-pod라는 이름으로 생성한다.

```
vi sa-nginx-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  namespace: test-sa
```



3단계 생성(POD)

앞에 내용 이어서...

```
spec:
  serviceAccountName: test-sa-pod
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
kubect! apply -f sa-nginx-pod.yaml
```



4단계 생성(ROLE)

서비스 계정이 사용할 역할을 생성 및 할당한다. 이때 할당되는 계정은 서비스 계정이다.

```
kubectl apply -f - <<EOF
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: test-sa
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
EOF
```

계
속



5단계 생성(ROLEBINDING)

역할을 네임스페이스와 연결한다. 이전과 다른 점은 ServiceAccount를 통해서 연결 한다.

```
kubectl apply -f - <<EOF
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: test-sa
subjects:
- kind: ServiceAccount
  name: test-sa-pod
  namespace: test-sa
EOF
```



생성(ROLEBINDING)

계속 이어서...

```
roleRef:  
  kind: Role  
  name: pod-reader  
  apiGroup: rbac.authorization.k8s.io  
EOF
```

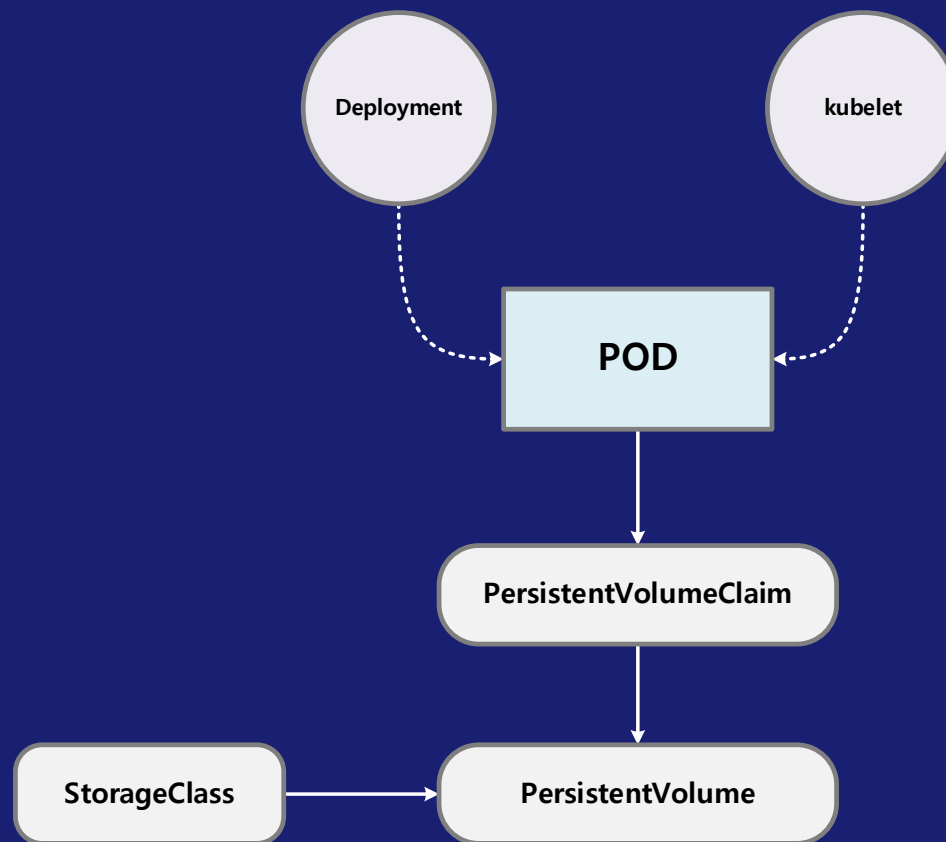


스토리지

확장 명령어



쿠버네티스 스토리지



설치 및 구성

설치 및 구성은 기본 값인 CHART NOVALUE를 사용해서 설치한다.



스토리지 소개

기능은 다음과 같다.

구성 요소	설명	주요 특징	사용 사례
StorageClass	동적 프로비저닝 시 사용할 스토리지의 속성(타입, 성능, 접근 모드 등)을 정의하는 리소스	PVC에 의해 참조됨 스토리지 정책 및 클래스 지정 다양한 백엔드지원	스토리지 자원 자동 생성, 정책 기반 관리
PersistentVolume	클러스터 관리자가 생성하거나 StorageClass를 통해 동적으로 프로비저닝되는 실제 스토리지 자원	클러스터 수준의 독립적 리소스 다양한 저장소 백엔드 지원 수동/동적 생성 가능	데이터 저장, 공유 스토리지 할당
PersistentVolume Claim	애플리케이션에서 필요한 스토리지 용량과 접근 모드를 요청하는 리소스	PV와 바인딩 요구 사항 명시 Pod에 스토리지 할당 시 사용됨	Pod에 스토리지 할당, 데이터 보존 및 공유



NFS 서버 설치 및 구성

NFS서비스를 구성한다. 모든 서버에는 nfs-utils 패키지가 설치가 되어 있어야 한다.

```
dnf install nfs-utils -y
setenforce 0
getenforce
mkdir -p /nfs
cat <<EOF> /etc/exports
/nfs *(rw,no_root_squash)
EOF
```



NFS 서버 설치 및 구성

위의 내용 계속 이어서...

```
exportfs -avrs  
systemctl enable --now nfs-server  
semanage fcontext -a -t public_content_rw_t '/nfs(/.*)?'  
restorecon -RFvv /nfs
```



CHART 설치(NO VALUE)

NFS 기본 값으로 설치하는 과정. 이 랩에서는 이 옵션을 사용해서 설치한다.

```
helm repo add nfs-subdir-external-provisioner \  
https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner/  
helm repo update  
showmount -e <NFS_SERVER>  
helm install nfs-provisioner nfs-subdir-external-provisioner/nfs-subdir-  
external-provisioner \  
  --namespace nfs-provisioner --create-namespace \  
  --set nfs.server=10.0.2.5 \  
  --set nfs.path=/nfs
```



CHART 설치(NO VALUE)

위의 내용 계속 이어서...

```
NAME: nfs-provisioner
LAST DEPLOYED: Wed Mar  9 14:20:30 2025
NAMESPACE: nfs-provisioner
STATUS: deployed
REVISION: 1
TEST SUITE: None
```



CHART 설치(NO VALUE)

아래 명령어로 올바르게 설치가 되었는지 확인한다.

```
kubectl get pods -w -n nfs-provisioner  
kubectl get sc
```



CHART 설치(VALUE)

Chart기반으로 role 및 서비스 계정 구성하는 경우 다음과 같이 `values.yaml`파일을 생성한다.

```
vi values.yaml
serviceAccount:
  create: true
  name: nfs-provisioner-sa
  annotations: {} # 필요시 여기에 추가 주석 설정
rbac:
  create: true
  rules:
    - apiGroups: [""]
      resources: ["persistentvolumes"]
      verbs: ["get", "list", "watch", "create", "delete"]
```

계
속



CHART 설치(VALUE)

앞에 내용 계속 이어서...

```
- apiGroups: [""]  
  resources: ["persistentvolumeclaims"]  
  verbs: ["get", "list", "watch", "update"]  
- apiGroups: ["storage.k8s.io"]  
  resources: ["storageclasses"]  
  verbs: ["get", "list", "watch"]
```



CHART 설치(VALUE)

앞에 내용 계속 이어서...

```
nfs:  
  server: 192.168.1.100  
  path: /exported/path  
storageClass:  
  name: nfs-storage  
  reclaimPolicy: Delete  
  volumeBindingMode: Immediate
```



CHART 설치(VALUE)

values.yaml 파일 생성 후, 아래 명령어 CSI NFS 드라이버를 설치 및 구성한다.

```
helm install nfs-provisioner nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
--namespace nfs-provisioner \
--create-namespace -f values.yaml
NAME: nfs-provisioner
LAST DEPLOYED: Wed Mar  9 14:20:30 2025
NAMESPACE: nfs-provisioner
STATUS: deployed
REVISION: 1
TEST SUITE: None
```



CHART 설치(VALUE)

최종적으로 잘 구성이 되었는지 확인한다. 이 기반으로 PV/PVC를 구성 및 생성한다. 또한, SC를 통해서 PV없이 PVC를 생성한다.

```
kubectl get serviceaccount -n nfs-provisioner  
kubectl get clusterrole,clusterrolebinding | grep nfs-subdir-external-  
provisioner  
kubectl get sc
```



KUBERNETES STORAGE 유형

PV는 스토리지 클래스를 사용하지 않고 스토리지 서버 혹은 장비에 연결하는 일종의 드라이버 역할을 한다. 앞서 스토리지 클래스를 구성했기 때문에, PV를 통해서 PVC를 생성 및 구성한다.

구성 요소	설명	특징 및 사용 사례
PersistentVolume (PV)	클러스터 내에서 관리되는 실제 스토리지 자원입니다. 관리자가 생성하거나 동적 프로비저닝(StorageClass 사용)으로 생성됩니다.	- 클러스터 전역 자원 - NFS, iSCSI, 클라우드 스토리지 등 다양한 백엔드를 지원
- 관리자가 직접 생성할 수도 있음
PersistentVolumeClaim (PVC)	Pod가 스토리지 사용을 요청하는 리소스입니다. 사용자가 원하는 용량, 접근 모드, StorageClass 등의 요구사항을 명시하며, PV와 바인딩됩니다.	- Pod의 스토리지 요청을 추상화 - 자동으로 PV와 매칭되어 바인딩 됨 - 애플리케이션 데이터 저장에 사용 됨



KUBERNETES STORAGE

좀 더 쉽게 설명하면, 아래와 같이 비유가 가능하다.

참고로, 쿠버네티스에서 제공하는 PV/PVC/STORAGE CLASS는 직접적인 백 엔드 저장소를 지원 및 제공하지 않는다.

용어	설명	비유
PVC (Persistent Volume Claim)	Pod가 필요한 스토리지를 요청하는 역할	"내가 사용할 디스크 주세요!"
PV (Persistent Volume)	실제 저장소 (관리자가 미리 만들어 놓음)	"여기 사용할 디스크 있어!"



PV(STORAGE PROVIDER)

NFS기반으로 연결하는 경우 다음과 같이 PV를 구성한다.

```
cat <<EOF> pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
  labels:
    type: nfs
```



PV(STORAGE PROVIDER)

NFS기반으로 연결하는 경우 다음과 같이 PV를 구성한다.

```
spec:
  storageClassName: "nfs-client"
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  nfs:
    server: storage.example.com
    path: "/nfs/manual-pv"
```

EOF

```
kubectl apply -f pv.yaml
persistentvolume/static-pv created
```



PVC(스토리지 클래스) 구성

생성한 PV에 PVC를 연결한다.

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  storageClassName: "nfs-client"
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

EOF



PVC(스토리지 클래스) 연결 확인

PVC가 구성이 되면 대략 아래와 같이 화면에 출력이 된다.

```
kubectl get pvc
```

```
nfs-pvc      Bound      pvc-12345678-90ab-cdef-1234-    1Gi      RWX
```



디스크 POD에 연결하기

POD에 외부 디스크를 구성하기 위해서는 Deployment로 구성 및 선언해야 한다.

```
kubectl apply -f - <<EOF
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-with-nfs
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx-nfs
```



계
속

PVC(DEPLOY)

```
template:
  metadata:
    labels:
      app: nginx-nfs
  spec:
    containers:
      - name: nginx
        image: nginx:latest
        volumeMounts:
          - name: nfs-volume
            mountPath: /usr/share/nginx/html
```



PVC(DEPLOY)

이어서...

```
volumes:  
  - name: nfs-volume  
    persistentVolumeClaim:  
      claimName: nfs-pvc
```

EOF

```
kubectl apply -f nginx-with-nfs.yaml  
deployment.apps/nginx-with-nfs created
```



스케일링 및 서비스(PDB)

확장 명령어



배포 전략

쿠버네티스 바닐라 버전 기반으로 배포전략은 다음과 같이 제공이 된다. 쿠버네티스 서비스 아키텍처는 무 중단 서비스를 구현이다.

배포 전략	설명	사용 예시
Rolling Update	기존 Pod를 순차적으로 교체 (무중단 배포)	<code>kubectl set image deployment/nginx nginx=nginx:latest</code>
Recreate	기존 Pod를 모두 삭제한 후 새로운 Pod를 생성	<code>kubectl rollout restart deployment/nginx</code>
Blue-Green Deployment	새 버전의 배포 그룹을 생성 후 트래픽을 전환	<code>kubectl apply -f blue-green.yaml</code>
Canary Deployment	일부 사용자만 새 버전을 사용하도록 설정	<code>kubectl apply -f canary.yaml</code>



배포 전략(lifecycle)

쿠버네티스 애플리케이션 종료는 보통 두 가지 방식이 있다.

- 런타임에서 종료(return, exit 0)
- 애플리케이션 종료 후, 컨테이너 종료(return, exit 0)

컨테이너 플랫폼에서는 한 개 이상의 애플리케이션들이 실행된다.

종종 컨테이너 런타임이 종료 신호(SIGTERM)를 받으면 애플리케이션보다 빠르게 종료하는 경우가 있다. 이러한 이유로 종종 프로세스 상태가 D, Z으로 변경이 되는 경우가 있다.

이를 방지하기 위해서 다음과 같은 코드를 추가 하기도 한다.

```
lifecycle:
  preStop:
    exec:
      command: ["/bin/sh", "-c", "sleep 10"]
```



배포 전략(PDB)

HPA와 같은 스케일링 기반으로 서비스가 구성되는 경우, 스케줄러가 모든 노드 상태를 실시간으로 알 수 없기 때문에 종종, 예상된 숫자보다 더 작게 스케일 아웃(Scale Out)이 되는 경우가 있다.

이러한 이유로, POD개수를 유지할 수 있도록 명시해야 한다.

쿠버네티스 버전 1.21+에서 이러한 부분을 해결하기 위해서 PDB(PodDisruptionBudget) 기능이 추가가 되었다. 이 기능이 적용되는 다음과 같은 자원이다. 셀렉터 기반으로 동작한다.

- Deployment
- ReplicationController
- ReplicaSet
- StatefulSet



배포 전략 테스트(deploy)

테스트 하기 위해서 아래와 같이 nginx POD를 생성한다.

```
kubectl apply -f - <<EOF
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: pdb-nginx
```

```
  labels:
```

```
    app: pdb-nginx
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: pdb-nginx
```



배포 전략 테스트(deploy)

```
template:  
  metadata:  
    labels:  
      app: pdb-nginx  
  spec:  
    containers:  
      - name: nginx  
        image: nginx:1.27
```



배포 전략 테스트(deploy)

```
ports:
- containerPort: 80
readinessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 3
  periodSeconds: 5
```



배포 전략 테스트(deploy)

위의 내용 계속 이어서...

```
livenessProbe:
  httpGet:
    path: /
    port: 80
  initialDelaySeconds: 10
  periodSeconds: 10
```

EOF

```
kubectl apply -f pdb-nginx.yaml
kubectl get -f pdb-nginx.yaml -w
```



배포 전략(PDB)

이 기능을 적용하면 다음과 같다.

```
kubectl apply -f - <<EOF
```

```
apiVersion: policy/v1
```

```
kind: PodDisruptionBudget
```

```
metadata:
```

```
  name: pdb-nginx
```

```
spec:
```

```
  minAvailable: 2
```

```
  selector:
```

```
    matchLabels:
```

```
      app: pdb-nginx
```

```
EOF
```

```
kubectl apply -f pdb-nginx.yaml
```



배포 전략(PDB)

올바르게 구성이 되었는지 확인 및 검증한다.

- Min available: 2
- Selector: app=nginx
- Allowed disruptions

```
kubectl apply -f pdb-nginx.yaml
```

```
kubectl get -f pdb-nginx.yaml
```

```
kubectl describe -f pdb-nginx.yaml
```

```
kubectl get pod -l app=nginx -o wide
```

```
kubectl get pdb nginx-pdb -o wide
```

```
kubectl get pod -l app=nginx -o wide
```

```
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
```



HPA/VPA

확장 명령어



테스트 POD 생성

테스트를 위해서 아래와 같이 작성한다.

```
kubectl apply -f - <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: scale-nginx
  labels:
    app: scale-nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: scale-nginx
```



테스트 POD 생성

위의 내용 계속...

```
template:
  metadata:
    labels:
      app: scale-nginx
  spec:
    containers:
      - name: sample-container
        image: busybox
        command: ["/bin/sh"]
        args:
          - -c
          - "while true; do yes > /dev/null; sleep 1; done"
```



테스트 POD 생성

최종적으로 자원 크기를 제한한다. 앞서 이야기 하였지만 쿠버네티스는 CPU, MEM지원한다.

```
resources:  
  requests:  
    cpu: "100m"  
    memory: "128Mi"  
  limits:  
    cpu: "200m"  
    memory: "256Mi"
```

EOF



HPA 구성 및 생성

애플리케이션 실행 시, 워크로드에 따라서 POD의 개수를 늘리고 줄이기 위해서 HPA기능을 구성해야 한다. HPA는 Horizontal Pod Autoscaler의 약자다. 코드는 다음과 같이 작성이 가능하다.

```
kubectl apply -f - <<EOF
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nginx-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: scale-nginx
  minReplicas: 1
```

계
속



HPA 구성 및 생성

CPU사용율이 50%가 넘어가면 POD 확장을 시작한다.

```
maxReplicas: 5
metrics:
- type: Resource
  resource:
    name: cpu
    target:
      type: Utilization
      averageUtilization: 50
```

EOF



HPA 개수 조절 및 확인

HPA와 같은 스케일링 기반으로 서비스가 구성되는 경우, 스케줄러가 모든 노드 상태를 실시간으로 알 수 없기 때문에 종종, 예상된 숫자보다 더 작게 스케일 아웃(Scale Out)이 되는 경우가 있다. 이러한 이유로, POD개수를 유지할 수 있도록 명시해야 한다. 앞에 내용에 추가한다.

```
maxReplicas: 10
```

```
metrics:
```

```
- type: Resource
```

```
  resource:
```

```
    name: cpu
```

```
    target:
```

```
      type: Utilization
```

```
      averageUtilization: 50
```

```
kubectl edit nginx-hpa
```



VPA 설치

VPA는 아직 공식적으로 지원하지 않는다. YAML 매니 페스트로 설치하는 지원한다. VPA를 지원하는 쿠버네티스 미 들웨어는 현재 다음과 같다.

1. 쿠버네티스 바닐라
2. 레드햇 오픈시프트
3. 수세 RKE2

```
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/autoscaler/vpa-release-
1.0/vertical-pod-autoscaler/deploy/vpa-v1-crd-gen.yaml
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/autoscaler/vpa-release-
1.0/vertical-pod-autoscaler/deploy/vpa-rbac.yaml
```



설치 확인

올바르게 설치가 되었으면 다음 명령어 VPA 자원 조회가 가능하다.

```
kubectl get vpa
```

NAME	MODE	CPU	MEM	PROVIDED	AGE
hamster-vpa		587m	250Mi	True	11m



테스트 자원 설치

테스트 자원은 VPA 사이트에서 햄스터(hamster)라는 이름으로 지원하고 있다.

```
kubectl apply -f
https://raw.githubusercontent.com/kubernetes/autoscaler/master/vertical-
pod-autoscaler/examples/hamster.yaml
kubectl get -f
https://raw.githubusercontent.com/kubernetes/autoscaler/master/vertical-
pod-autoscaler/examples/hamster.yaml
```



VPA 설정

VPA 설정은 보통 아래와 같다. VPA 햄스터에 있는 내용이다. 강사 설명 진행.

```
apiVersion: "autoscaling.k8s.io/v1"
kind: VerticalPodAutoscaler
metadata:
  name: hamster-vpa
spec:
  # recommenders:
  #   - name: 'alternative'
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: hamster
```



VPA 설정

VPA 설정은 보통 아래와 같다.

```
resourcePolicy:
  containerPolicies:
    - containerName: '*'
      minAllowed:
        cpu: 100m
        memory: 50Mi
      maxAllowed:
        cpu: 1
        memory: 500Mi
      controlledResources: ["cpu", "memory"]
```

이 영역에서 컨테이너가 사용이 가능한 최소/최대 CPU/MEM 크기를 명시한다.



배포 전략(VPA)

VPA가 적용이 올바르게 되는지 확인하기 위해서 HPA를 통해서 POD 개수를 늘리면 CPU/MEM가 VPA수치까지 사용하는지 확인한다.

```
kubectl top pod -l app=hamster  
kubectl scale deploy hamster --replicas=5  
kubectl top pod -l app=hamster
```



배포 전략(VPA)

VPA는 HPA와 반대로 수직적으로 자원을 조절한다. HPA, VPA자원은 뒤에서 좀 더 확인한다.

```
kubectl apply -f scale-vpa-busybox.yaml  
verticalpodautoscaler.autoscaling.k8s.io/sample-vpa created  
kubectl describe vpa vpa-busybox
```



롤링 업데이트

확장 명령어



롤링 업데이트

롤링 업데이트는 기존 서비스에 영향 없이 업데이트 및 릴리즈 하는 방법이다. 롤링 업데이트를 통해서 언제든지 사용자는 롤백이 가능하다. 기본적으로 다음과 같은 기능을 제공한다.

단계	명령어	설명
기존 Deployment 생성	<code>kubectl apply -f nginx-deployment.yaml</code>	nginx 1.19 버전 배포
현재 상태 확인	<code>kubectl get deployments</code>	배포된 Pod 확인
업데이트 적용	<code>kubectl set image deployment/nginx-deployment nginx=nginx:1.21</code>	nginx 버전 업데이트
업데이트 진행 확인	<code>kubectl rollout status deployment/nginx-deployment</code>	롤링 업데이트 진행 상황 확인
롤백 (되돌리기)	<code>kubectl rollout undo deployment/nginx-deployment</code>	이전 버전으로 복구



롤링 업데이트

롤링 업데이트를 사용하기 위해서 다음과 같이 구성이 가능하다. 간단하게 Deployment구성 후, 테스트 한다.

```
cat <<EOF> nginx-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
```



롤링 업데이트

롤링 업데이트를 사용하기 위해서 다음과 같이 구성이 가능하다. 간단하게 Deployment구성 후, 테스트 한다.

```
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: nginx:1.19
        ports:
          - containerPort: 80
```

EOF

```
kubectl apply -f nginx-deployment.yaml
```



롤링 업데이트

set 명령어를 통해서 버전을 변경한다. 버전이 변경이 되면, 즉시 롤링 업데이트가 시작이 된다.

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.21
```

```
deployment.apps/nginx-deployment image updated
```

```
kubectl rollout status deployment/nginx-deployment
```

```
deployment "nginx-deployment" successfully rolled out
```

```
kubectl get pods
```

NAME	READY	STATUS	RESTARTS	
AGEnginx-deployment-5678fc88d8-uvwxy	1/1	Running	0	
5snginx-deployment-5678fc88d8-qrstz	1/1	Running	0	
10snginx-deployment-5678fc88d8-klmno	1/1	Running	0	30s



롤링 업데이트

이전 버전으로 롤백은 다음과 같다. 롤백은 Deployment를 사용하는 RS(ReplicaSet)를 통해서 수행이 된다.

```
# kubectl rollout undo deployment/nginx-deployment
```

```
deployment.apps/nginx-deployment rolled back
```

```
# kubectl get rs
```

```
# kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-deployment-6799fc88d8-xxxxx	1/1	Running	0	5s
nginx-deployment-6799fc88d8-yyyyy	1/1	Running	0	10s
nginx-deployment-6799fc88d8-zzzzz	1/1	Running	0	30s



블루/그린, 카나리 업데이트

확장 명령어



B/G, CANARY 설명

블루/그린 및 카나리는 다음과 같은 차이가 있다. 이 차이점을 통해서 어떠한 방식으로 서비스에 적용할 지 결정한다.

배포 방식	방식	특징
Blue/Green	새 환경을 배포 후 트래픽을 전환	즉시 전환 가능, 빠른 롤백 가능
Canary	일부 트래픽만 새 버전으로 이동	점진적 배포, 작은 범위에서 테스트 가능



BLUE 생성 및 적용

쿠버네티스에서 Blue/Green를 사용하면, 기존 버전을 유지하면서 새로운 버전으로 전환이 가능하다. 이하 B/G를 사용하기 위해서 다음과 같이 Deployment를 생성한다.

```
kubectl apply -f - <<EOF
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-blue
```

```
  labels:
```

```
    app: nginx
```

```
    version: blue
```



BLUE 생성 및 적용

위의 내용 계속 이어서...

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      version: blue
  template:
    metadata:
      labels:
        app: nginx
        version: blue
```



BLUE 생성 및 적용

위의 내용 계속 이어서...

```
spec:
  containers:
  - name: nginx
    image: nginx:1.19
    ports:
    - containerPort: 80
```

EOF

```
kubectl apply -f nginx-blue.yaml
```



BLUE 서비스 생성 및 적용

블루 POD로 트래픽을 전달한 SVC를 구성한다.

```
kubectl apply -f - <<EOF
```

```
apiVersion: v1
```

```
kind: Service
```

```
metadata:
```

```
  name: nginx-service
```

```
spec:
```

```
  selector:
```

```
    app: nginx
```

```
    version: blue
```



BLUE 서비스 생성 및 적용

위의 내용 계속 이어서...

```
ports:
  - protocol: TCP
    port: 80
    targetPort: 80
EOF
kubectl apply -f svc-nginx-service.yaml
```



GREEN 생성 및 적용

그린(Green) 역할을 할 POD를 생성한다.

```
kubectl apply -f - <<EOF
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-green
```

```
  labels:
```

```
    app: nginx
```

```
    version: green
```



계
속

GREEN 생성 및 적용

위의 내용 계속 이어서...

```
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
      version: green
  template:
    metadata:
      labels:
        app: nginx
        version: green
```



계
속

GREEN 생성 및 적용

이전 버전과 다르게 이미지 버전이 1.21로 명시가 되어있다.

```
spec:
  containers:
  - name: nginx
    image: nginx:1.21
    ports:
    - containerPort: 80
```

EOF

```
kubectl get pod
```



BLUE/GREEN 전환 및 패치

현재 서비스는 그린으로 트래픽이 흘러가지 않고, 블루로 흘러가고 있다. 트래픽을 그린으로 흘리기 위해서 다음과 같이 명령어를 실행한다.

```
kubectl patch service nginx-service -p '{"spec":{"selector":{"app":"nginx",  
"version":"green"}}}'
```

혹은 에디터 명령어로 수정이 가능하다.

```
kubectl edit service nginx-service
```

문제가 있다고 판단이 되면, 언제든지 다시 SVC에서 블루 버전으로 변경이 가능하다.

```
kubectl patch service nginx-service -p '{"spec":{"selector":{"app":"nginx",  
"version":"blue"}}}'
```



BLUE/GREEN

올바르게 서비스 구성이 되었으면, 구 버전인 블루 POD를 제거한다.

```
kubectl delete deployment nginx-blue  
deployment.apps/nginx-blue deleted
```



CANARY 설명

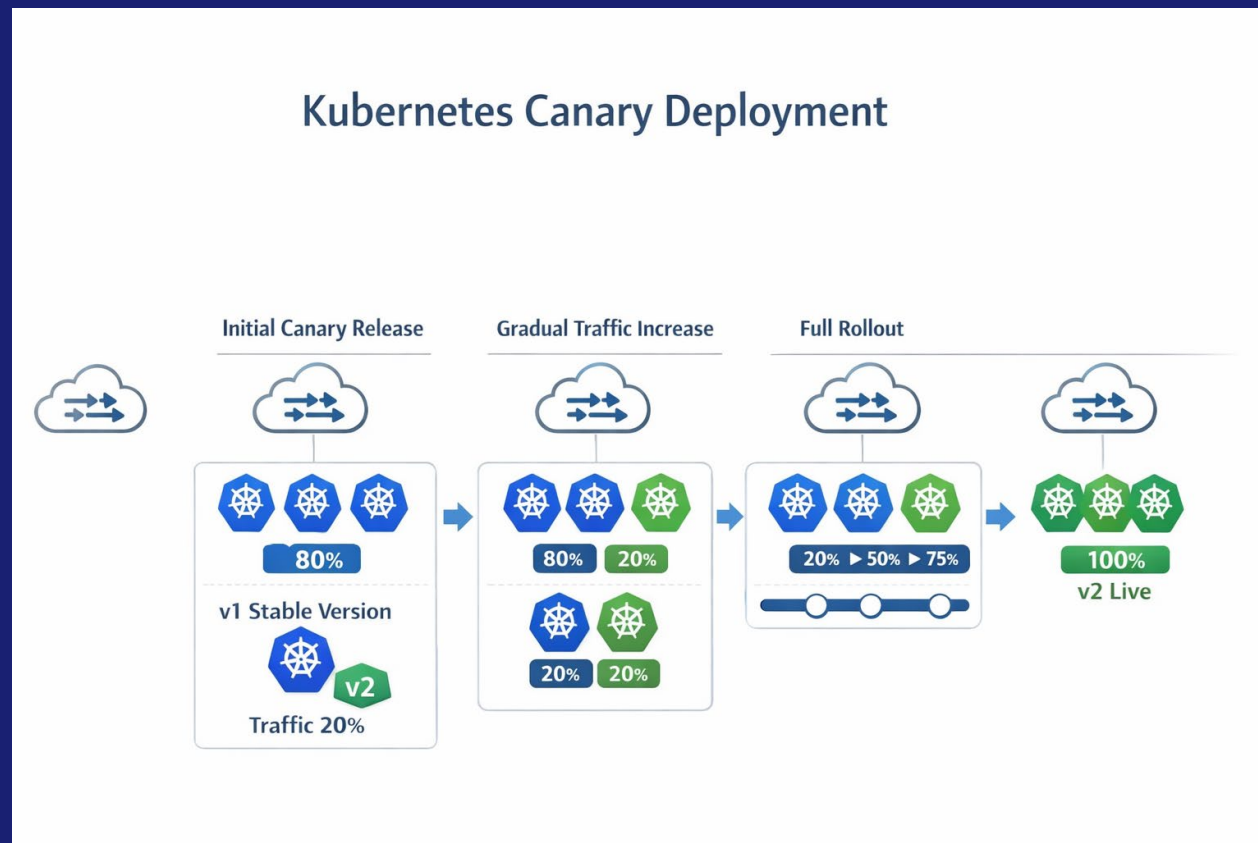
카나리(CANARY)는 B/G와 다르게 점진적으로 POD를 생성하면서 전환을 시작한다.

이와 비슷한 업데이트가 A/B 업데이트라고 부르기도 한다. 하지만, 최종적인 동작 방식은 다르다. A/B를 사용하기 위해서 Istio와 같은 서비스 라우팅 기능이 필요하다.

1. 레드햇 오픈 시프트는 이 부분을 라우터에 포함이 되어 있으며, 바닐라 버전은 일반적으로 Istio기반으로 처리한다.
2. 바닐라 버전에서는 A/B 업데이트 기능을 구현 및 사용하기 위해서는 Ingress 서비스가 필요하다.
 1. Ingress-Nginx는 더 이상 개발 및 유지보수가 되지 않는다.
 2. Ingress를 사용하면 HAProxy이나 혹은 다른 솔루션 사용 권장
 3. API-GATEWAY 사용을 앞으로 권장한다
3. 바닐라 버전에서는 카나리 기능 구현이 꽤나 불편한 편이다.



카나리 동작 방식



바닐라 기반 CANARY POD(1.19)

먼저, nginx-stable버전을 만들어서 배포한다.

```
vi nginx-stable.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-stable
  labels:
    app: nginx
    version: stable
```



바닐라 기반 CANARY POD(1.19)

```
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: nginx  
      version: stable
```



바닐라 기반 CANARY POD(1.19)

```
template:
  metadata:
    labels:
      app: nginx
      version: stable
  spec:
    containers:
      - name: nginx
        image: nginx:1.19
        ports:
          - containerPort: 80
```



CANARY POD 생성(1.19)

아래와 같이 적용한다.

```
kubectl apply -f stable-nginx.yaml  
kubectl get -f stable-nginx.yaml
```



바닐라 기반 CANARY 적용(1.21)

nginx-canary버전을 만들어서 배포한다.

```
vi nginx-canary.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-canary
  labels:
    app: nginx
    version: canary
```



바닐라 기반 CANARY 적용(1.21)

```
spec:  
  replicas: 1  
  selector:  
    matchLabels:  
      app: nginx  
      version: canary
```



바닐라 기반 CANARY 적용(1.21)

```
template:
  metadata:
    labels:
      app: nginx
      version: canary
  spec:
    containers:
      - name: nginx
        image: nginx:1.21
        ports:
          - containerPort: 80
```



바닐라 기반 CANARY 적용(1.21)

아래와 같이 적용한다.

```
kubectl apply -f canary-nginx.yaml  
kubectl get -f canary-nginx.yaml
```



CANARY 서비스 전환 및 테스트

위와 같이 하면, 1.19 3개, 1.21 1개의 POD가 동작. 이를 SVC를 통해서 연결 및 구성한다. 먼저, 안정버전으로 서비스를 구성한다.

```
cat <<EOF> svc-nginx-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
```



CANARY 서비스 구성

위의 내용 계속 이어서...

```
spec:
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
EOF
kubectl apply -f svc-nginx-service.yaml
```



CANARY 서비스 분배(바닐라 기준)

위의 서비스는 다음처럼 서비스가 분배가 되어있다. nginx-canary 버전이 문제 없이 동작하면, 기존에 사용하던 nginx-stable 버전을 제거하여 새로운 nginx-canary로 완전 전환한다.

`nginx-stable` (`replicas: 3, version=stable`)

`nginx-canary` (`replicas: 1, version=canary`)



`nginx-service`



CANARY 서비스 정리

문제가 없다고 판단이 되면, 두 가지 방법으로 서비스 정리가 가능하다.

1. 기존에 사용한 Deployment 제거
2. 혹은 이미지 버전만 변경(권장하지 않음)
3. 셀렉트 레이블 통하여 특정 POD만 외부에 노출

안정적인 운영을 위해서는 1번으로 구성 및 운영을 권장한다. 퍼센테이지로(%) Canary 서비스 운영이 필요한 경우, Ingress 서비스를 통해서 구성이 가능하다.



CANARY 서비스 정리

Ingress에서 사용하는 경우, 아래와 같이 적용이 가능하다. Canary서비스로 동작하는 부분에 아래와 같이 적용 및 구성한다.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-canary
  annotations:
    nginx.ingress.kubernetes.io/canary: "true"
    nginx.ingress.kubernetes.io/canary-weight: "20"
```



정리 1

기능	설명	사용 목적
Liveness Probe	애플리케이션이 정상 작동 중인지 확인	비정상 상태일 경우 자동 재시작
Readiness Probe	트래픽을 받을 준비가 되었는지 확인	준비되지 않으면 서비스에서 제외
Startup Probe	컨테이너가 완전히 시작되었는지 확인	부팅 시간이 긴 애플리케이션



정리 2

기능	설명	사용 목적
Rolling Update	무중단 배포 방식	운영 환경에서 애플리케이션 업데이트
Recreate Update	기존 Pod 삭제 후 신규 배포	데이터 정합성이 중요한 경우
Graceful Shutdown	애플리케이션 정상 종료	데이터 손실 방지 및 안정적인 종료
HPA	부하에 따라 자동으로 Pod 개수 조정	트래픽 급증 대응
PDB	최소한의 Pod 개수를 유지	장애 발생 시 가용성 보장



정리 3

배포 방식	방식	특징
Blue/Green	새 환경을 배포 후 트래픽을 전환	즉시 전환 가능, 빠른 롤백 가능
Canary	일부 트래픽만 새 버전으로 이동	점진적 배포, 작은 범위에서 테스트 가능



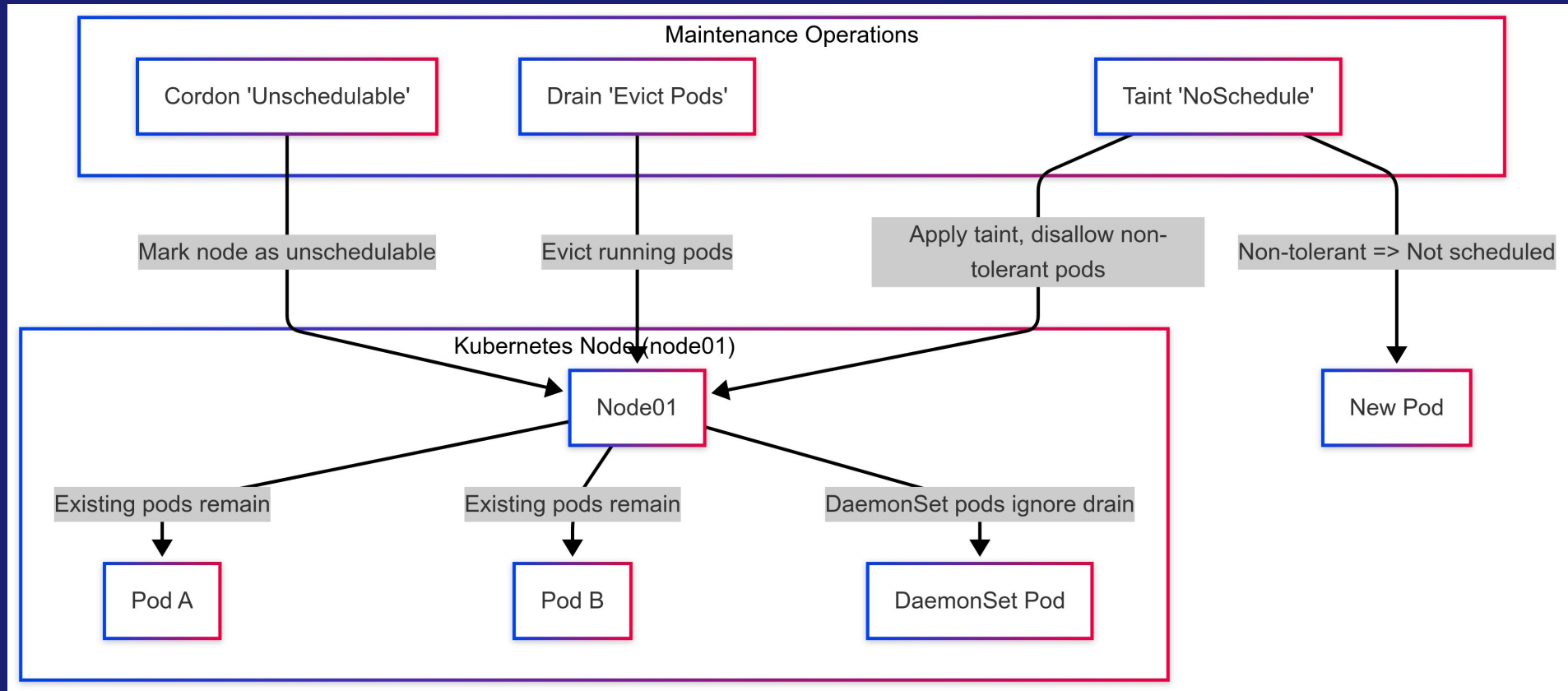
DRAIN/TAINT/CORDON

확장 명령어

유지보수 명령어



설명



설명

기능을 비교하면 다음과 같다.

Command	Purpose	Usage Example
kubectl cordon	노드를 unschedulable 상태로 전환하여 신규 파드가 스케줄되지 않도록 함	kubectl cordon node01
kubectl drain	노드에서 실행 중인 파드들을 안전하게 축출하여 유지보수 모드로 전환	kubectl drain node01 --ignore-daemonsets --delete-local-data
kubectl taint	노드에 taint를 적용하여 toleration이 없는 파드가 스케줄되지 않도록 함	kubectl taint nodes node01 key1=value1:NoSchedule
		kubectl taint nodes node01 key1:NoSchedule-



CORDON(스케줄링 제외)

특정 노드에 스케줄링이 되지 않도록 합니다. 모든 POD는 생성이 되지 않습니다.

```
kubectl cordon node01  
node/node01 cordoned
```



DRAIN(POD 배출)

유지보수를 위해서 사용중인 POD를 삭제 후 다른 노드에 생성, 위의 옵션은 --ignore-daemonsets라는 옵션이 있기 때문에 DaemonSet POD는 영향 받지 않는다.

```
kubectl drain node01 --ignore-daemonsets --delete-local-data node/node01
cordoned
node/node01 cordoned
evicting pod nginx-deployment-xxxx from node node01
pod "nginx-deployment-xxxx" evicted
...
node/node01 drained
```



TAINT(오염)

특정 노드에 Taint를 선언하여 POD생성이 되지 않는다. 다만, Toleration에 따라서 POD가 생성이 될 수 있다.

```
kubectl taint nodes node01 key1=value1:NoSchedule
```

```
node/node01 tainted
```

```
kubectl taint nodes node01 key1:NoSchedule-
```

```
node/node01 untainted
```



LIVENESS/READINESS

확장 명령어



LIVENESS 설명

컨테이너는 프로세스 관점에서 실행 중일 수 있지만, 실제로는 무한 루프, 데드락, 내부 오류로 인해 더 이상 정상적인 기능을 수행하지 못하는 상태가 될 수 있다.

이러한 경우 Kubernetes는 단순히 프로세스가 살아 있다는 이유만으로는 문제를 감지할 수 없기 때문에, Liveness Probe를 통해 컨테이너의 실질적인 생존 상태를 주기적으로 확인한다.

Liveness Probe가 실패하면 Kubernetes는 해당 컨테이너를 비정상 상태로 판단하고 즉시 재시작한다. 이 동작은 사용자의 개입 없이 자동으로 수행되며, 애플리케이션 자체 복구(Self-healing)의 핵심 요소로 작동한다.



LIVENESS 설명

즉, Liveness Probe의 목적은 이 컨테이너를 계속 살려둘 가치가 있는가, 아니면 죽였다가 다시 살릴 것인가를 판단하는 데 있다.

Liveness Probe는 주로 다음과 같은 상황에서 사용된다.

- 프로세스는 살아 있지만 응답하지 않는 경우
- 특정 엔드포인트가 더 이상 정상 동작하지 않는 경우
- 재시작이 문제 해결에 가장 빠른 방법인 애플리케이션

결론적으로, Liveness Probe는 컨테이너의 생존 여부를 판단하여 자동 재시작을 유도하는 안전장치라고 볼 수 있다.



LIVENESS 정리

Liveness Probe는 "이 컨테이너가 아직 살아 있는가?"를 판단하기 위한 Kubernetes의 생존 확인 메커니즘이다.



READINESS 설명

컨테이너가 실행되었다는 사실과, 실제 서비스 요청을 처리할 수 있는 상태는 서로 다를 수 있다.

애플리케이션은 기동 후에도 설정 로딩, 캐시 초기화, 외부 시스템 연결(DB, API) 등의 과정을 거치며, 이 단계에서는 요청을 처리하는 것이 오히려 장애를 유발할 수 있다.

Readiness Probe는 이러한 상태를 고려하여, 컨테이너가 아직 준비되지 않았다고 판단되면 Service 트래픽 대상에서 제외한다. 중요한 점은, Readiness Probe 실패 시 컨테이너는 종료되지 않고 그대로 유지된다는 것이다.



READINESS 설명

즉, Readiness Probe의 목적은 **살아는 있지만, 아직 일은 시키지 말자는** 판단을 Kubernetes에게 전달하는 데 있다.

이 메커니즘은 다음과 같은 상황에서 매우 중요하다.

- 롤링 업데이트 중 신규 Pod가 완전히 준비되기 전
- 외부 의존성(DB, 메시지 큐)이 일시적으로 불안정한 경우
- Canary/A-B 배포에서 특정 Pod를 임시로 트래픽에서 제외할 때

결론적으로, Readiness Probe는 무중단 배포와 안정적인 트래픽 제어를 가능하게 하는 핵심 장치이며, Liveness Probe와 달리 재시작이 아닌 "트래픽 제어"를 목적으로 한다.

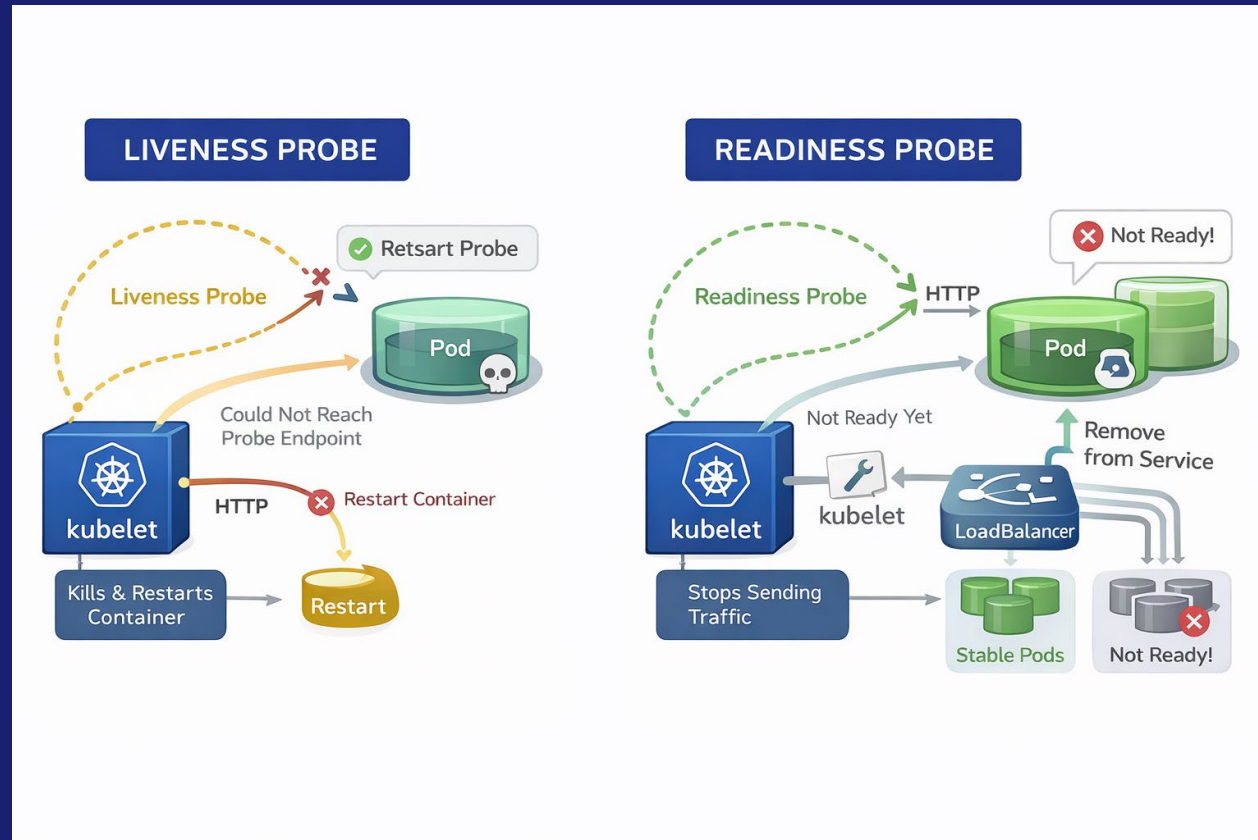


READINESS 정리

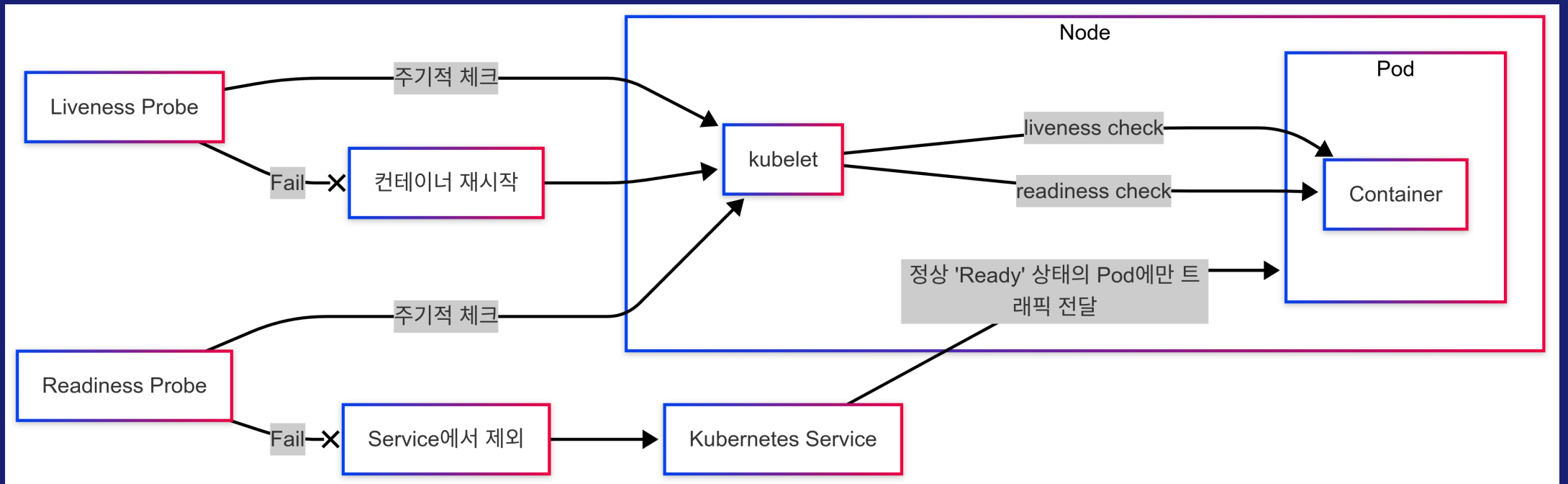
Readiness Probe는 "이 컨테이너가 지금 트래픽을 받아도 되는 상태인가?"를 판단하기 위한 준비 상태 확인 메커니즘이다.



LIVENESS/READINESS 비교



LIVENESS/READINESS 동작 차이



개념 설명

관리 방법	설명
Liveness Probe	애플리케이션이 정상 작동 중인지 감지하여, 비정상 상태일 경우 자동 재시작
Readiness Probe	애플리케이션이 트래픽을 받을 준비가 되었는지 감지하여, 준비되지 않으면 서비스에서 제외
Startup Probe	애플리케이션이 완전히 시작되었는지 확인하여, 일정 시간 동안 기다려 준 후 Liveness Probe 적용



프로브(LIVENESS) 적용

프로브를 통해서 모니터링은 다음과 같이 한다. 컨테이너가 죽었는지 상태 확인을 한다.

```
apiVersion: v1
kind: Pod
metadata:
  name: liveness-probe-pod
```



프로브(LIVENESS) 적용

1. 5초 후 liveness 시작
2. 10초마다 확인

이 엔드 포인트가 실패하면 kubelet은 컨테이너를 재 시작한다.

```
spec:
  containers:
  - name: nginx
    image: nginx
    livenessProbe:
      httpGet:
        path: /health
        port: 80
      initialDelaySeconds: 5
      periodSeconds: 10
```



프로브(READINESS)

컨테이너는 동작하였고, 지금 트래픽을 받아도 되는 상태인지 확인하기 위해서 아래와 같이 명령어를 실행한다. 위의 내용에 추가한다. 컨테이너에서 사용하는 특정 포트 및 URL을 명시한다.

1. 5초 후 readiness 시작
2. 10초마다 확인

readinessProbe:

httpGet:

path: /ready

port: 80

initialDelaySeconds: 5

periodSeconds: 10



프로브(READINESS)

컨테이너 시작 시간이 상대적으로 긴 경우(대략 1분 이상), 다음과 같이 조건을 걸어서 올바르게 동작하도록 한다.

1. 최대 30회까지 기동 완료 확인(여기서는 총 300초)
2. 각 10초 대기

이 이상으로 이벤트가 발생하면 작업을 중지한다.

startupProbe:

httpGet:

path: /startup

port: 80

failureThreshold: 30

periodSeconds: 10



nodeAffinity

확장 명령어

노드 어/애피니티(US/UK)



노드 셀렉터 및 선호도

노드 셀렉터 및 선호도의 차이점은 다음과 같다. 앞에서 다룬 레이블과 비슷하지만, 조금 다른 역할이다.

기능	nodeSelector	nodeAffinity
사용 방식	단순한 키-값 매칭	표현식으로 유연한 조건 설정 가능
강제성	해당 노드가 아니면 실행 안 됨	"preferredDuringSchedulingIgnoredDuringExecution" 옵션을 사용하면 선호도만 설정 가능
예제 사용법	nodeSelector 필드 사용	nodeAffinity 필드 사용



nodeAffinity 설명

nodeAffinity는 Kubernetes 스케줄러가 Pod를 어떤 노드에 배치할지 결정할 때 사용하는 "노드 선택 조건"이다. 기본적으로 Kubernetes는 리소스(CPU, 메모리)가 충분한 노드라면 어느 곳이든 Pod를 배치하려고 한다.

하지만 실제 운영 환경에서는 특정 Pod를 특정 성격의 노드에만 배치해야 하는 경우가 자주 발생한다.

예를 들어,

- GPU가 있는 노드에만 AI Pod를 배치해야 하거나
- SSD가 장착된 노드에 데이터베이스를 올려야 하거나
- 규제 또는 보안 요구로 특정 워크로드를 분리해야 하는 경우

이러한 요구를 충족하기 위해 nodeAffinity를 사용한다.



nodeAffinity 설명

nodeAffinity는 노드에 설정된 label을 기준으로 Pod가 배치될 수 있는 조건을 정의하며, "반드시 지켜야 하는 조건"과 "가능하면 지키고 싶은 조건"을 명확히 구분할 수 있다.

이를 통해 사용자는

- 배치 제약을 명시적으로 선언할 수 있고
- 스케줄링 정책을 코드(YAML)로 문서화할 수 있으며
- 클러스터가 커질수록 더욱 예측 가능한 Pod 배치를 만들 수 있다.

결론적으로, nodeAffinity는 "이 Pod는 아무 노드나 가면 안 된다"는 운영자의 의도를 Kubernetes에 전달하는 공식적인 방법이다.



노드 셀렉터

노드 셀렉터는 특정 노드에 레이블 구성 후, 해당 노드로 POD를 생성할 수 있도록 구분한다. 셀렉터 예제로 많이 사용하는 케이스는 디스크나 혹은 GPU부분이다. 예를 들어서 디스크가 SSD, HDD이렇게 있다고 가정한다.

그러면, SSD나 혹은 HDD(SAS)로 구성된 노드에 레이블을 걸어서 사용자가 원하는 상황에 따라서 선택 및 사용할 수 있도록 한다.

```
kubectl label nodes <노드이름> disktype=ssd
```

```
kubectl label nodes cluster1-node1.example.com disktype=ssd
```



노드 셀렉터 구성 예제

```
cat <<EOF> node-selector.yaml
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: node-selector-pod
```

```
spec:
```

```
  containers:
```

```
  - name: nginx
```

```
    image: nginx
```

```
  nodeSelector:
```

```
    disktype: ssd
```

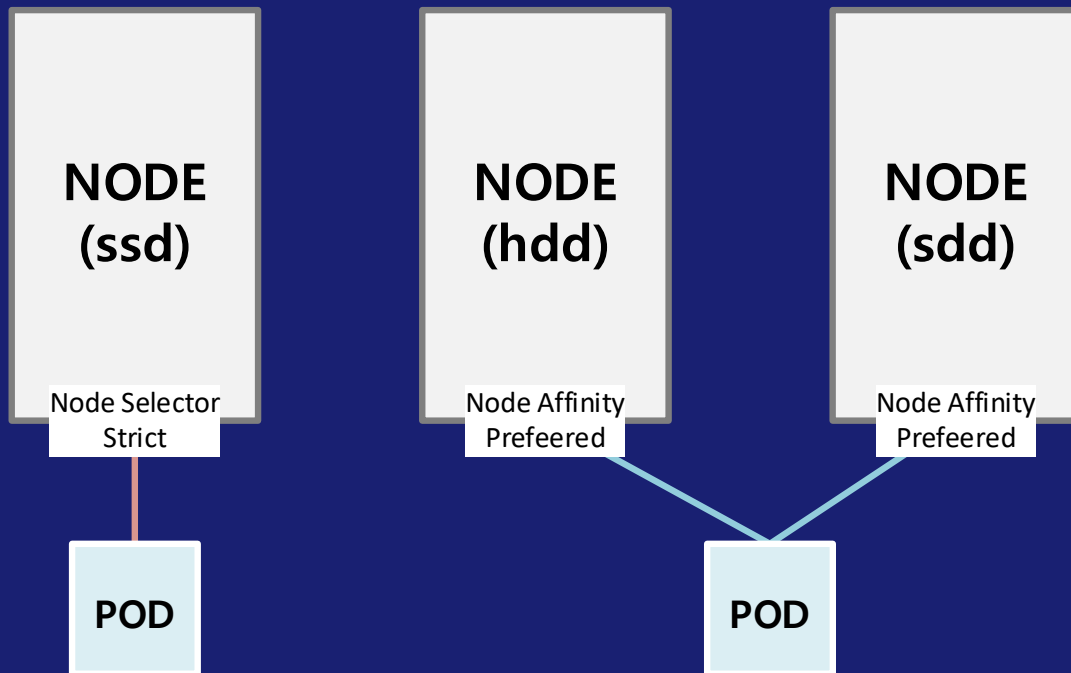
```
EOF
```

```
kubectl apply -f node-selector.yaml  
kubectl get -f node-selector.yaml
```



노드 선호도 (Node Affinity)

노드 선호도는 셀렉터와 비슷하지만, 다음과 같이 다른 부분이 있다. 셀렉터는 제한에 가깝고, 선호도는 여러 노드에 복합적으로 조건을 적용하는 방법.



노드 선호도 (Node Affinity)

```
cat <<EOF> node-affinity.yaml
apiVersion: v1
kind: Pod
metadata:
  name: node-affinity-pod
spec:
  containers:
  - name: nginx
    image: nginx
```



노드 선호도 (Node Affinity)

```
affinity:  
  nodeAffinity:  
    requiredDuringSchedulingIgnoredDuringExecution:  
      nodeSelectorTerms:  
        - matchExpressions:  
          - key: disktype  
            operator: In  
            values:  
              - ssd
```

EOF

```
kubectl apply -f node-affinity.yaml
```



위의 노드 선호도 조건 설명

스케줄러는 다음과 같이 판단한다.

- 노드의 label 목록을 확인한다
- disktype=ssd 조건을 만족하는 노드만 후보로 남긴다
- 조건을 만족하는 노드가 하나도 없다면
 - Pod는 Pending 상태로 대기한다

즉, 이 설정은 "이 Pod는 SSD가 없는 노드에서는 절대 실행되면 안 된다"라는 운영자의 의도를 명확히 전달한다.



ReplicaSet/ReplicationController Deployment/DaemonSet StatefulSet

확장 명령어



기능 미리보기 및 설명

쿠버네티스에서 제공하는 POD생성 및 복제 서비스는 다음과 같이 릴리즈가 되었다. 진행 전, 참조하도록 한다. 왜냐면, 이 내용들은 아래에서 계속 언급이 된다.

기능	도입된 Kubernetes 버전	연도
ReplicationController	v1.0 (Initial/Legacy)	2015
DaemonSet	v1.2~v1.6 (Beta), v1.9 (Stable) v1.x (Early), v1.9 (Stable)	2015~2017
ReplicaSet	v1.2 (Beta), v1.9 (Stable) extensions/v1beta1 → apps/v1	2016 (Beta), 2017 (Stable)
StatefulSet	v1.5 (Beta), v1.9 (Stable) apps/v1beta1 → apps/v1	2016 (Beta), 2017 (Stable)
Deployment	v1.2 (Beta), v1.9 (Stable) extensions/v1beta1 → apps/v1	2016 (Beta), 2017 (Stable)
DeploymentConfig	OpenShift Specific	2015



ReplicaSet 설명

ReplicaSet은 기본적으로 Deployment와 같이 사용한다. 하지만, 상황에 따라서 독립적으로 사용이 가능하다.

ReplicaSet의 역할은 다음과 같다.

1. 명시된 개수만큼 POD를 유지한다.
2. 제거된 POD가 확인이 되면, 즉시 바로 재구성이 된다.
3. 버전이 변경이 되는 경우, ReplicaSet은 기존 내용을 유지하고 새로운 ReplicaSet 정책 기반으로 POD를 구성한다.
4. Deployment가 없는 경우, Revision기능을 사용 할 수 없다.

대다수 서비스 애플리케이션은 ReplicaSet기반으로 동작하기 때문에, 최소 한 개의 ReplicaSet 설정은 권장한다. 일반적인 서비스 애플리케이션은 Deployment를 통해서 배포가 되기 때문에, 별도로 작성하는 경우는 거의 드물다.



ReplicaSet

최신 버전의 쿠버네티스는 가급적이면(거의 100%) ReplicaSet기반으로 POD생성을 권장한다. 그 이유는 다음과 같다.

기능	ReplicaSet (RS)	ReplicationController (RC)
오브젝트 버전	apps/v1	v1
Pod 개수 관리	가능	가능
레이블 셀렉터 지원	정규식 기반 matchExpressions 지원	matchLabels 기반으로 적용.
Pod 자동 복구	가능	가능
Deployment와 연동	사용 가능	사용 불가능
추천 여부	(권장) 최신 버전이며, Deployment와 함께 사용 가능	(구버전, 더 이상 사용 권장되지 않음)



ReplicaSet 구성

ReplicaSet만 사용하여 POD를 구성하는 경우 다음과 같이 생성 및 구성한다.

```
cat <<EOF> replicaset.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: rs-nginx
  labels:
    app: nginx
    tier: lab-rs
```



ReplicaSet 구성

```
spec:
  replicas: 3
  selector:
    matchLabels:
      tier: lab-rs
  template:
    metadata:
      labels:
        tier: lab-rs
    spec:
      containers:
      - name: nginx
        image: docker.io/library/nginx:latest
```

EOF



ReplicaSet 구성

등록된 내용을 확인 시, 다음과 같이 명령어를 실행하여 올바르게 생성 및 구성이 되었는지 확인한다.

```
kubectl apply -f replicaset.yaml  
kubectl get -f replicaset.yaml
```



ReplicaController

컨트롤러는 초창기 쿠버네티스에서 사용하던 자원이다. **ReplicaController(RC)**와 **Deployment**의 도입 시기는 다음과 같다.

Deployment 등장 과정 요약

1. Kubernetes 1.0 (2015년 7월): ReplicationController 도입
2. Kubernetes 1.2 (2016년 3월): ReplicaSet 및 Deployment (Beta) 도입
3. Kubernetes 1.9 (2017년 12월): Deployment GA(Stable)

초기부터 사용한 사용자를 위해서 **ReplicationController**는 호환성을 염두하고 존재하며, 가급적이면 **ReplicationController** 대신, **ReplicaSet**과 **Deployment** 사용을 권장한다.



ReplicaController 설정 및 구성

컨트롤러는 ReplicaSet하고 거의 동일하다. 차이점은 위에서 설명 하였다. 작성이 필요한 경우, 아래와 같이 구성이 가능하다.

```
cat <<EOF> replicacontroller.yaml
apiVersion: v1
kind: ReplicationController
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    app: nginx
```



ReplicaController 설정 및 구성

위의 내용 계속...

```
template:
  metadata:
    name: nginx
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: nginx
        ports:
          - containerPort: 80
```

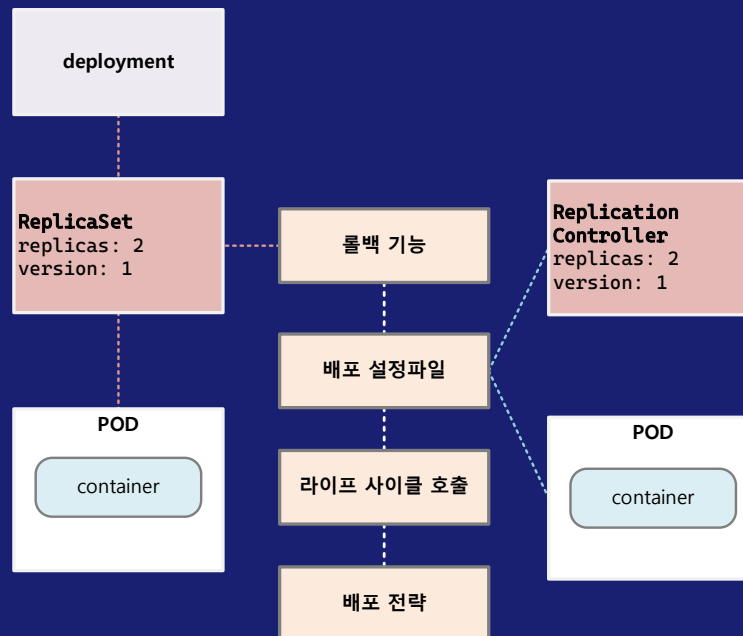
EOF



Deployment 설명

Deployment는 ReplicaSet과 함께 도입이 되었다.

GA기준 1.9에서 안정화가 되었다. Deployment의 주요 역할은 애플리케이션 설정 내용을 가지고 ReplicaSet으로 전달 및 배포가 주요 목적이다.



Deployment 구성

Deployment 구성은 다음과 같이 구성이 되어있다.

```
cat <<EOF> deployment.yaml
```

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-deployment
```

```
  labels:
```

```
    app: nginx
```

```
spec:
```

```
  replicas: 3
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

이 부분은 기존과 동일하다.

replicas: 3, POD 3개 생성

Deployment가 관리할 RS를 검색.
RS에 해당 레이블이 존재해야 한다.

계
속



Deployment 구성

계속...

```
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
    - name: nginx
      image: nginx:1.14.2
      ports:
      - containerPort: 80
```

"template"은 "matchLabels:"과 동일해야 한다.

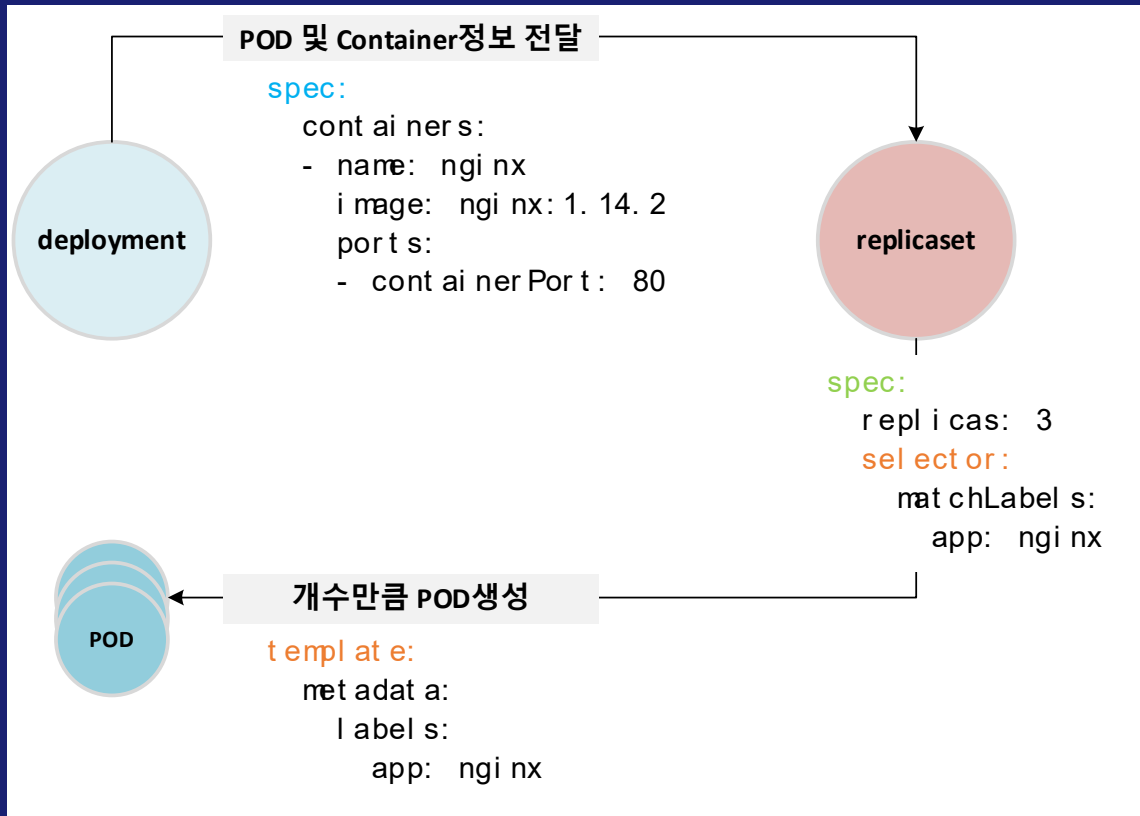
EOF

```
kubectl apply -f deployment.yaml
```



Deployment 동작 방식 설명

도식으로 표현하면 다음과 같이 동작한다.



Deployment 확인

Deployment에 구성된 자원은 하나만 존재하며, Deployment가 갱신이 되면, 해당 내용에 대해서 ReplicaSet에서 기록을 가지고 있다.

이미지나 혹은 설정 내용에 변경이 발생하면, 다음처럼 ReplicaSet에서 기록이 남는다.

```
kubectl get deploy
```

NAME	READY	AVAILABLE
AGEdeployment.apps/nginx-deployment	3/3	3

```
kubectl get replicaset
```

NAME	DESIRED	CURRENT
nginx-deployment-665fd86dbb	1	1
nginx-deployment-77d8468669	3	3



DaemonSet 설명

DaemonSet은 Deployment와 기능상 동일하다. DaemonSet도 Deployment와 동일하게 2015년도에 정식으로 GA되었다.

Deployment와 제일 큰 다른 부분은 Deployment는 Scheduler를 통해서 클러스터에 구성된 노드에 배포가 주요 목적이다.

하지만, DaemonSet은 Deployment와 동일하게 Scheduler를 통하지만, 각 노드당 하나씩 POD생성이 주요 목적이다. 다시 말하지만, 컨테이너가 아닌 POD를 각 노드에 구성이다.

쿠버네티스에서 대표적인 DaemonSet POD는 kube-proxy이다. kube-proxy는 모든 노드에 최소 한 개씩 구성이 된다.

제한적으로 DaemonSet이 구성이 되는 서비스는 kube-apiserver, kube-scheduler, kube-controller-manager와 같은 POD가 있다. 이 서비스들은 Compute노드를 제외한, Controller노드에 구성이 된다.



DaemonSet 선언 방법

대몬셋은 다음과 같이 구성 및 작성한다. 올바른 학습을 위해서 최소 2개 이상의 컴퓨트 노드를 권장한다. 그렇지 않는 경우, 컴퓨트 노드에만 생성이 된다.

컨트롤러에도 생성하기 위해서는 톨러레이션(toleration)를 같이 구성해야 한다.

```
spec:
```

```
  spec:
```

```
    tolerations:
```

- key: node-role.kubernetes.io/control-plane
operator: Exists
effect: NoSchedule
- key: node-role.kubernetes.io/master
operator: Exists
effect: NoSchedule



DaemonSet 테스트 구성 1

예를 들어서 모든 노드에 로그 분석 에이전트가 필요한 경우 아래와 같이 생성 및 구성.

```
cat <<EOF> daemonset.yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd-elasticsearch
  namespace: kube-system
  labels:
    k8s-app: fluentd-logging
spec:
  selector:
    matchLabels:
      name: fluentd-elasticsearch
```



DaemonSet 테스트 구성 2

```
template:  
  metadata:  
    labels:  
      name: fluentd-elasticsearch  
  spec:  
    containers:  
      - name: fluentd-elasticsearch  
        image: quay.io/fluentd_elasticsearch/fluentd:v2.5.2
```



DaemonSet 테스트 구성 3

```
resources:  
  limits:  
    memory: 200Mi  
  requests:  
    cpu: 100m  
    memory: 200Mi  
volumeMounts:  
- name: varlog  
  mountPath: /var/log
```



DaemonSet 테스트 구성 4

자원 생성 후, 올바르게 반영이 되었는지 확인한다.

```
terminationGracePeriodSeconds: 30
volumes:
- name: varlog
  hostPath:
    path: /var/log
```

EOF

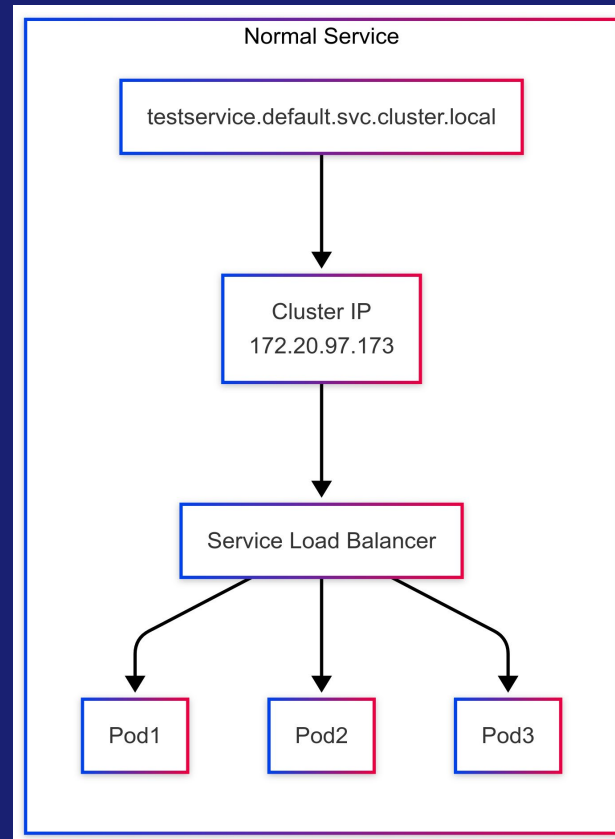
```
kubectl apply -f daemonset.yaml
```

```
kubectl get -f daemonset.yaml
```

```
kubectl describe -f daemonset.yaml
```



StatefulSet Normal



StatefulSet (ClusterIP Service 기반)

StatefulSet은 상태를 가지는 애플리케이션을 안정적으로 운영하기 위한 컨트롤러이다.

각 Pod는 고유한 이름(identity) 과 순서(index) 를 가지며, 재시작되거나 재 스케줄링 되더라도 동일한 이름과 스토리지를 유지한다.

일반적인 StatefulSet 구성에서는 ClusterIP Service가 함께 사용된다. 이 경우 클라이언트는 Service의 단일 ClusterIP로 접근하며, Service 내부에서 로드밸런싱을 통해 여러 Pod로 트래픽이 분산된다.



StatefulSet (ClusterIP Service 기반)

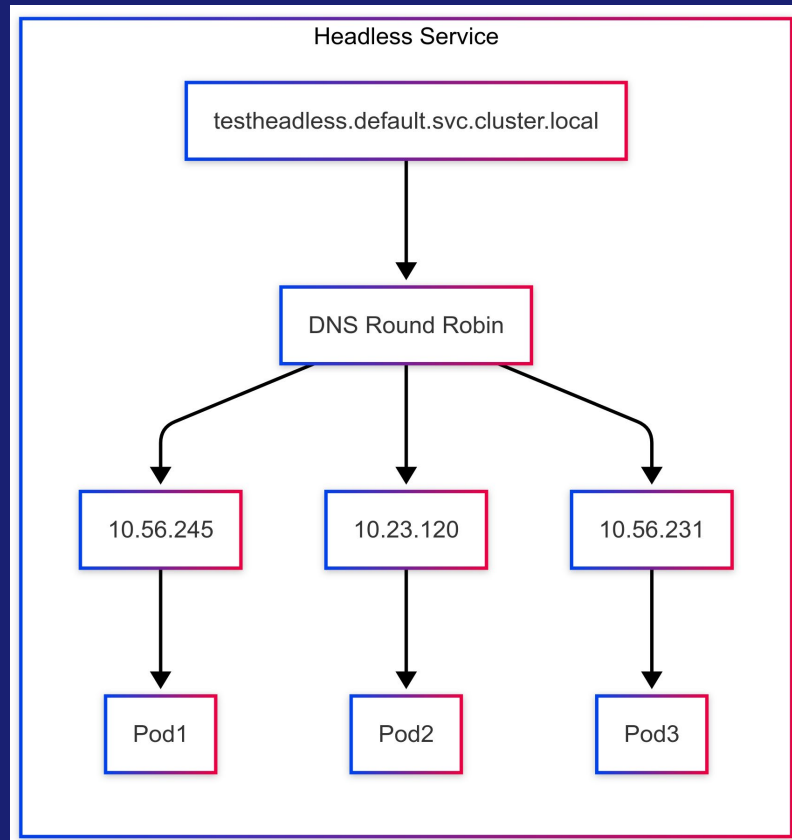
이 방식은 다음과 같은 특징을 가진다.

- 외부 또는 내부 클라이언트는 개별 Pod를 직접 인식하지 않는다
- Pod의 순서와 이름은 유지되지만, 접근 관점에서는 무상태 서비스처럼 동작한다
- 주로 읽기 전용 서비스, 프록시 계층, 단순 분산 처리에 적합하다

즉, StatefulSet의 "상태 유지" 기능은 사용하지만, 네트워크 접근은 일반 서비스처럼 단순화한 구조라고 볼 수 있다.



StatefulSet Headless



Headless Service 기반 StatefulSet

Headless Service는 clusterIP: None 으로 정의되며, Service 자체에 IP가 할당되지 않는다. 대신 Kubernetes DNS는 각 Pod의 IP를 그대로 반환한다.

StatefulSet과 Headless Service를 함께 사용하면, 각 Pod는 고유한 DNS 이름을 갖게 된다.

- `web-0.nginx.default.svc.cluster.local`
- `web-1.nginx.default.svc.cluster.local`



Headless Service 기반 StatefulSet

이 구조의 핵심 동작은 다음과 같다.

- DNS 조회 시 Pod 목록(IP들) 이 반환된다
- 로드 밸런싱은 Kubernetes가 아니라 클라이언트 또는 애플리케이션이 직접 수행한다
- 각 Pod는 명확하게 식별 가능하며, 특정 Pod에 직접 연결할 수 있다

이 방식은 다음과 같은 워크로드에 적합하다.

- 데이터베이스 (MySQL, PostgreSQL, Cassandra 등)
- 메시지 브로커 (Kafka, Zookeeper)
- 마스터/슬레이브 또는 리더/팔로워 구조

즉, StatefulSet의 본래 의도에 가장 충실한 사용 방식이라고 볼 수 있다.



StatefulSet 서비스 구성

StatefulSet 서비스를 생성 및 구성한다. 구성 시 클러스터 아이피가 없어야 한다.

```
cat <<EOF> sts-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx
  labels:
    app: nginx
```



StatefulSet 서비스 구성

위의 내용 계속 이어서...

```
spec:
  ports:
  - port: 80
    name: web
  clusterIP: None
  selector:
    app: nginx
EOF
kubectl apply -f sts-svc.yaml
kubectl get -f sts-svc.yaml
```



StatefulSet 서비스 구성

StatefulSet 자원을 생성한다.

```
cat <<EOF> sts-deploy.yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: web
spec:
  serviceName: "nginx"
  replicas: 2
  selector:
    matchLabels:
      app: nginx
```



계
속

StatefulSet 서비스 구성

앞에 내용 계속 이어서...

```
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: registry.k8s.io/nginx-slim:0.21
        ports:
          - containerPort: 80
            name: web
```

EOF



StatefulSet 서비스 적용

아래와 같이 적용한다.

```
kubectl apply -f sts.yaml  
kubectl get -f sts.yaml
```



Jobs/CronJob

확장 명령어



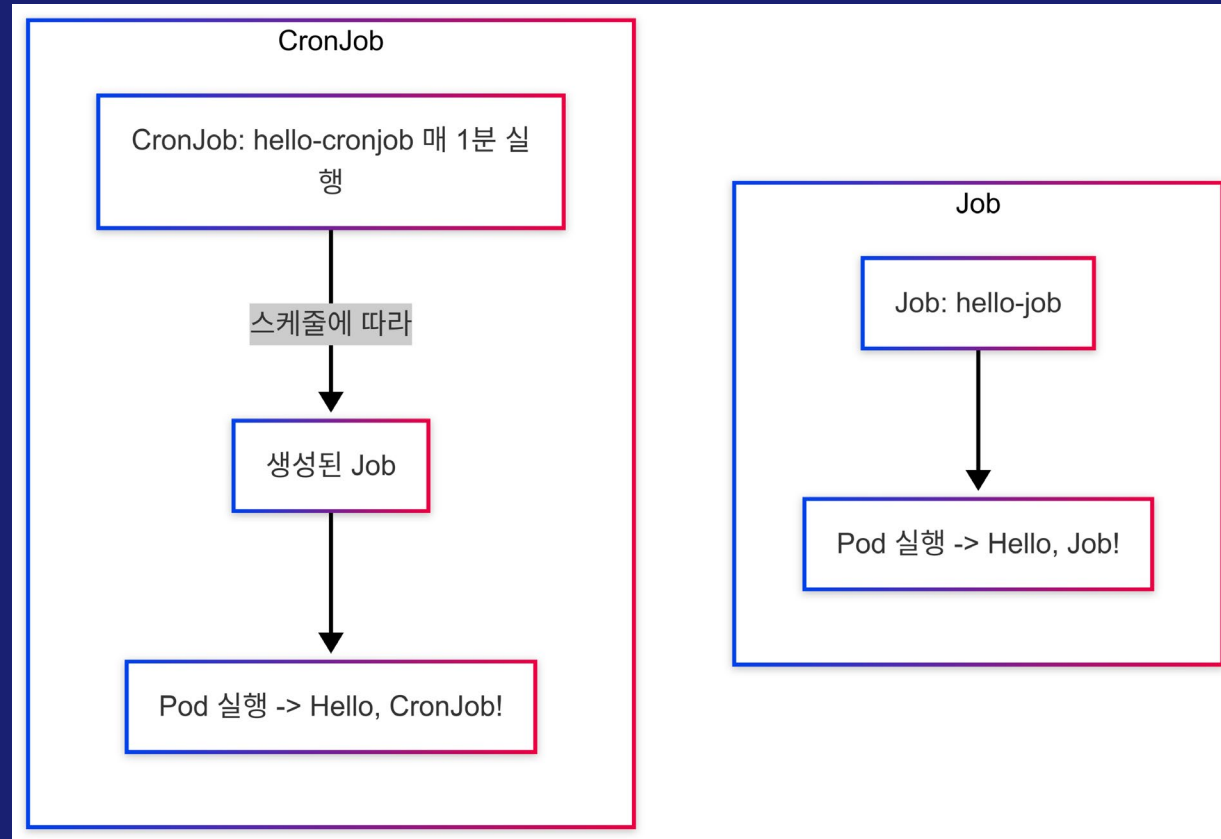
자원 설명

Job, CronJob의 기능 차이는 다음과 같다.

항목	Job	CronJob
목적	일회성 또는 단발성 작업을 실행하여 완료될 때까지 관리	정해진 일정에 따라 주기적으로 작업을 실행 (크론 스케줄러 사용)
실행 방식	Job 리소스가 생성되면 단 한 번 실행되고 완료될 때까지 실행	CronJob 스케줄에 따라 Job 리소스를 주기적으로 생성하여 실행
용도	데이터 마이그레이션, 일회성 배치 작업, 단발 테스트 등	정기적인 백업, 리포트 생성, 스케줄링된 데이터 처리 등
관리	Job이 완료되면 기본적으로 리소스가 남을 수 있으며, ttlSecondsAfterFinished 옵션을 통해 정리 가능	CronJob은 Job을 주기적으로 생성하므로, 완료된 Job 관리 및 정리 필요



자원 구조



자원 선언 및 구성

다음과 같이 작성 및 생성이 가능하다.

```
cat <<EOF> hello-job.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
```



자원 선언 및 구성

위의 내용 계속 이어서...

```
spec:
  template:
    spec:
      containers:
      - name: hello
        image: busybox
        command: ["echo", "Hello, Job!"]
        restartPolicy: Never
      backoffLimit: 4
```

EOF

```
kubectl -f hello-job.yaml
```



자원 선언 및 구성

다음과 같이 작성 및 생성이 가능하다.

```
kubectl apply -f hello-job.yaml
job.batch/hello-job created
kubectl get pods --selector=job-name=hello-job
NAME                READY   STATUS    RESTARTS   AGE
hello-job-xxxxx     0/1     Completed 0           30s
kubectl logs <pod-name>
Hello, Job!
```



CRONJOB 자원 선언 및 구성

JOB과 비슷하지만, 특정 시간마다 실행이 된다.

```
cat <<EOF> hello-cronjob.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello-cronjob
spec:
  schedule: "*/1 * * * *"
```



CRONJOB 자원 선언 및 구성

JOB과 비슷하지만, 특정 시간마다 실행이 된다.

```
jobTemplate:
  spec:
    template:
      spec:
        containers:
        - name: hello
          image: busybox
          command: ["echo", "Hello, CronJob!"]
          restartPolicy: Never
        successfulJobsHistoryLimit: 3
        failedJobsHistoryLimit: 1
```

```
kubectl apply -f hello-cronjob.yaml
```



CRONJOB 자원 적용 및 확인

JOB과 비슷하지만, 특정 시간마다 실행이 된다.

```
kubectl apply -f cronjob-example.yaml  
cronjob.batch/hello-cronjob created
```

```
kubectl get jobs --watch
```

NAME	COMPLETIONS	DURATION	AGE
hello-cronjob-1615939200	1/1	2s	1m
hello-cronjob-1615939260	1/1	2s	30s ...

```
kubectl get pods --selector=job-name=hello-cronjob-1615939200
```

```
kubectl logs <pod-name>
```

```
Hello, CronJob!
```



JOB 자원 적용 및 확인

명령어로 생성 및 구성이 가능하다.

```
kubectl create job hello-job --image=busybox -- /bin/sh -c "echo Hello, Job!"
```

```
job.batch/hello-job created
```

```
kubectl get pods --selector=job-name=hello-job
```

```
kubectl logs <pod-name>
```

```
Hello, Job!
```



JOB 자원 적용 및 확인

명령어로 생성 및 구성이 가능하다.

```
kubectl create cronjob hello-cronjob --image=busybox --schedule="*/1 * * *  
*" -- /bin/sh -c "echo Hello, CronJob!"  
cronjob.batch/hello-cronjob created  
kubectl get cronjob hello-cronjob  
kubectl get jobs --watch  
kubectl get pods --selector=job-name=<cronjob-generated-job-name>  
kubectl logs <pod-name>  
Hello, CronJob!
```



변수전달

확장 명령어



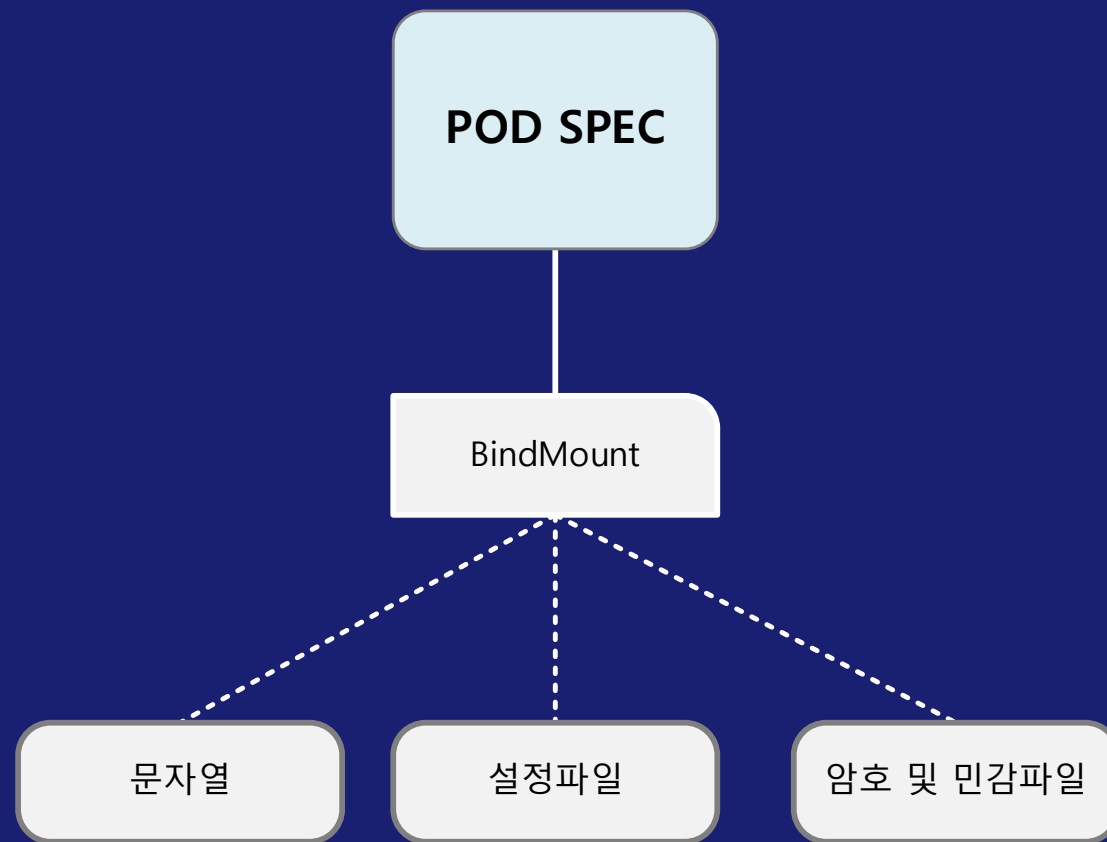
변수

쿠버네티스에서 변수 전달은 다음과 같은 방법으로 전달이 가능하다. 여기서 configmap, secret은 뒤에서 별도로 다룬다. 여기서는 ENV를 통해서 변수를 다루는 방법을 이야기 한다.

방법	설명	사용 사례 및 장점	단점 또는 주의사항
Literal 정의	Pod 스펙 내에 직접 key-value 형태로 환경 변수를 정의	간단한 변수 전달에 적합 빠르게 설정 가능	보안이 필요한 값은 노출 위험 있음
ConfigMap 사용	ConfigMap에 저장된 데이터를 환경 변수로 주입하여 여러 Pod에 서 재사용 가능	구성 정보를 중앙 관리할 수 있음 환경 별 구성 변경에 유리	ConfigMap 자체의 보안 이슈(민감 데이터 부적합)
Secret 사용	Secret에 저장된 민감한 데이터를 환경 변수로 주입	암호, 토큰 등 민감한 정보 를 안전하게 전달 가능	Base64 인코딩되어 있으 므로, 추가 디코딩 필요
Downward API	Pod의 메타데이터(예: 이름, 네임 스페이스, IP 등)를 환경 변수로 전 달	동적으로 Pod 정보를 전달 할 때 유용	전달할 수 있는 정보에 제 한이 있음



변수



POD 설정파일 및 환경 변수

확장 명령어



애플리케이션 인자 값 설정 및 Containerfile

시험과 상관 없지만, 어떠한 방식으로 전달이 되는지 확인한다.

쿠버네티스는 기본적으로 이미지 빌드를 제공하지 않는다. 이미지를 올바르게 구성 및 빌드가 되어야지 쿠버네티스에서 전달하는 자원을 올바르게 활용이 가능하다.

쿠버네티스는 두 가지 형태로 자원 접근을 제공한다.

1. ConfigMap

2. Secret

ConfigMap은 환경 설정 파일 또는 변수를 관리(예: DB_HOST, APP_MODE 등)를 전달 시, 사용한다. Secret은 민감한 정보 예를 들어, 비밀번호, API 키 등 민감한 정보 관리 (예: DB_PASSWORD)를 전달 시, 사용한다.



CONTAINERFILE 빌드

Containerfile기반으로 다음과 같이 호출 및 작성이 가능하다. 이를 기반으로 Configmap, Secret를 만들어서 간단하게 확인한다.

```
vi Containerfile
```

```
FROM nginx:alpine
```

```
CMD ["sh", "-c", "echo 'DB_HOST: ' $DB_HOST; echo 'DB_PASSWORD: ' $DB_PASSWORD; nginx -g 'daemon off;']
```

```
buildah bud -t my-nginx .
```

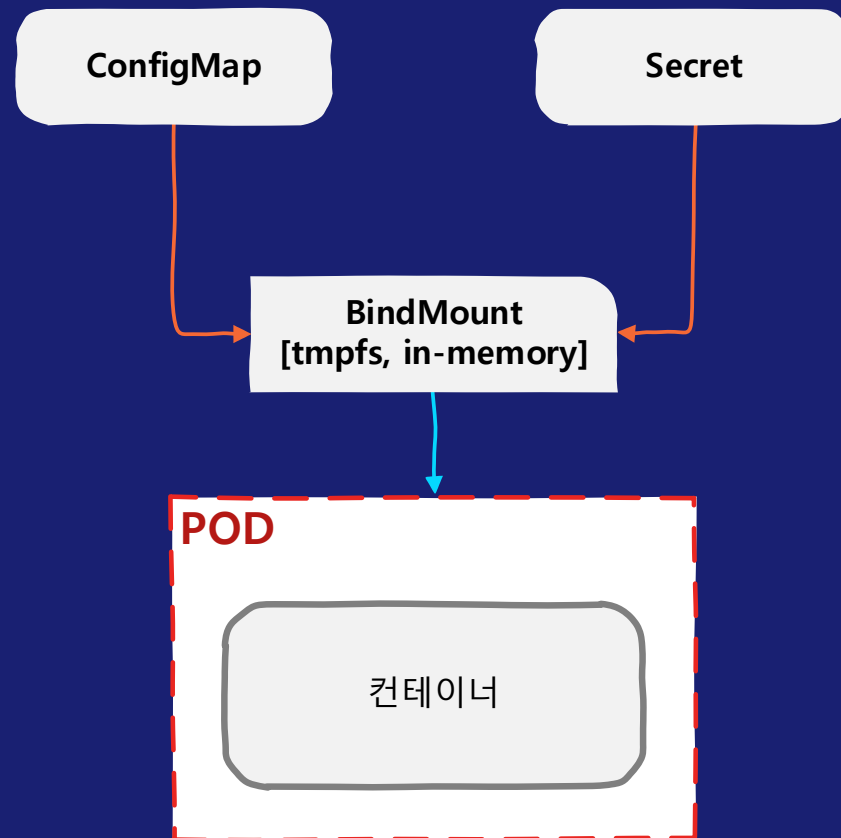


ConfigMap, Secret

확장 명령어



자원 설명



항목별 설명

위의 기능들을 정리하면 다음과 같다.

기능	설명	적용 명령어
ConfigMap 생성	환경 변수 관리	<code>kubectl apply -f my-config.yaml</code>
Secret 생성	비밀번호, API 키 관리	<code>kubectl apply -f my-secret.yaml</code>
Deployment에서 사용	환경 변수로 전달	<code>kubectl apply -f nginx-deployment.yaml</code>
데이터 확인	ConfigMap & Secret 조회	<code>kubectl get configmap my-config -o yaml</code>
		<code>kubectl get secret my-secret -o yaml</code>
Secret 복호화	base64 인코딩된 데이터 확인	<code>kubectl get secret my-secret -o jsonpath="{.data.DB_PASSWORD}"</code>



항목별 설명

항목	ConfigMap	Secret
파일로 마운트 가능 여부	가능 (configMap 볼륨 사용)	가능 (secret 볼륨 사용)
Pod 내부에서 사용 가능	가능 (env, volume)	가능 (env, volume)
보안 수준	낮음 (평문 저장)	높음 (Base64 인코딩)
RBAC 적용 가능 여부	불가능 (일반 리소스)	가능 (kubectl get secret 권한 필요)
사용 예제	<code>kubectl create configmap my-config --from-literal=DB_HOST=mydatabase.local</code>	<code>kubectl create secret generic my-secret --from-literal=DB_PASSWORD=SuperSecret123</code>



ConfigMap 생성 및 구성

명령어로 생성하는 방법은 다음과 같다.

```
kubectl create configmap my-config --from-literal=DB_HOST=mydatabase.local
kubectl get cm
cat <<EOF> my-config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  DB_HOST: "mydatabase.local"
EOF
kubectl apply -f my-config.yaml
```



Secret 생성 및 구성

명령어로 생성하는 방법은 다음과 같다. 파일로 다음과 같이 작성 및 생성이 가능하다. YAML 파일에는 인코딩 해서 인코딩 값을 파일 직접 작성해야 한다.

```
kubectl create secret generic my-secret \
--from-literal=DB_PASSWORD=SuperSecret123
```

```
kubectl get secret
echo -n "SuperSecret123" | base64
U3VwZXJTZWNYZXQxMjM=
```



Secret 파일 생성

다음과 같이 파일로 작성한다.

```
cat <<EOF> my-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
type: Opaque
data:
  DB_PASSWORD: "U3VwZXJTZWNYZXQxMjM=" # base64 인코딩된 값
EOF
kubectl apply -f my-secret.yaml
kubectl get secret my-secret -o jsonpath="{.data.DB_PASSWORD}" | base64 --
decode
```



DEPLOYMENT에 configmap, secret 구성

디플로이먼트에 적용하여, 올바르게 POD에 구성이 되었는지 확인한다.

```
cat <<EOF> data-nginx-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: data-nginx-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
```



DEPLOYMENT에 configmap, secret 구성

계속 이어서...

```
template:
  metadata:
    labels:
      app: nginx
  spec:
    containers:
      - name: nginx
        image: nginx
```



DEPLOYMENT에 configmap, secret 구성

계속 이어서...

```
env:  
- name: DB_HOST  
  valueFrom:  
    configMapKeyRef:  
      name: my-config  
      key: DB_HOST
```



DEPLOYMENT 반영 및 적용

아래는 POD에 env:를 통해서 ConfigMap, SecretKey를 POD를 통해서 컨테이너에게 전달한다.

```
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: my-secret
      key: DB_PASSWORD
```

EOF

```
kubectl apply -f data-nginx-deployment.yaml
kubectl get -f data-nginx-deployment.yaml
```



DEPLOYMENT 적용

아래는 POD에 env:를 통해서 ConfigMap, SecretKey를 POD를 통해서 컨테이너에게 전달한다.

```
kubectl logs -l app=nginx  
DB_HOST: mydatabase.local  
DB_PASSWORD: SuperSecret123  
kubectl exec -it -l nginx -- bash
```



NetworkPolicy

확장 명령어



설명

네임스페이스 간의 접근을 위한 기능으로, L2/L3/L4 레이어를 거치지 않고 라우팅을 설정함으로써 오버헤드 없이 신속하게 접근할 수 있다. 동작 방식은 아래 컴포넌트 기반으로 구성이 된다.

- Container Network Interface(CNI)
- Netfilter(iptables, nftables)

위의 기능 기반으로 백-엔드에서 다음과 같이 구성한다. 다음과 같은 조건으로 구성이 되었다.

1. Namespace: test-np
2. Pod: nginx-server
3. 허용: allowed-pod만 80 포트로 접근 가능
4. 차단: denied-pod는 접근 차단



설명

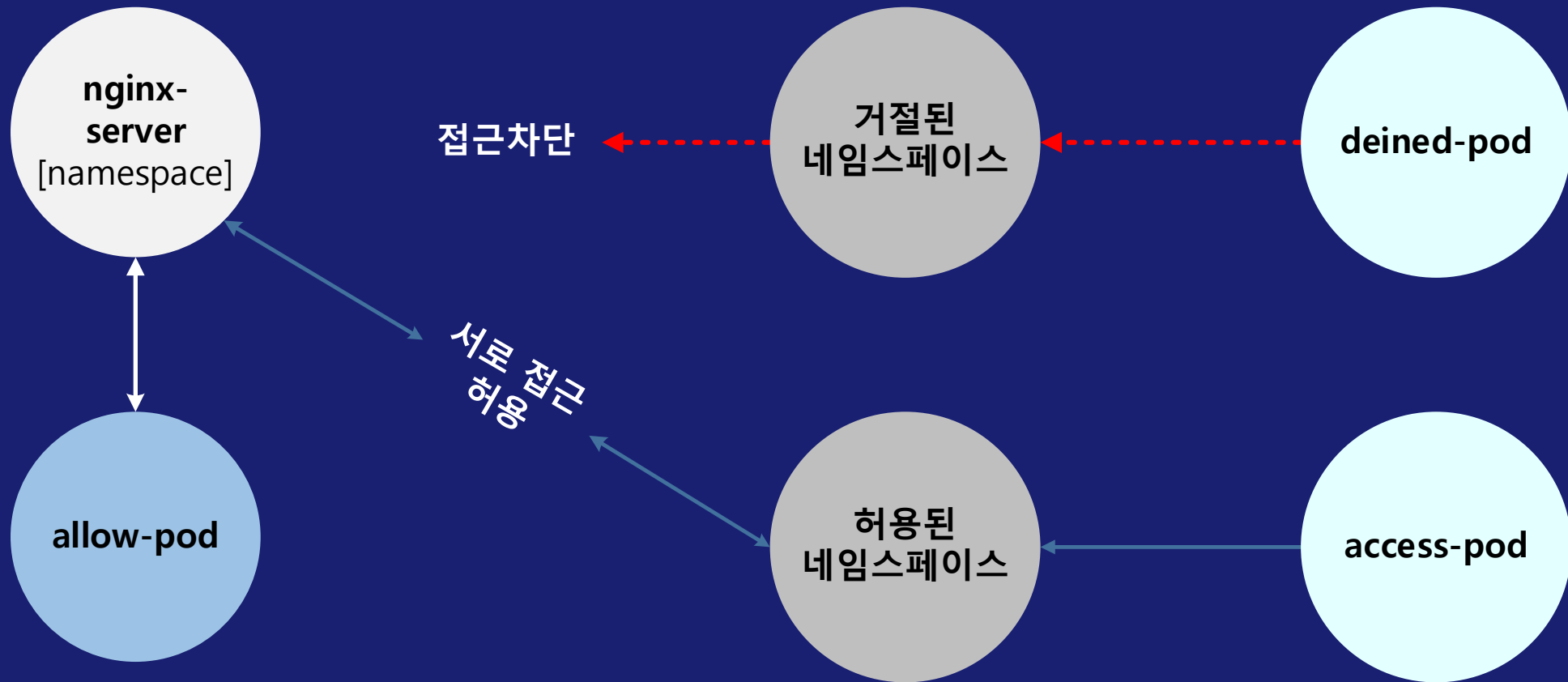
랩 설치 시, Flannel로 구성한 경우, ToR기능이 없기 때문에 NetworkPolicy가 동작하지 않는다. 이러한 이유로 NetworkPolicy 기능 사용이 필요한 경우, 다음 옵션으로 활성화가 가능하다.

`--cni calico`

위 옵션은 기본적으로 vxlan 형식으로 설치가 되기 때문에 사용에 문제가 없다.



설명



네트워크 정책 동작 방식(백엔드)

다음과 같이 시스템에서 확인이 가능하다.

1번 정책은 허용, 2번 정책은 거절. 즉, SOURCE에 명시가 되어 있는 아이피가 아닌 경우, 10.244.1.10에 접근을 거절한다.

```
iptables -L -v -n --line-numbers
```

```
Chain KUBE-NWPLCY-XXXXXX (1 references)
```

num	pkts	bytes	target	prot	opt	in	out	source	destination
1	0	0	ACCEPT	tcp	--	*	*	10.244.1.5	10.244.1.10
tcp dpt:80									
2	0	0	DROP	all	--	*	*	0.0.0.0/0	10.244.1.10



네트워크 정책 동작 방식(백엔드)

Cilium를 사용하는 경우, 다음과 같이 nftables에 구성이 된다. CNI마다 다르지만, 보통 구성은 비슷하다.

네임스페이스의 특정 POD를 제약을 하지만, 네임스페이스 자체는 트래픽 라우팅을 조정할 수 없기 때문에, POD 아 이피 기반으로 한다. 이를 구현하기 위해서 모든 POD는 올바른 레이블이 할당 및 구성이 되어 있어야 한다.

```
iptables -L -v -n --line-numbers
table ip kube-nwpolicy
{
    chain ingress {
        ip saddr 10.244.1.5 tcp dport 80 accept
        drop
    }
}
```



랩 시작 전 네임스페이스 생성

랩 에서 사용할 네임스페이스를 미리 생성 및 구성한다.

```
cat <<EOF> ns-test-np.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: test-np
  labels:
    project: restricted
    env: dev
EOF
kubectl apply -f ns-test-np.yaml
```



랩 시작 전 네임스페이스 생성

랩 에서 사용할 네임스페이스를 미리 생성 및 구성한다.

```
cat <<EOF> ns-dev-ns.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev-ns
  labels:
    env: dev
EOF
kubectl apply -f ns-dev-ns.yaml
```



랩 시작 전 네임스페이스 생성

랩 에서 사용할 네임스페이스를 미리 생성 및 구성한다.

```
cat <<EOF> ns-restricted-ns.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: restricted-ns
  labels:
    project: restricted
EOF
kubectl apply -f ns-restricted-ns.yaml
kubectl get ns | grep -e dev-ns -e test-ns -e restricted-ns
```



POD 생성(클라이언트)

테스트용 POD를 생성한다.

```
cat <<EOF> allowed-client.yaml
apiVersion: v1
kind: Pod
metadata:
  name: allowed-client
  namespace: test-np
  labels:
    role: allowed
```



POD 생성(클라이언트)

테스트용 POD를 생성한다.

```
spec:
  containers:
  - name: curl
    image: curlimages/curl
    command: ["sleep", "3600"]
```

EOF

```
kubectl apply -f allowed-client.yaml
kubectl get -f allowed-client.yaml
```



POD 생성(프론트, 허용대상)

테스트용 POD를 생성한다.

```
cat <<EOF> frontend-client.yaml
apiVersion: v1
kind: Pod
metadata:
  name: frontend-client
  namespace: dev-ns
  labels:
    team: frontend
```



POD 생성(프론트, 허용대상)

테스트용 POD를 생성한다.

```
spec:
  containers:
  - name: curl
    image: curlimages/curl
    command: ["sleep", "3600"]
EOF
kubectl apply -f frontend-client.yaml
kubectl get -f frontend-client.yaml
```



POD 생성(웹서버, 보호대상)

테스트용 POD를 생성한다.

```
cat <<EOF> web-server.yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-server
  namespace: test-np
  labels:
    app: web
```



POD 생성(웹서버, 보호대상)

테스트용 POD를 생성한다.

```
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
    - containerPort: 443
```

EOF

```
kubectl apply -f web-server.yaml
```

```
kubectl get -f web-server.yaml
```



기본 차단 정책(ingress)

NetworkPolicy는 명시적으로 deny를 쓰지 않는다. 아무것도 허용하지 않는 정책이 있어야 차단이 시작된다.

```
cat <<EOF> default-deny-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: test-np
spec:
  podSelector: {}
  policyTypes:
    - Ingress
EOF
kubectl apply -f default-deny-ingress.yaml
kubectl get -f default-deny-ingress.yaml
```



테스트용 서비스 구성

NetworkPolicy는 명시적으로 deny를 쓰지 않는다. 아무것도 허용하지 않는 정책이 있어야 차단이 시작된다.

```
cat <<EOF> svc-web-svc.yaml
apiVersion: v1
kind: Service
metadata:
  name: web-svc
  namespace: test-np
spec:
  selector:
    app: web
```



테스트용 서비스 구성

NetworkPolicy는 명시적으로 deny를 쓰지 않는다. 아무것도 허용하지 않는 정책이 있어야 차단이 시작된다.

```
ports:
- name: http
  port: 80
  targetPort: 80
  protocol: TCP
- name: https
  port: 443
  targetPort: 443
  protocol: TCP
```

EOF

```
kubectl apply -f svc-web-svc.yaml
kubectl get -f svc-web-svc.yaml
```



확인 및 검증

아래 명령어로 확인 및 검증한다. 자원 및 차단 상태를 확인한다.

```
kubectl get ns --show-labels | grep -e test-np -e dev-ns -e restricted-ns  
kubectl get pod,svc -n test-np -o wide  
kubectl get networkpolicy -n test-np
```



차단 POD 생성(restricted)

차단 POD를 생성 후 올바르게 동작하는지 확인한다. 3600초 동안 POD는 대기한다.

```
kubectl run restricted-client \  
  -n restricted-ns \  
  --image=curlimages/curl \  
  --restart=Never \  
  -- sleep 3600
```



차단 테스트 (restricted namespace)

아래 명령어로 확인 및 검증한다.

- project=restricted 네임스페이스는 전부 차단.

```
kubectl exec -n restricted-ns restricted-client -- \
  curl -m 3 http://web-svc.test-np.svc.cluster.local
curl: (28) Connection timed out
```



허용 테스트 ① (role=allowed Pod)

아래 명령어로 확인 및 검증한다.

- allow-specific-pod NetworkPolicy 동작 확인

```
kubectl run allowed-client \  
  -n test-np \  
  --image=curlimages/curl:8.6.0 \  
  --labels="role=allowed" \  
  --restart=Never \  
  --command -- sleep 3600
```

```
kubectl exec -n test-np allowed-client -- \  
  curl http://web-svc  
<html> ... </html>
```



허용 테스트 ② (env=dev + team=frontend)

아래 명령어로 확인 및 검증한다.

- namespaceSelector + podSelector 조합 성공

```
kubectl run frontend-client \  
  -n dev-ns \  
  --image=curlimages/curl:8.6.0 \  
  --labels="team=frontend" \  
  --restart=Never \  
  --command -- sleep 3600
```

```
kubectl exec -n dev-ns frontend-client -- \  
  curl https://web-svc.test-np.svc.cluster.local:443 -k  
<html> ... </html>
```



쿠버네티스 고급기능

ingress



인그레스 서비스 설명

인그레스 서비스는 이전 아키텍처에서 프록시 서버(Proxy Server)와 동일한 기능을 한다.

Ingress서비스가 나오기 전에는 NodePort 혹은 Load Balancer서비스를 통해서 외부에서 접근이 되어야 한다. 다만, 이 방식의 문제는 서비스가 구성이 될 때, 관리자는 다음과 같은 부분을 수동으로 구성해야 한다.

- 서비스에서 사용할 아이피 할당(VIP)
- VIP를 통해서 접근 할 도메인 설정(Nginx, HAProxy, Appliance...)

이러한 이유로 쿠버네티스는 자동화 및 빠른 서비스 배포를 위해서 Ingress 서비스가 추가가 되었다.



인그레스 서비스 설명

Ingress는 Kubernetes 클러스터 외부의 HTTP/HTTPS 요청을 내부 서비스로 전달하기 위한 L7(애플리케이션 계층) 라우팅 리소스이다.

Ingress 자체는 트래픽을 처리하지 않으며, 실제 요청 처리는 NGINX, HAProxy, Traefik과 같은 Ingress Controller가 담당한다.

Ingress를 사용하면 하나의 진입 지점에서 다수의 서비스에 대해 Host 또는 Path 기반 라우팅, TLS 종료, Rewrite 구성이 가능하다.

Ingress API는 더 이상 새로운 기능이 추가되지 않는 Feature Freeze 상태이며, 향후 Kubernetes 네트워크 확장은 Gateway API를 중심으로 발전하고 있다.

Kubernetes에서는 기존 Ingress API를 유지하지만, 새로운 네트워크 기능은 Gateway API에서 개발되고 있다.

<https://gateway-api.sigs.k8s.io/>



인그레스 서비스 설명

Ingress는 구조가 단순하고 사용이 쉬운 장점이 있지만 다음과 같은 단점이 있다.

1. 세밀한 트래픽 제어가 어려움
2. 역할 분리가 불명확
3. 인증·권한·정책 확장이 제한적인 구조라는 한계

이러한 이유로 Kubernetes 커뮤니티에서는 Ingress API를 유지하되, 신규 기능 확장보다는 안정성 유지 수준으로 관리하고 있으며, 차세대 표준으로 Gateway API 사용을 권장하는 방향으로 전환 중이다.

Ingress 자원을 그대로 유지가 되지만 점점 Gateway API서비스로 전환을 권장하고 있다.



쿠버네티스 잉그레스 미래

Ingress 서비스 중 많이 사용하는 Nginx는 다음처럼 발표가 되었다.

<https://kubernetes.io/blog/2025/11/11/ingress-nginx-retirement/>

생태계의 안전성과 보안을 최우선으로 고려하여, Kubernetes SIG Network와 보안 대응 위원회(Security Response Committee)는 Ingress NGINX의 단계적 종료(retirement)를 예고합니다.

Ingress NGINX는 2026년 3월까지 최선의 노력(best-effort) 수준의 유지보수가 계속됩니다.
그 이후에는 추가 릴리스, 버그 수정, 그리고 새롭게 발견되는 보안 취약점에 대한 업데이트가 더 이상 제공되지 않습니다.

기존에 배포된 Ingress NGINX는 계속 동작하며, 설치 아티팩트 또한 계속 제공될 예정입니다.
Ingress NGINX에서 다른 대안으로의 마이그레이션을 권장합니다. Gateway API는 Ingress의 현대적인 대체 기술로, 우선적으로 고려할 것을 권장합니다.

만약 Ingress를 계속 사용해야 한다면, Kubernetes 공식 문서에 나열된 다양한 대체 Ingress Controller를 검토할 수 있습니다.



인그레스 VS GATEWAY API

항목	Ingress	Gateway API
역할 분리	없음	Gateway / Route 분리
트래픽 제어	제한적	매우 정교
확장성	낮음	높음
미래 표준	유지	권장



소프트웨어 기능 비교 #1

대표적인 Ingress 소프트웨어를 정리하면 다음과 같다. 기업에서 많이 사용하는 레드햇 OpenShift 포함이다.

항목	Ingress Controller	Istio Ingress Gateway	OpenShift Router
개념	외부 트래픽 라우팅	Istio 서비스 메쉬의 일부 고급 트래픽 관리를 제공	OpenShift 전용 라우터로, HAProxy 기반으로 동작
라우팅 기능	URL 및 호스트 기반 라우팅	정교한 트래픽 관리 A/B 테스트, canary 자체 지원	URL 및 호스트 기반 라우팅 지원
TLS 종료	지원	TLS 종료 및 mTLS 지원 암호화된 트래픽 처리 가능	자동 TLS 종료 및 관리
로드밸런싱	로드밸런싱 제공	고급 로드밸런싱 및 트래픽 제어 기 능 제공	로드밸런싱 제공



소프트웨어 기능 비교 #2

오픈시프트 라우터 경우에는 Istio에서 제공하는 AB 및 Canary를 지원한다.

항목	Ingress Controller	Istio Ingress Gateway	OpenShift Router
정책/보안	기본적인 인증/인가 기능 (어노테이션 활용)	mTLS, 인증 고급 보안 정책 적용 가능	OpenShift 보안 정책과 연동
서비스 메쉬 통합	주로 단독으로 사용되며, 서비스 메쉬와는 별개	Istio 서비스 메쉬 내의 게이트웨이로 동적 구성 지원	OpenShift 클러스터 내부에 통합되어 운영됨
모니터링/로깅	제한적 (별도의 도구가 필요)	텔레메트리 로깅 모니터링 기능 제공	OpenShift의 모니터링 시스템과 연동





로드밸런서 구성 및 테스트

METALLB



진행 전!

진행 전, 왜 MetalLB를 설치 및 구성을 권장하는지 아래 정리한다.

1. Ingress 경우에는 특히, Nginx는 Pure Load Balancer로 MetalLB를 권장한다.
2. 혹은 NodePort를 사용해서 구성한다. 다만, 이 방식은 kube-proxy를 통해서 포트를 제공한다.
3. hostNetwork:를 사용해서 구성한다. 하지만, 이 방식은 권장하지 않는다.

쿠버네티스 기반의 RKE2는 NodePort 혹은 hostNetwork: 기반으로 구성이 된다. 하지만, 바닐라 쿠버네티스에 서는 hostNetwork: 사용보다는 Pure Load balancer 사용을 적극적으로 권장한다.

정보 확인 주소

<https://github.com/kubernetes/ingress-nginx/blob/main/docs/deploy/baremetal.md>



METALLB 소개

Kubernetes는 서비스(Service)를 통해 파드(Pod)를 외부에 노출하는 여러 방식을 제공한다.

그중 LoadBalancer 타입 서비스는 클라우드 환경에서는 매우 자연스럽게 동작하지만, 베어메탈이나 프라이빗 클라우드 환경에서는 기본적으로 동작하지 않는다.

이유는 간단하다. Kubernetes 자체는 "외부에서 접근 가능한 IP를 어떻게 할당 할 것인가?"에 대해 구현을 포함하지 않고, 이를 외부 인프라에 위임하도록 설계되었기 때문이다.

이 공백을 메우기 위해 등장한 대표적인 솔루션이 MetalLB이다.



MetalLB의 역할과 목적

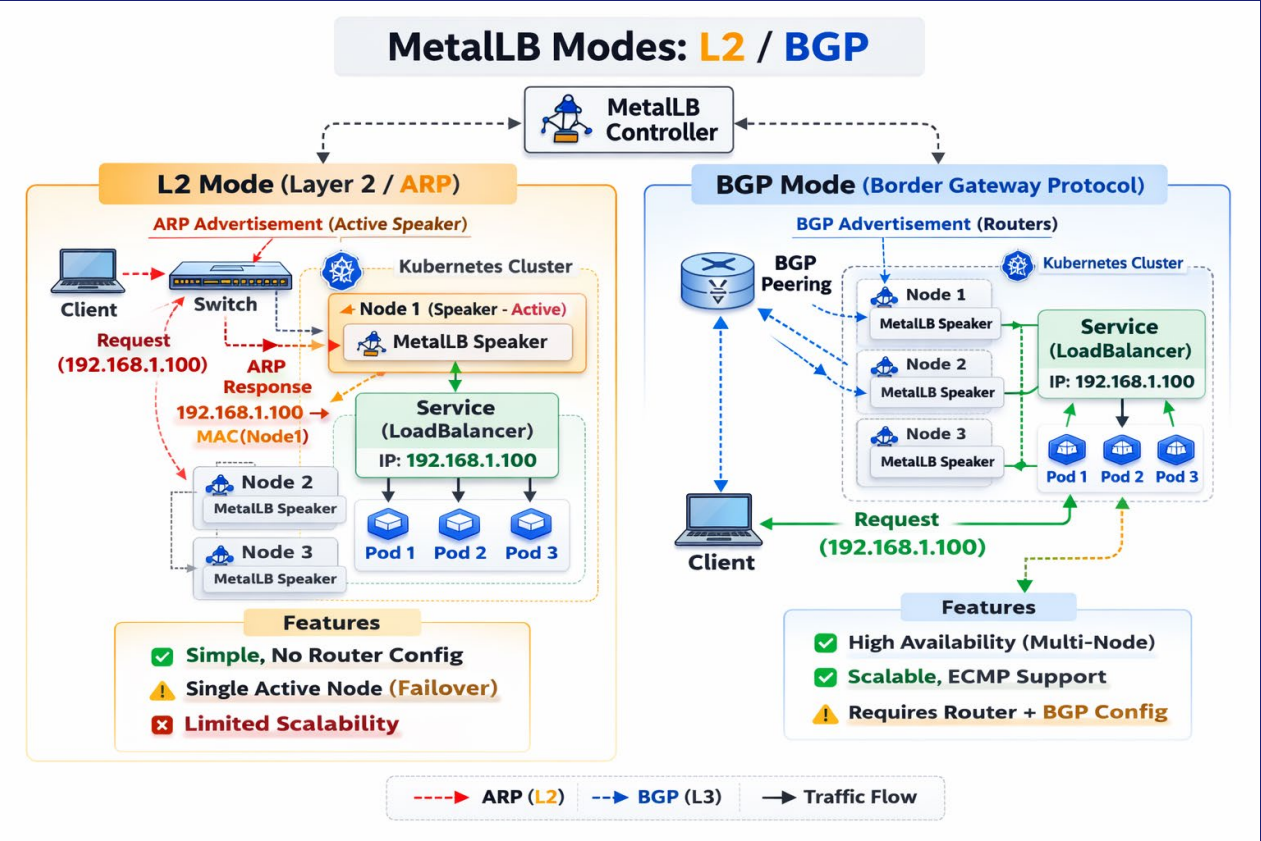
MetalLB는 베어메탈 및 프라이빗 환경에서 Kubernetes의 LoadBalancer 서비스를 구현해주는 컴포넌트이다. 즉, 클라우드 제공자의 로드밸런서(AWS ELB, GCP LB 등)가 없는 환경에서도 Service type=LoadBalancer를 정상적으로 사용할 수 있도록 만들어준다.

MetalLB의 핵심 역할은 다음과 같다.

- 외부에서 접근 가능한 가상 IP(VIP)를 할당
- 해당 IP로 유입된 트래픽을 Kubernetes Service로 전달
- Service 뒤에 연결된 Pod들로 트래픽 분산

MetalLB는 애플리케이션 로드밸런싱을 직접 수행하지 않으며, IP 할당과 네트워크 광고 역할에 집중한다는 점이 특징이다.





MetalLB의 동작 방식

L2 모드

L2 모드는 가장 단순하고 직관적인 방식이다. MetalLB는 특정 노드 하나를 선택해 VIP의 소유자(owner)로 지정하고, 해당 노드가 ARP(NDP) 응답을 통해 "이 IP는 내가 가진 IP"라고 네트워크에 알린다.

외부에서 VIP로 들어온 트래픽은 자연스럽게 그 노드로 전달되고, 이후 Kubernetes Service를 통해 Pod로 분산된다.

이 방식은 다음과 같은 장점을 가지고 있다.

- 설정이 단순
- 외부 네트워크 장비 설정이 필요 없다.
- 교육/랩/프로덕트 프라이빗 클라우드 환경에 매우 적합하다

대다수 바닐라 쿠버네티스는 오픈소스 버전의 OpenStack(온프레미스 환경)에서 가장 현실적인 선택이다.



MetalLB의 동작 방식

BGP 모드

BGP 모드는 MetalLB가 외부 라우터 또는 스위치와 BGP 피어링을 맺고, VIP를 라우팅 정보로 광고하는 방식이다.

이 방식의 동작은 다음과 같은 메커니즘을 가지고 있다.

- 여러 노드가 동시에 VIP를 광고 가능
- 고가용성과 대규모 확장에 유리하지만, 네트워크 장비에 대한 제어 권한과 BGP 이해가 필요

주로 대형 데이터센터 환경에서 사용되며, 프라이빗 클라우드나 OpenStack 환경에서는 적용이 쉽지 않다.



MetalLB의 구성 요소

MetalLB는 Kubernetes CRD(Custom Resource Definition)를 기반으로 구성된다.

- IPAddressPool
MetalLB가 사용할 수 있는 IP 주소 범위를 정의한다.
- L2Advertisement/BGPPeer
정의된 IP 풀을 어떤 방식(L2 또는 BGP)으로 네트워크에 광고할지 결정한다.

이러한 구성은 YAML 기반으로 선언적으로 관리되며, Helm을 통한 설치와 GitOps 방식 운영에 잘 어울린다.



MetalLB가 수행하지 않는 것

MetalLB는 종종 로드밸런서라는 이름 때문에 오해를 받지만, 다음과 같은 작업은 수행하지 않는다.

- L7 트래픽 처리 (HTTP 라우팅, TLS 종료 등)
- 실제 로드밸런싱 알고리즘 구현
- 애플리케이션 레벨 프록시 역할

이러한 기능은 Ingress Controller(NGINX, HAProxy, Envoy 등)의 영역이며, MetalLB는 그 앞단에서 IP를 제공하는 역할을 담당한다.

또한, 오픈 시프트의 라우트와 동일한 기능이 아니다.



METALLB VS KUBE ROUTER

쿠버네티스에서 제공하는 KUBE ROUTER는 얼핏 보기에는 L4/L7기능을 제공하는 것처럼 보이지만, 실제로는 전혀 기능이 다르다.

구분	MetalLB	kube-router
목적	LoadBalancer 구현	CNI + Routing
L2 모드	가능	불가능
BGP 모드	가능	제한적 가능
Helm/CRD	매우 성숙	제한적
OpenStack 적합성	매우 적절	사용 가능하나, 부적절
운영 난이도	낮음	높음
교육/랩	최적	부적합



아이피 대역 선언

MetalLB에서 사용할 아이피 대역을 아래와 같이 할당 및 선언한다.

```
cat <<EOF> metallb-ippool.yaml
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: pool-10-10-250
  namespace: metallb-system
spec:
  addresses:
    - 10.10.250.1-10.10.250.250
EOF
kubectl apply -f metallb-ippool.yaml
```



계
속

L2 광고 선언

```
cat <<EOF> metallb-l2adv.yaml
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: l2-ens3
  namespace: metallb-system
spec:
  ipAddressPools:
    - pool-10-10-250
  interfaces:
    - ens3
```

EOF

```
kubectl apply -f metallb-l2adv.yaml
```



설치 시작

위의 내용이 준비가 되었으면, 아래와 같이 Helm으로 설치를 진행한다.

```
helm repo add metallb https://metallb.github.io/metallb  
helm repo update
```

```
kubectl create namespace metallb-system --dry-run=client -o yaml | kubectl  
apply -f -
```

```
helm install metallb metallb/metallb \  
-n metallb-system
```



설치 및 로드밸런서 확인

올바르게 설치가 되었는지 아래 명령어로 확인이 가능하다.

```
kubectl create deploy lb-test --image=nginx --replicas=1  
kubectl expose deploy lb-test --port 80 --type LoadBalancer
```

```
kubectl get svc lb-test -o wide
```

```
lb-test    LoadBalancer    10.111.115.54    10.10.250.2    80:30462/TCP
```



확인(OSP)

오픈스택에서 Octavia를 사용하지 않는 경우, PORT기반으로 아이피 할당 후 MetalLB에서 사용한다. 일반적인 베어메탈 경우에는 별도로 선택할 내용은 따로 없다.

☐ lb-ip-240 • 192.168.0.240 fa:16:3e:a8:d8:8e 연결 해제됨 Down True

이름
lb-ip-241

☒ 관리 상태 활성화 ⓘ

장치 ID ⓘ

장치 소유자 ⓘ

IP 주소 또는 서브넷을 지정합니다 ⓘ
고정 IP 주소 ▼

고정 IP 주소* ⓘ
192.168.0.241

MAC 주소 ⓘ

☐ 포트 보안 ⓘ

VNIC 유형 ⓘ
표준 ▼

☐ lb-ip-240 • 192.168.0.240 fa:16:3e:a8:d8:8e 연결 해제됨 Down True

```
# kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
nginx-lb	LoadBalancer	11.88.223.28	192.168.0.240	80:30760/TCP	23m
nginx-lb-2	LoadBalancer	11.88.184.114	192.168.0.241	80:31822/TCP	3s
nginx-np	NodePort	11.88.176.235	<none>	80:31526/TCP	81m

```
# arp -e
```

192.168.0.240	(incomplete)	cni0
192.168.0.241	(incomplete)	cni0



인그레스 서비스 테스트

INGRESS



Kubernetes Ingress Service 개요와 변화 방향

Ingress는 Kubernetes 클러스터 외부의 HTTP/HTTPS 요청을 내부 서비스로 전달하기 위한 L7(애플리케이션 계층) 라우팅 리소스이다.

Ingress 자체는 트래픽을 처리하지 않으며, 실제 요청 처리는 NGINX, HAProxy, Traefik과 같은 Ingress Controller가 담당한다. Ingress를 사용하면 하나의 진입 지점에서 다수의 서비스에 대해 Host 또는 Path 기반 라우팅, TLS 종료, Rewrite 구성이 가능하다.



Ingress의 한계와 커뮤니티 변화

Ingress는 구조가 단순하고 사용이 쉬운 장점. 하...하지만!!

- 세밀한 트래픽 제어가 어렵고
- 역할 분리가 불명확하며
- 인증·권한·정책 확장이 제한적인 구조라는 한계가 존재한다.

이러한 이유로 Kubernetes 커뮤니티에서는 Ingress API를 유지하되, 신규 기능 확장보다는 안정성 유지 수준으로 관리하고 있으며, 차세대 표준으로 Gateway API 사용을 권장하는 방향으로 전환 중이다.

Ingress는 Service Mesh 기준으로는 트래픽 제어와 정책 표현에 적합하지 않은 구조이며, 이러한 한계로 인해 Gateway API가 표준으로 제안되었다.

Ingress는 주소 및 도메인 기반 모델이고, Router/Route 개념을 일반화한 Gateway API가 Kubernetes 네트워크의 표준 방향이 되었다.

하지만, 완전히 Ingress 서비스는 사라지지 않는다.



인그레스 랩 주의사항

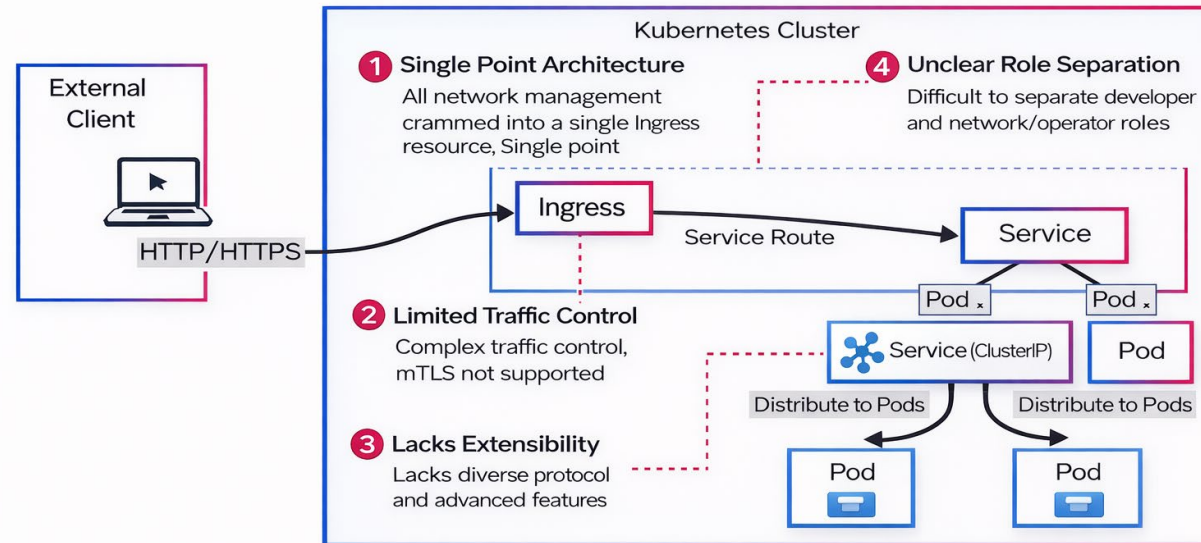
현재 랩은 DNS서버가 구성이 안되어 있다. 이 부분은 강사가 별도로 설명.



인그레스 동작 설명 #1

Why Kubernetes Ingress is No Longer Central to Network Management?

Ingress has long been the entry point for external traffic to Kubernetes clusters, but for several reasons, it has struggled to remain the central component of network management. The key structural limitations and issues with Ingress are:



Conclusion: Ingress is sufficient for simple web routing but clearly limited for use as a platform-wide



인그레스 동작 설명 #2(단순버전)



NGINX 제품 설명

일반적으로 많이 사용하는 제품. 지속적인 지원 및 API 서비스 사용을 원하는 경우 상용으로 사용을 권장.

개발회사

- 쿠버네티스 SIG 및 Nginx(f5)

주요특징

- 비교적 오래되고 안정적인 프로젝트, TCP/UDP, gRPC 등 L4/L7 레벨 트래픽 처리 지원

확장성 및 추가 기능

- 기본 인증, IP 화이트리스트 등 Helm Chart, Operator 등 배포 방식 다양

사용 사례/장점

- 가장 널리 사용되는 Ingress Controller 커뮤니티 자료와 사례가 풍부



HAPROXY 제품 설명 1

레드햇 오픈시프트에서 제공하는 라우터가 HAPROXY 기반으로 지원 및 제공.

개발회사

- HAProxy Technologies

주요 특징

- 고성능 L4/L7 로드밸런싱
- 다양한 헬스 체크 및 동적 설정 지원
- Kubernetes 인프라와의 연동성 개선



HAPROXY 제품 설명 2

확장성 및 추가 기능

- Lua 스크립트 확장 가능
- TLS 종료, SNI
- 인증 등 고급 기능 지원

사용 사례/장점

- HAProxy의 높은 성능과 안정성을 그대로 사용
- 대규모 트래픽 환경에서 효과적



Traefik 제품 설명

현재 Ingress, Gateway를 동시에 지원하는 오픈소스 솔루션. 상용 버전도 별도로 지원.

개발회사

- Traefik Labs

주요 특징

- 라우팅 규칙을 선언적으로 설정, 다양한 백엔드와 통합, ACME/Let's Encrypt 자동 TLS

확장성 및 추가 기능

- 인그레스 라우팅 외에도 Service Mesh가능, 여러 프로토콜 지원

사용 사례/장점

- 설정이 비교적 간단하고 UI/대시보드 제공, DevOps/마이크로서비스 환경에서 자주 사용



ISTIO 제품 설명(API GATEWAY) 1

제일 많이 사용하는 Proxy서비스. 쿠버네티스와 제일 통합이 잘 되어 있고, 사용하기가 편하다.

개발회사

- Istio

주요 특징

- Envoy Proxy 기반
- Service Mesh 환경에서 인그레스 역할 담당
- 세밀한 트래픽 제어, 정책 적용 가능



ISTIO 제품 설명(API GATEWAY) 2

확장성 및 추가 기능

- Istio 설치가 필요, mTLS
- 라우팅 규칙 등 Service Mesh 기능과 결합

사용 사례/장점

- Istio Mesh를 이미 사용 중인 환경
- L7 레벨 세밀한 트래픽 제어와 정책 적용이 필요한 경우



INGRESS 설치

설치가 되어 있지 않으면, 간단하게 설치가 가능하다. 모든 패키지는 가급적이면 HelmChart기반으로 설치 및 관리한다. 쿠버네티스 기본 패키지 관리자는 Helm이다.

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
```

```
helm install nginx-ingress ingress-nginx/ingress-nginx --namespace ingress-
nginx --create-namespace
```

```
NAME: nginx-ingress
```

```
LAST DEPLOYED: ...
```

```
NAMESPACE: ingress-nginx
```

```
STATUS: deployed
```



구성

인프라가 올바르게 구성이 되어 있다면, 다음과 같이 인그레스 서비스를 구성한다. 매우 기본적인 기능만 사용한다.

```
cat <<EOF> web-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web-ingress
  namespace: default
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  rules:
```



구성

임시 도메인은 `example.com`으로 선언, 문제가 없으면 접근이 가능하다.

```
- host: example.com
  http:
    paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

EOF

```
kubectl apply -f web-ingress.yaml
```



확인 및 접근

올바르게 구성이 되었는지 확인한다.

```
kubectl apply -f web-ingress.yaml
```

```
kubectl get -f web-ingress.yaml
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
web-ingress	<none>	example.com	203.0.113.10	80	1m

```
kubectl get svc -n ingress-nginx
```

ingress-nginx	nginx-ingress-nginx-controller
LoadBalancer	11.88.128.115 192.168.0.242

```
kubectl get ingress
```

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
web-ingress	<none>	example.com		80	4m14s



INFRA IN KUBERNETES

GOGS

REGISTRY



쿠버네티스 인프라 구성 설명

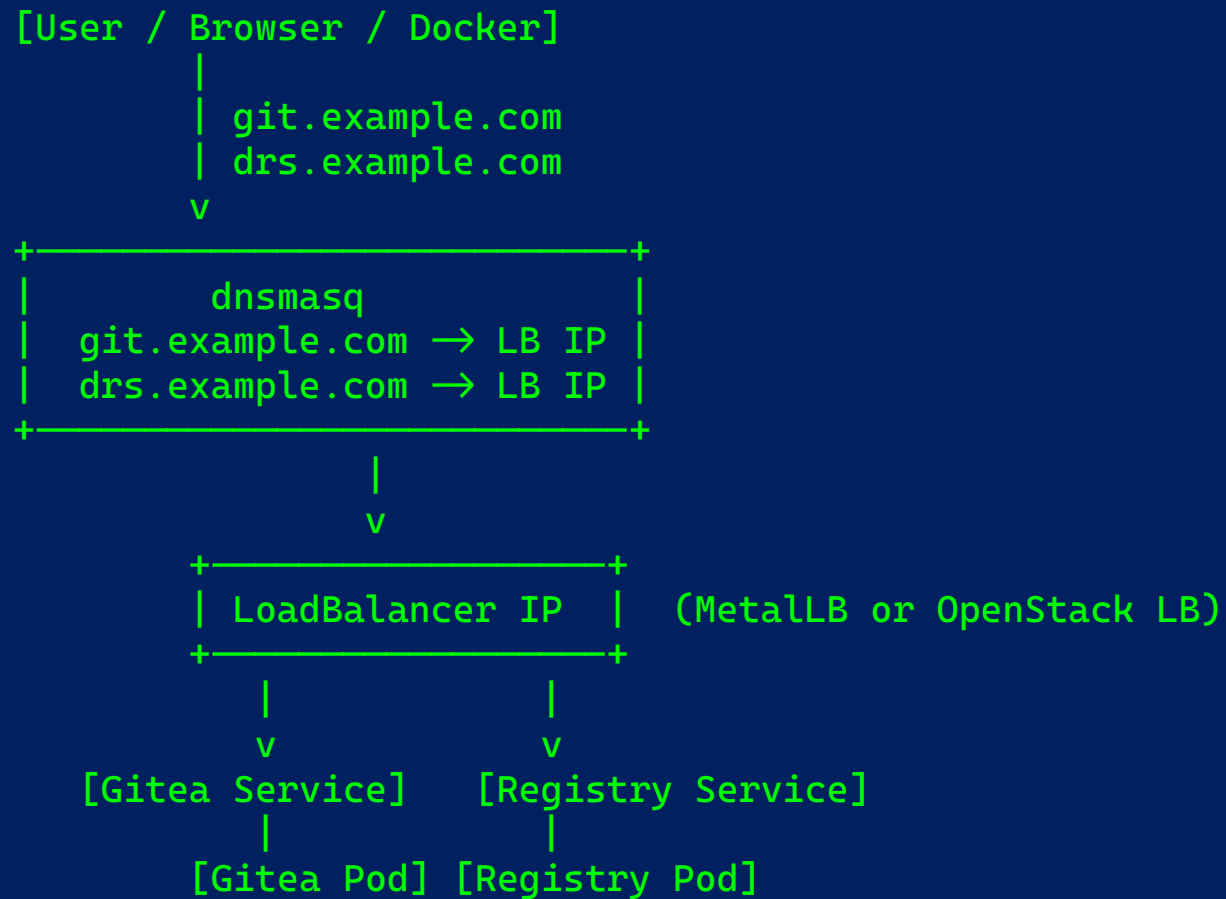
기존 포드만에 구성이 된 인프라 서비스를 쿠버네티스 내부로 가지고 오기 위해서 아래와 같이 구성이 가능함. 다만, 구성하기 위해서 다음과 같은 서비스가 구성이 되어 있어야 됨.

1. LoadBalancer(External/CNCF 둘 다 상관없음)
2. Proxy Server(HAProxy, Ingress, OpenELB)

클러스터 내부로 인프라를 가지고 오기 위해서 아래와 같이 YAML작성 후, 쿠버네티스 클러스터에 적용.



인프라 서비스 아키텍처



인프라 서비스 설치 스크립트

마지막 과정인 간단한 CI/CD 활용을 위해서 아래와 같이 스크립트를 실행한다. 다음과 같은 부분이 자동으로 처리가 된다.

1. gitea

2. docker registry v2

3. dnsmasq

```
curl -fsSL http://10.0.0.250/scripts/deploy.sh | bash
```

```
vi /etc/dnsmasq.conf
```

```
interface=ens3
```

```
systemctl restart dnsmasq
```



CI/CD

TEKTON

GIT SERVER

IMAGE REGISTRY



TEKTON

CI 자동화 맛보기



Tekton이란?

Tekton 개요

Kubernetes Native CI/CD 프레임워크

- CNCF Incubating 프로젝트
- CI/CD 파이프라인을 Kubernetes 리소스(CRD) 로 정의
- 컨테이너 기반 실행, 서버리스 구조
- CI 기능이 주요 핵심이며, POD 기반으로 CD 구현 및 지원
- YAML 기반으로 빌드 된 이미지 배포 및 서비스 구성도 지원



테크톤 특징

Kubernetes 중심 설계

- Pipeline, Task, Step 모두 CRD

Pod 단위 실행

- 각 Step은 컨테이너로 실행

YAML 기반 선언형 구성

- GitOps와 궁합 매우 좋음

클러스터 리소스 활용 최적화

- 필요할 때만 Pod 생성 → 자동 종료



테크톤 주요 구성 요소

주요 구성 요소

1. Task: 단일 작업 단위 (build, test 등)
2. Pipeline: Task들의 실행 흐름
3. PipelineRun/TaskRun: 실제 실행 객체
4. Trigger: Git Webhook 등 이벤트 기반 실행



Tekton vs Jenkins 기능 비교

구분	Tekton	Jenkins
아키텍처	Kubernetes Native	중앙 서버(Master)
실행 방식	Pod 기반 (컨테이너)	Agent(Node) 기반
확장성	쿠버네티스 자원과 모두 연결이 가능	Jenkins X가 아닌 경우 제한적
구성 방식	YAML/CRD	Jenkinsfile (Groovy)
상태 관리	Stateless	Stateful
장애 영향	개별 Task 영향	Master 장애 시 전체 영향
GitOps 친화성	매우 높음	보통
러닝 커브	다소 높음	비교적 낮음



Tekton VS Jenkins? #1

언제 Tekton을 선택할까?

1. Kubernetes 기반 플랫폼
2. 폐쇄망/On-Prem 환경
3. GitOps/IaC 중심 운영
4. 멀티 파이프라인, 멀티 팀 환경



Tekton VS Jenkins? #2

언제 Jenkins가 나올까?

1. 단일 서버 CI
2. 기존 Jenkins 자산 많을 때
3. 빠른 PoC/소규모 팀



ArgoCD vs Tekton 한눈에 보기

역할 정의

Argo CD

- Kubernetes GitOps 기반 CD(Continuous Delivery) 도구
- Git=단 하나의 진실(Source of Truth)
- 클러스터의 애플리케이션 및 서비스 상태를 Git과 동일하게 유지

Tekton

- Kubernetes Native CI/Pipeline 엔진
- 빌드, 테스트, 이미지 생성 등 작업 실행 담당
- 컨테이너 기반 작업 흐름 실행



ArgoCD vs Tekton 비교

구분	ArgoCD	Tekton
주요 역할	CD (배포)	CI/Pipeline
중심 개념	GitOps	Task/Pipeline
실행 단위	Kubernetes 리소스 Sync	Pod (컨테이너)
상태 관리	선언형(State Sync)	실행형(Job)
트리거	Git 변경	Event/Webhook
UI	강력한 Web UI	CLI 중심 (tkn)
장애 영향	Sync 단위	Task 단위
Kubernetes 의존성	매우 높음	매우 높음



같이 사용하는 경우

```
[ Developer ]
  |
  v
[ Git Push ]
  |
  v
[ Tekton ]
- Build
- Test
- Image Push
  |
  v
[ Git (Manifest Update) ]
  |
  v
[ ArgoCD ]
- Sync
- Deploy
- Drift Detection
```



마지막으로...

Tekton 기반 CD(Continuous Delivery)를 사용할 때의 장점과 단점

Tekton은 본래 CI 중심의 파이프라인 엔진이지만, Task와 Pipeline을 활용해 배포까지 포함한 CD 흐름을 구성할 수도 있다.

특히 폐쇄망, On-Premise, Kubernetes Native 환경에서는 Tekton 기반 CD를 선택하는 경우도 많다. 다만, Tekton을 CD까지 확장해 사용하는 것은 강력한 유연성을 얻는 대신 운영 복잡성을 감수하는 선택이 된다.

→ Kubernetes 자체가 배포 엔진!!



마지막으로...

장점, 1. 완전한 Kubernetes Native CD

Tekton 기반 CD는 배포 과정 전체를 Kubernetes 리소스와 Pod 실행으로 처리한다. 별도의 CD 서버나 외부 에이전트 없이, 클러스터 내부에서 배포가 수행된다.

- Kubernetes RBAC 그대로 활용 가능
- 네트워크, 보안 정책 일관성 유지
- 클러스터 리소스 사용 최적화



마지막으로...

장점, 2. 높은 유연성과 자동화 자유도

Tekton CD는 정형화된 배포 방식이 아니라, 배포 자체를 Task로 정의한다.

- Helm 배포
- kubectl apply
- Canary/Blue-Green
- DB Migration 포함 배포

모두 컨테이너 Step으로 구성 가능하다.

- Jenkins보다 선언적이고,
- 전통적인 CD 툴보다 자유도가 훨씬 높다.



테크톤 설치

```
helm repo add tektoncd https://tektoncd.github.io/charts
```

```
"tektoncd" has been added to your repositories
```

```
helm repo update
```

```
... (repository 업데이트 로그 출력)
```



테크톤 설치

```
helm install tekton-pipelines tektoncd/tekton-pipelines \
-n tekton-pipelines --create-namespace
```

```
NAME: tekton-pipelines
```

```
LAST DEPLOYED: Wed Mar  9 12:34:56 2025
```

```
NAMESPACE: tekton-pipelines
```

```
STATUS: deployed
```

```
REVISION: 1
```

```
TEST SUITE: None
```



설치 확인

```
kubectl get pods -n tekton-pipelines
```

NAME	READY	STATUS	RESTARTS	AGE
tekton-pipelines-ctrloller-xxxxxxxxxx	1/1	Running	0	2m
tekton-pipelines-webhook-xxxxxxxxxx	1/1	Running	0	2m



확장 테스크 설치

```
tkn hub install task buildah --namespace blog
```

```
tkn hub install task kubernetes-actions --namespace blog
```

```
tkn hub install task git-clone --namespace blog
```

```
tkn hub install task maven --namespace blog
```

```
tkn task list --namespace blog
```

```
kubectl get pod -n tekton-pipelines
```

**DEPRECATED DISABLED
CHART REPO!**



확장 테스크 설치(변경 방식)

```
git clone https://github.com/tektoncd/catalog.git
```

```
cd catalog
```

```
kubectl apply -n blog -f task/git-clone/0.9/git-clone.yaml
```

```
kubectl apply -n blog -f task/buildah/0.9/buildah.yaml
```

```
kubectl apply -n blog -f task/kubernetes-actions/0.2/kubernetes-actions.yaml
```

```
kubectl apply -n blog -f task/maven/0.4/maven.yaml
```



CLUSTER-ROLE 구성

ROLE 구성을 ClusterRole기반으로 구성이 가능하다. 특정 네임스페이스에서만 선언도 가능하다. 전체 클러스터에 선언 시, 다음과 같이 가능하다.

```
cat <<EOF> tekton-clusterrole.yaml
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  namespace: '*'
  name: tkn-action
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["*"]
EOF
kubectl apply -f tekton-clusterrole.yaml
```



TEKTON SERVICE ACCOUNT

테스톤 서비스 계정을 생성한다.

```
cat <<EOF> tekton-sa.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: tkn-action
  namespace: blog-v2
EOF
kubectl apply -f tekton-sa.yaml
```



CLUSTER ROLE BINDING 구성

테크톤 SA 계정과 연결한다.

```
cat <<EOF> 02-tekton-clusterrolebind.yaml
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: rolebind-for-deploy
subjects:
- kind: ServiceAccount
  name: tkn-action
  namespace: blog-v2
```



CLUSTER ROLE BINDING 구성

앞에 내용 계속 이어서...

```
roleRef:
  kind: ClusterRole
  name: tkn-action
  apiGroup: rbac.authorization.k8s.io
EOF
kubectl apply -f 02-tekton-clusterrolebind.yaml
```



Helm Values 수정

Helm에서 선언을 원하는 경우, values.yaml에 선언을 원하는 경우 아래와 같이 작성한다.

```
cat <<EOF> values.yaml
rbac:
  create: true
  roles:
    controller:
      rules:
        - apiGroups: [""]
          resources: ["pods", "services", "endpoints"]
          verbs: ["get", "list", "watch", "create", "update", "patch",
"delete"]
```



Helm Values 수정

앞에 내용 계속 이어서...

```
webhook:  
  rules:  
    - apiGroups: [""]  
      resources: ["configmaps"]  
      verbs: ["get", "list", "watch"]
```

EOF



Values 적용 및 재시작

위의 내용으로 다시 HelmChart에 반영한다. 반영 후 반드시 서비스는 다시 rollout를 수행해야 한다.

```
helm repo add tektoncd https://tektoncd.github.io/charts
helm repo update
helm install tekton-pipelines tektoncd/tekton-pipelines \
-n tekton-pipelines --create-namespace -f values.yaml
kubectl rollout restart deploy/tekton-pipelines-controller -n tekton-
pipelines
```



쿠버네티스 클러스터 모니터링

마지막

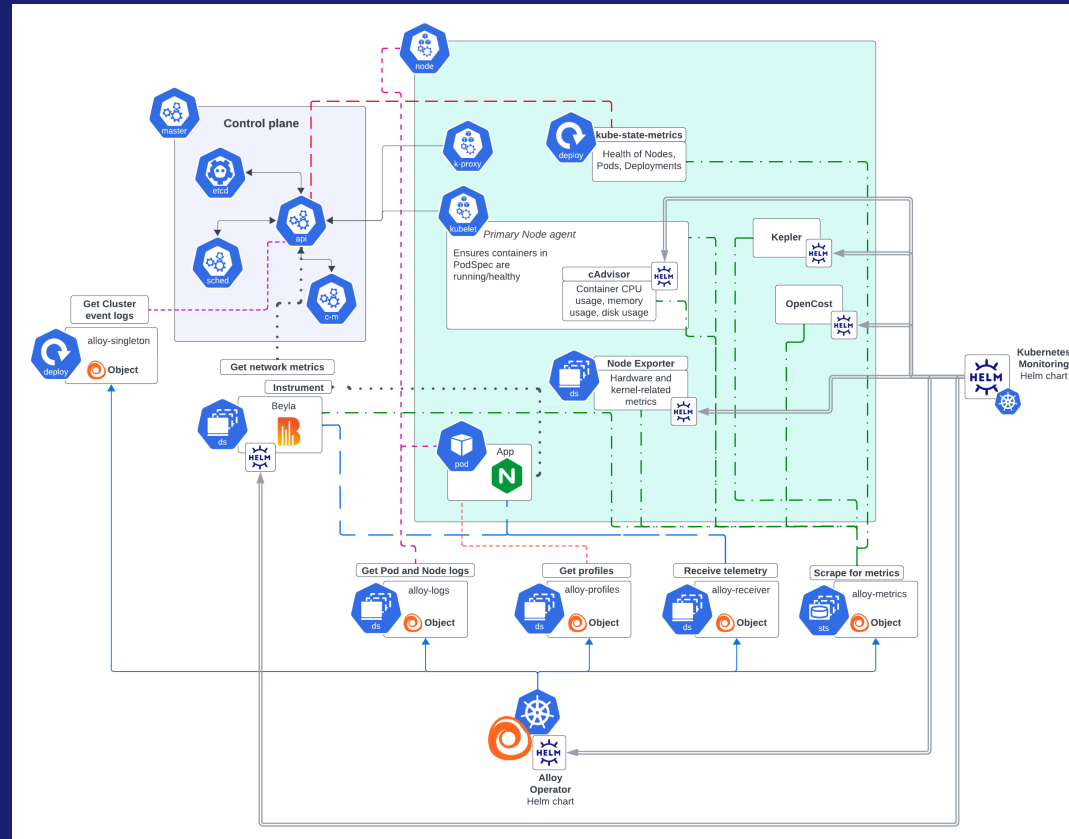


Prometheus

모니터링



Prometheus 설명



Prometheus란 무엇인가

Prometheus는 클라우드 네이티브 환경을 위해 설계된 **메트릭 기반 모니터링 시스템**이다.

시스템과 애플리케이션에서 발생하는 다양한 상태 정보를 **시간 기반 데이터(Time Series)** 형태로 수집하고 저장하며, 이를 통해 인프라와 서비스의 현재 상태를 정량적으로 관찰할 수 있다.

Prometheus는 Pull 방식으로 동작하며, 대상 시스템(Node, Pod, Application 등)에 직접 접근하여 메트릭을 주기적으로 수집한다.

이 구조는 Kubernetes 환경과 매우 잘 어울리며, 동적으로 생성·삭제되는 리소스도 안정적으로 감시할 수 있도록 설계되어 있다.

또한 PromQL이라는 자체 질의 언어를 제공하여, 수집된 메트릭을 기반으로 다양한 조건 검색과 분석이 가능하다.



Prometheus의 역할과 특징

Prometheus의 가장 핵심적인 역할은 "현재 시스템이 정상 상태인가?"를 판단할 수 있는 기준 데이터를 제공하는 것이다.

CPU 사용률, 메모리 사용량, 네트워크 트래픽, Pod 상태와 같은 지표를 지속적으로 수집함으로써 장애 발생 전후의 상태를 비교할 수 있다.

Prometheus 자체는 시각화나 사용자 인증 기능보다는 **메트릭 수집과 저장에 집중**하며, 실제 운영 환경에서는 Grafana와 같은 시각화 도구 또는 Alertmanager와 같은 알림 시스템과 함께 사용된다.

이러한 구조로 인해 Prometheus는 **모니터링의 데이터 소스(Source of Truth)** 역할을 담당한다.



Prometheus는 누가 만들었나?

Prometheus는 2012년, 음악 스트리밍 기업인 SoundCloud의 엔지니어 팀에서 처음 개발되었습니다. 대규모 분산 시스템을 운영하면서 기존 모니터링 도구의 한계를 느낀 것이 출발점이었습니다.

주요 창시자

- Julius Volz
- Matt Proud

이 두 사람이 초기 설계와 핵심 개념을 주도했고, Google의 내부 모니터링 시스템(Borgmon)에서 많은 영향을 받음.



설치

프로메테우스 설치는 두 가지 방법이 있다.

1. 오퍼레이터
2. 헬름차트

폐쇄망 및 직접 운영 및 배포를 하는 엔지니어 경우에는 헬름차트 기반으로 구성을 권장한다. 그게 아닌, 서비스 용도로 사용하는 경우 오퍼레이터 기반으로 구성을 권장한다.

```
kubectl create namespace kube-monitoring
helm repo add prometheus-community https://prometheus-
community.github.io/helm-charts
helm install prometheus prometheus-community/kube-prometheus-stack \
-n kube-monitoring
```



노드포트 변경

노드포트로 변경 후 접근을 위해서 아래와 같이 values.yaml 파일을 생성한다.

```
cat <<EOF> prometheus-values.yaml
grafana:
  service:
    type: NodePort
    nodePort: 32000
prometheus:
  service:
    type: NodePort
    nodePort: 32001
```



노드 포트 변경

```
alertmanager:  
  service:  
    type: NodePort  
    nodePort: 32002
```

EOF



노드 포트 변경 반영

노드포트로 변경 후 접근을 위해서 아래와 같이 values.yaml 파일을 생성한다.

```
helm upgrade --install prometheus prometheus-community/kube-prometheus-stack -n kube-monitoring -f prometheus-values.yaml
```

```
kubectl get svc -n kube-monitoring
```

```
kubectl --namespace kube-monitoring get secrets prometheus-grafana -o jsonpath="{.data.admin-password}" | base64 -d ; echo
```



Grafana

설치



Grafana란 무엇인가

Grafana는 Prometheus와 같은 다양한 데이터 소스를 시각화 하는 대시보드 중심의 모니터링 도구이다.

복잡한 수치 데이터와 메트릭을 사람이 이해하기 쉬운 그래프, 차트, 테이블 형태로 표현함으로써 시스템 상태를 직관적으로 파악할 수 있도록 돕는다.

Grafana는 단독으로 데이터를 수집하지 않으며, Prometheus를 비롯한 외부 데이터 소스와 연동하여 동작한다. 사용자는 Grafana를 통해 실시간 대시보드를 구성하고, 특정 시점의 상태를 분석하거나 장기간 추이를 비교할 수 있다.



Grafana의 역할과 Prometheus와의 관계

Grafana의 주요 역할은 모니터링 데이터를 사람이 이해할 수 있게 보여주는 것이다.

Prometheus가 수집한 메트릭을 기반으로 노드 상태, 파드 상태, 클러스터 전체 자원 사용량 등을 한눈에 확인할 수 있다.

또한 Grafana는 사용자 인증, 대시보드 공유, 권한 관리 기능을 제공하므로 운영 환경에서는 Prometheus UI를 직접 노출하지 않고 Grafana만 외부에 공개하는 구조가 일반적이다.

정리하면, Prometheus는 데이터를 수집하고 저장하는 엔진이며, Grafana는 그 데이터를 시각화하고 운영자가 판단할 수 있도록 돕는 도구이다.



Grafana는 누가 만들었나?

Grafana는 2014년, 스웨덴의 소프트웨어 엔지니어 Torkel Ödegaard가 처음 만들었습니다.

당시 그는 기존 모니터링 도구들의 시각화 기능이 부족하다고 느꼈고, 특히 Graphite 데이터를 더 직관적으로 보고 싶은 필요에서 Grafana를 개발했습니다.

1. Grafana는 오픈소스(OSS)로 유지
2. Grafana Labs는 엔터프라이즈 기능과 클라우드 서비스를 제공
3. 전 세계 수많은 기업과 쿠버네티스 환경에서 표준 도구로 사용 중



Grafana 설치

프로메테우스에서 수집한 데이터를 표시하기 위해서 그라파나를 통해서 화면에 표현 및 출력한다.

```
helm repo add grafana https://grafana.github.io/helm-charts
helm install grafana grafana/grafana --namespace kube-monitoring
```

```
kubectl get secret --namespace monitoring grafana -o
jsonpath="{.data.admin-password}" | base64 --decode ; echo
```

```
kubectl expose service grafana --namespace monitoring --type=NodePort --
target-port=3000 --name=grafana-ext
```



Grafana values 파일 수정

노드포트 접근 및 아이디/비밀번호를 사용하기 편하게 변경한다.

```
cat <<EOF > grafana-values.yaml
```

```
service:
```

```
  type: NodePort
```

```
  nodePort: 32000
```

```
adminUser: admin
```

```
adminPassword: admin123
```

```
persistence:
```

```
  enabled: false
```

```
EOF
```



계
속

Grafana values 파일 수정 반영

```
helm upgrade --install grafana grafana/grafana \
  -n kube-monitoring -f grafana-values.yaml
```



Grafana 접근 및 확인

올바르게 동작하는지 확인하기 위해서 그라파나 관리자 사이트에 접근한다.

```
kubectl get svc -n kube-monitoring
```

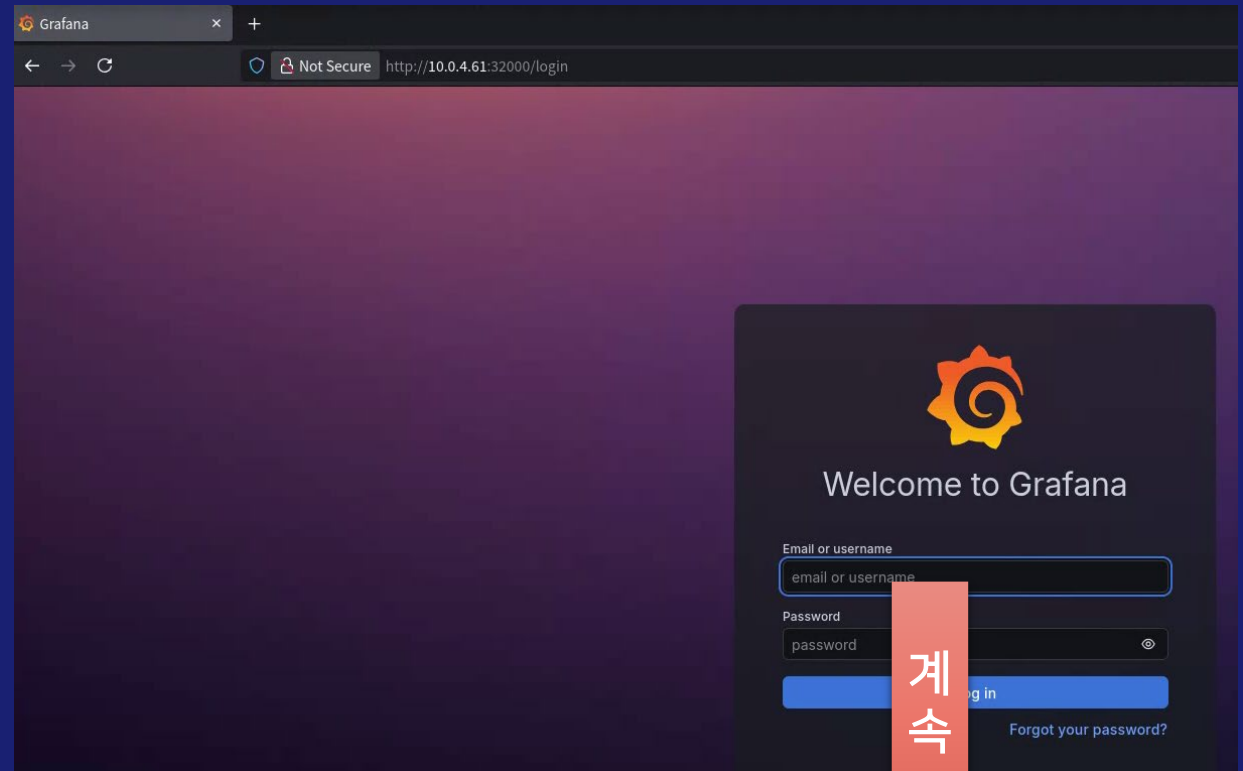
NAME	EXTERNAL-IP	PORT(S)	TYPE	AGE	CLUSTER-IP
grafana			NodePort		10.111.58.32
<none>		80:32000/TCP		3m47s	
prometheus-kube-prometheus-prometheus			NodePort		10.108.70.102
<none>		9090:32001/TCP,8080:30662/TCP		9m22s	



페이지 접근 및 확인

32000번 포트로 접근, 현재 HTTPS는 노드 포트로 열어두지 않았기 때문에 HTTP로 접근한다. 접근 시, rustdesk로 리눅스 클라이언트로 접근한다.

- 아이디: admin
- 비밀번호: admin123



프로메테우스와 연결

현재 그라파나는 프로메테우스와 연결이 필요하다. 아래와 같이 프로메테우스 주소 확인 후, 연결을 구성한다.

```
kubectl get svc -n kube-monitoring
```

```
prometheus-kube-prometheus-prometheus      NodePort      10.108.70.102  
<none>          9090:32001/TCP,8080:30662/TCP      9m22s  
prometheus-kube-prometheus-prometheus.kube-  
monitoring.svc.cluster.local:9090
```

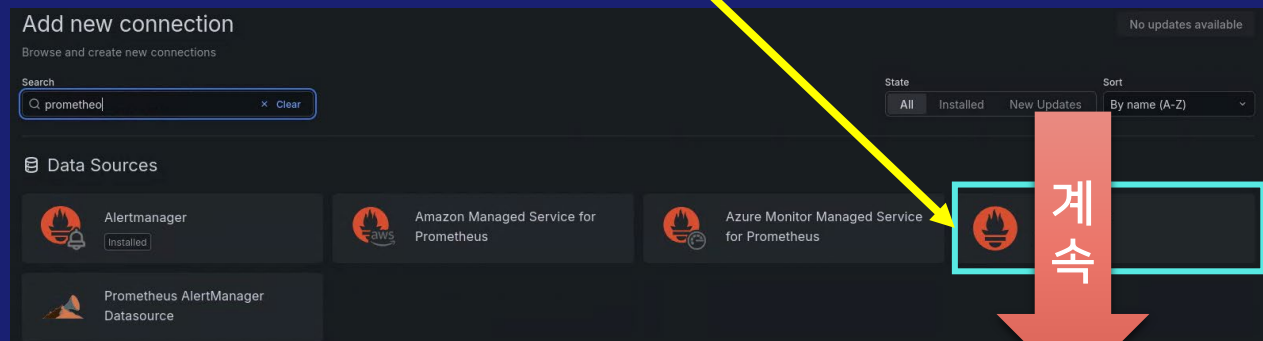
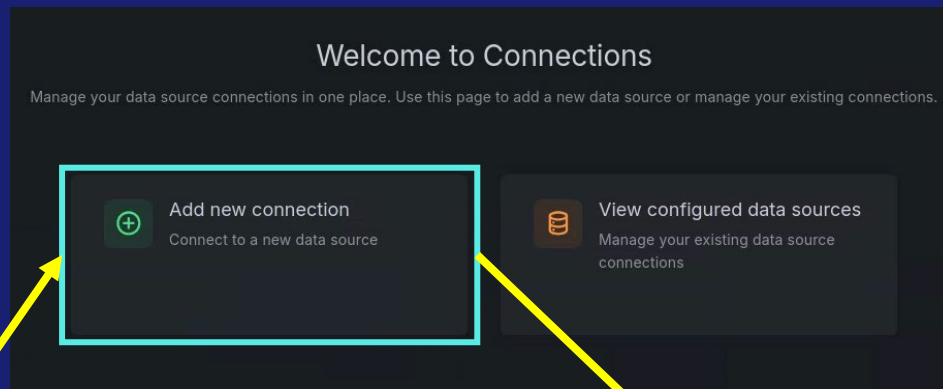
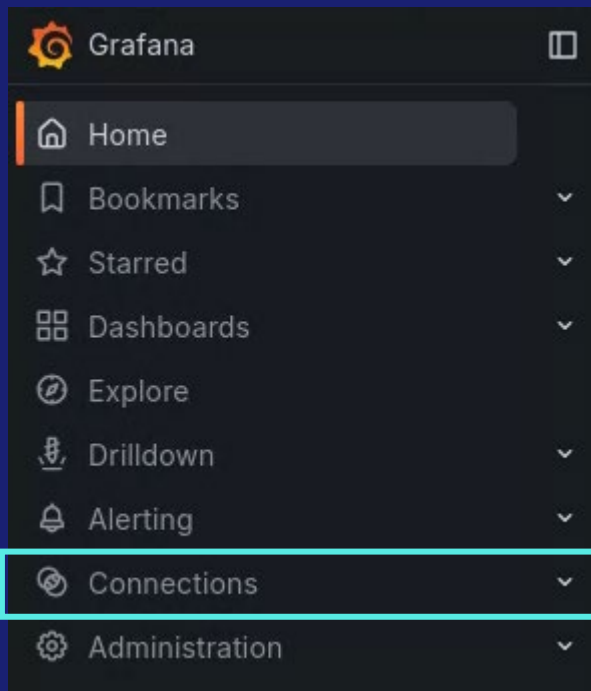
```
kubectl --namespace kube-monitoring get secrets prometheus-grafana -o  
jsonpath="{.data.admin-password}" | base64 -d ; echo
```

계
속



프로메테우스 데이터 스토어 연결 및 구성

현재 프로메테우스와 데이터 연결이 없기 때문에 Connections에서 연결을 구성한다.



프로메테우스 데이터 스토어 연결 및 구성

 Prometheus
Open source time series database & alerting

[Overview](#)

Prometheus Data Source - Native Plugin

Grafana ships with built in support for Prometheus, the open-source service monitoring system and time series database.

Read more about it here:
<http://docs.grafana.org/datasources/prometheus/>

Dashboards

Explore

Drilldown

Alerting

Connections

Add new connection

Data sources

Administration

Add new data source

From:
Grafana Labs

Connection

Prometheus server URL * [?](#) **http://prometheus-kube-prometheus-prometheus.kube**

Authentication

Authentication methods
Choose an authentication method to access the data source

Authentication method
Basic authentication

User * [?](#) admin

Password * [?](#)

TLS settings

Additional security measures that can be applied on top of authentication

☐ Add self-signed certificate [?](#)

☐ TLS Client Authentication [?](#)

☒ Skip TLS certificate validation [?](#)

✓ Successfully queried the Prometheus API.

Next, you can start to visualize data by building a dashboard , or by querying data in the Explore view .

Delete

Save & test

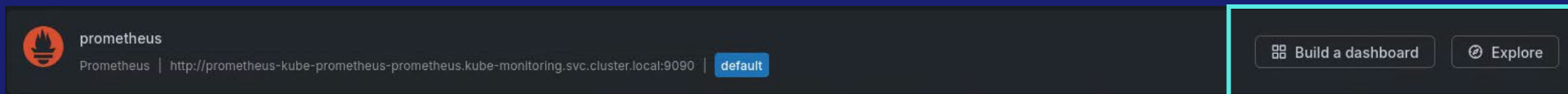
계속

쿠버네티스 기본

2026-03-12

568

프로메테우스 데이터 스토어 연결 및 구성



프로메테우스 데이터 스토어 연결 및 구성

